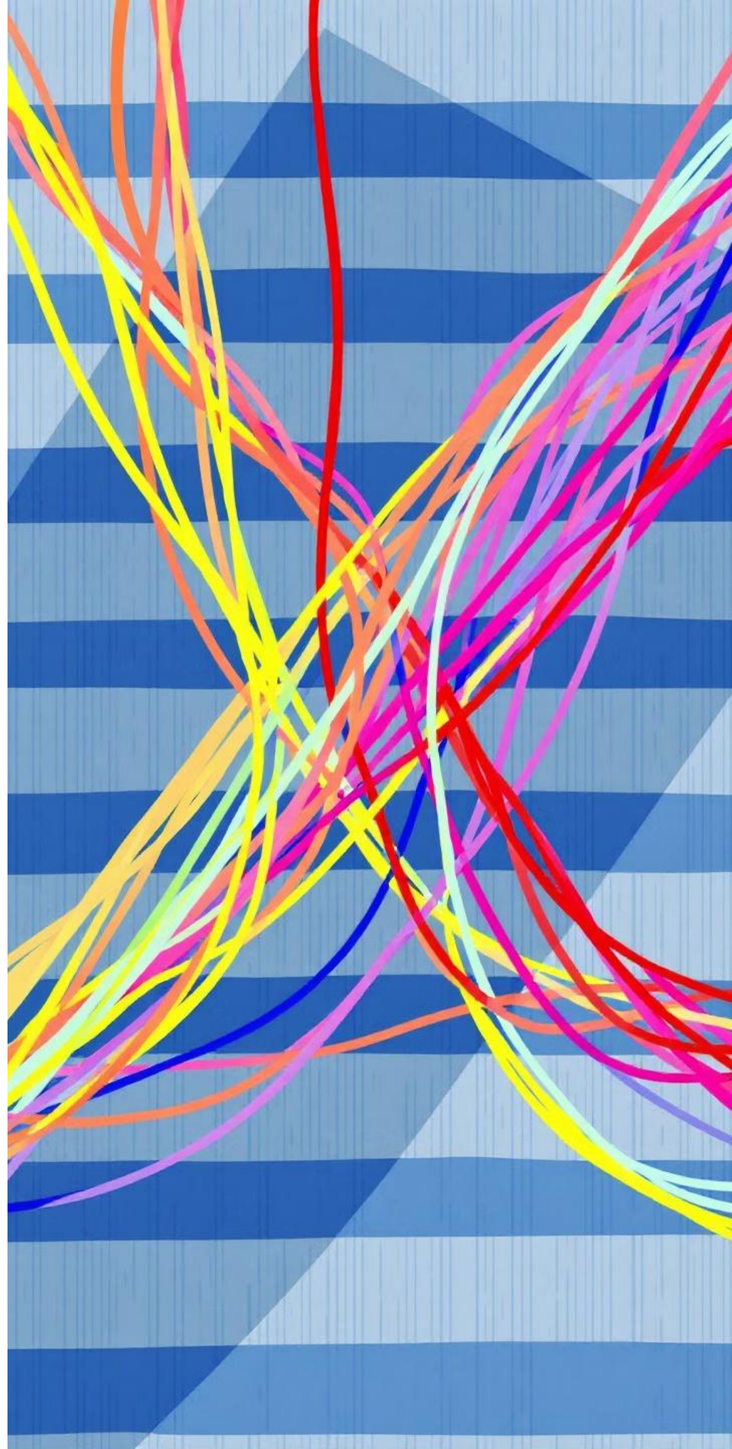
Programación de Servicios y Procesos



UT2. Programación multihilo



RESULTADO DE APRENDIZAJE: RA2	
RA2.- Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje de programación.	
CRITERIOS DE EVALUACIÓN	
a)	Se han identificado situaciones en las que resulte útil la utilización de varios hilos en un programa.
b)	Se han reconocido los mecanismos para crear, iniciar y finalizar hilos.
c)	Se han programado aplicaciones que implementen varios hilos.
d)	Se han identificado los posibles estados de ejecución de un hilo y programado aplicaciones que los gestionen.
e)	Se han utilizado mecanismos para compartir información entre varios hilos de un mismo proceso.
f)	Se han desarrollado programas formados por varios hilos sincronizados mediante técnicas específicas.
g)	Se ha establecido y controlado la prioridad de cada uno de los hilos de ejecución.
h)	Se han depurado y documentado los programas desarrollados.
i)	Se ha analizado el contexto de ejecución de los hilos
j)	Se han analizado librerías específicas del lenguaje de programación que permiten la programación multihilo.
k)	Se han reconocido los problemas derivados de la compartición de información entre los hilos de un mismo proceso



Índice



1 Introducción a la programación multihilo

2 Conceptos fundamentales

2.1 Hilos y procesos

2.2 Recursos compartidos por los hilos

2.3 Ventajas en el uso de hilos

3 Estados de un hilo y su ciclo de vida

3.1 Estados principales de un hilo

3.2 Librerías y clases multihilo en Java

3.3 Creación y gestión de hilos

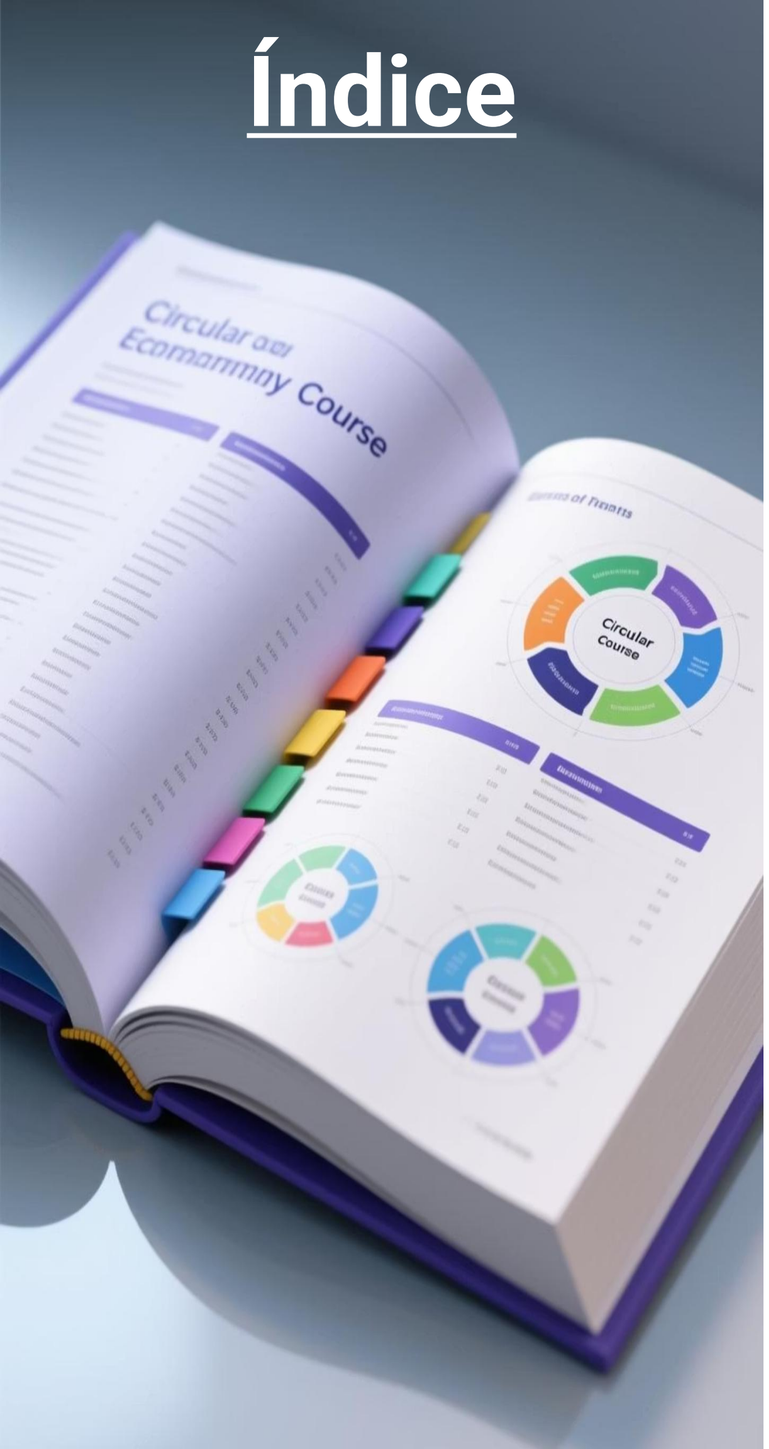
4 Intercambio de información entre hilos

4.1 Intercambio de datos entre hilos

4.2 Variables compartidas

4.3 Canales de comunicación

Índice



5 **Sincronización de hilos**

- 5.1 Problemas comunes de la sincronización de hilos
- 5.2 Mecanismos de sincronización de hilos
- 5.3 Mecanismos de sincronización avanzada con java.util.concurrent
- 5.4 Buenas prácticas y estrategias de sincronización de hilos

6 **Prioridades de los hilos**

- 6.1 Concepto de prioridad en los hilos
- 6.2 Niveles de prioridad
- 6.3 Establecimiento y modificación de prioridades
- 6.4 Efectos de las prioridades durante la ejecución
- 6.5 Buenas prácticas en el uso de prioridades

7 **Bibliografía**

PROGRAMACIÓN MULTIHILO

1 INTRODUCCIÓN A LA PROGRAMACIÓN MULTIHILO

UN PROGRAMA PUEDE HACER VARIAS TAREAS CONCURRENTEMENTE

PROGRAMA

DIVIDE EL TRABAJO

VARIOS **HILOS** EJECUTAN TAREAS EN PARALELO

UN ÚNICO FLUJO DE EJECUCIÓN

MISMO PROCESO

2 CONCEPTOS FUNDAMENTALES

2.1. HILOS Y PROCESOS

PROCESO

HILO 1 HILO 2 HILO 3 HILO 4

- EL PROCESO ES EL CONTENEDOR
- LOS HILOS SON FLUJOS DE EJECUCIÓN DENTRO DEL MISMO PROCESO

2.2. RECURSOS COMPARTIDOS POR LOS HILOS

MEMORIA VARIABLES ARCHIVOS OBJETOS / DATOS

RECURSOS COMPARTIDOS

2.3. VENTAJAS EN EL USO DE HILOS

- MAYOR CAPACIDAD DE RESPUESTA
- TAREAS SIMULTÁNEAS
- MEJOR APROVECHAMIENTO DE RECURSOS
- DIVISIÓN DEL TRABAJO

3 ESTADOS DE UN HILO Y SU CICLO DE VIDA

3.1. ESTADOS PRINCIPALES DE UN HILO

CREACIÓN → PREPARADO → EJECUCIÓN ↔ ESPERA O BLOQUEO → FINALIZACIÓN

3.2. LIBRERÍAS Y CLASES MULTIHILO EN JAVA

java.lang

java.util.concurrent

- CLASES PARA CREACIÓN, GESTIÓN Y SINCRONIZACIÓN DE HILOS

3.3. CREACIÓN Y GESTIÓN DE HILOS

CREAR → INICIAR → EJECUTAR → ESPERA → INTERRUMPIR → FINALIZAR

VARIOS HILOS
UNA MISMA APLICACIÓN
MÁS POSIBILIDADES

CONCURRENCIA
COORDINACIÓN
PROGRAMAS MÁS EFICIENTES

PROGRAMA

PROCESO

HILO 1
HILO 2
HILO 3
HILO 4

RECURSOS COMPARTIDOS

- TAREAS COMPLETADAS
- MEJOR RENDIMIENTO
- APLICACIONES MÁS RESPONSIVAS

4 INTERCAMBIO DE INFORMACIÓN ENTRE HILOS

4.1. INTERCAMBIO DE DATOS ENTRE HILOS

4.2. VARIABLES COMPARTIDAS

VARIABLE COMPARTIDA

4.3. CANALES DE COMUNICACIÓN

- LOS HILOS PUEDEN INTERCAMBIAR DATOS
- PUEDEN COMPARTIR VARIABLES
- EXISTEN DISTINTOS CANALES DE COMUNICACIÓN

5 SINCRONIZACIÓN DE HILOS

5.1. PROBLEMAS COMUNES DE LA SINCRONIZACIÓN DE HILOS

- ACCESOS SIMULTÁNEOS
- RESULTADOS INCONSISTENTES
- CONDICIONES DE CARRERA
- BLOQUEOS ENTRE TAREAS
- ESPERAS INDEFINIDAS

5.2. MECANISMOS DE SINCRONIZACIÓN DE HILOS

ACCESO CONTROLADO AL RECURSO

5.3. SINCRONIZACIÓN AVANZADA CON java.util.concurrent

- HERRAMIENTAS DE COORDINACIÓN
- COLAS Y SINCRONIZADORES
- GESTIÓN AVANZADA DE TAREAS

5.4. BUENAS PRÁCTICAS Y ESTRATEGIAS DE SINCRONIZACIÓN DE HILOS

- MINIMIZAR RECURSOS COMPARTIDOS
- ACCESOS BREVES
- COORDINAR CORRECTAMENTE
- EVITAR BLOQUEOS INNECESARIOS
- DISEÑO SENCILLO

6 PRIORIDADES DE LOS HILOS

6.1. CONCEPTO DE PRIORIDAD EN LOS HILOS

CPU

LOS HILOS COMPITEN POR TIEMPO DE EJECUCIÓN

6.2. NIVELES DE PRIORIDAD

BAJA MEDIA ALTA

6.3. ESTABLECIMIENTO Y MODIFICACIÓN DE PRIORIDADES

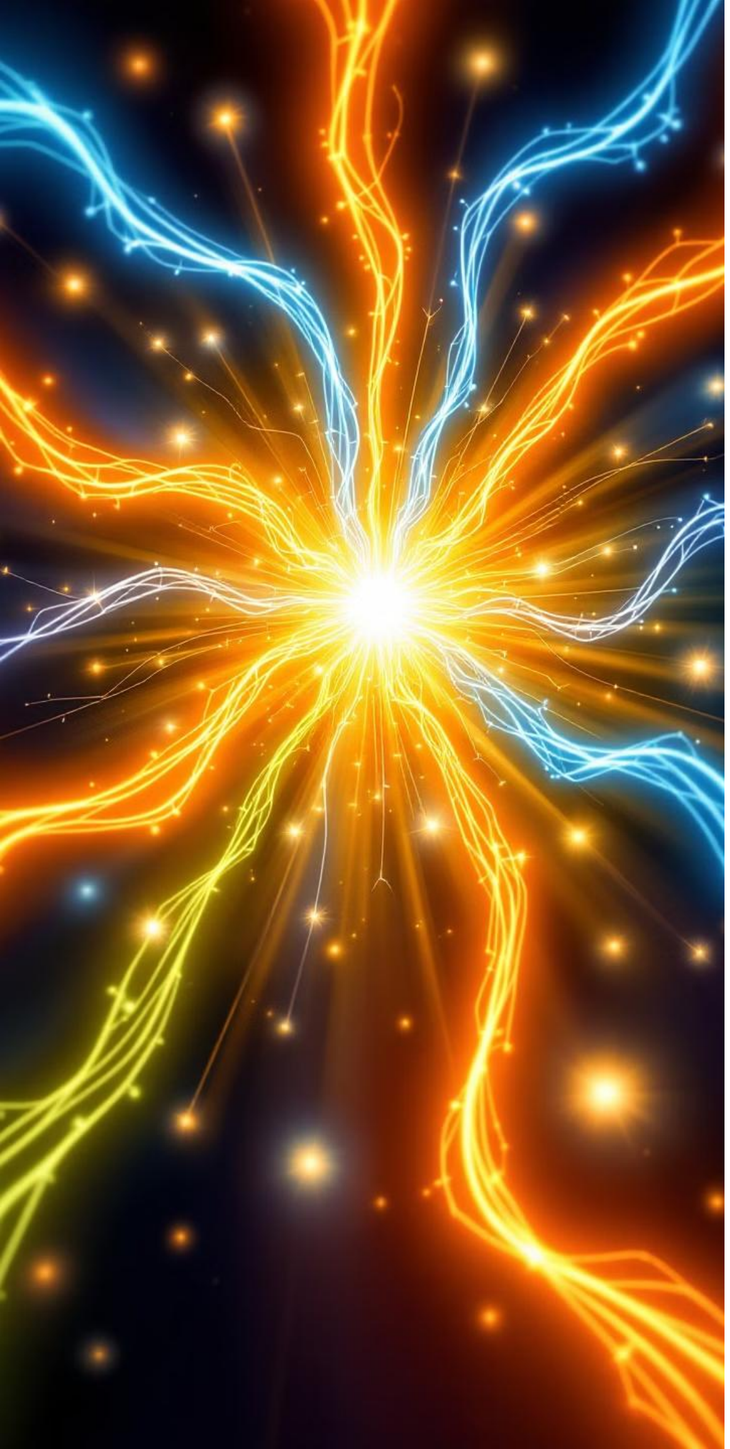
6.4. EFECTOS DE LAS PRIORIDADES DURANTE LA EJECUCIÓN

COLA DE TAREAS → CPU

LA PRIORIDAD INFLUYE EN LA PLANIFICACIÓN PERO NO GARANTIZA LA EJECUCIÓN INMEDIATA

6.5. BUENAS PRÁCTICAS EN EL USO DE PRIORIDADES

- USO MODERADO
- EVITAR ABUSOS
- ANALIZAR EL IMPACTO
- COMBINAR CON OTRAS TÉCNICAS
- PRUEBAS EN ESCENARIOS REALES



1. Introducción a la programación multihilo

OBJETIVOS DE LA UT

Conceptos fundamentales

Conocer y comprender los principios básicos de los hilos de ejecución y cómo funcionan.

Creación y gestión de hilos

Aprenderemos los mecanismos para crear y administrar múltiples hilos en una aplicación.

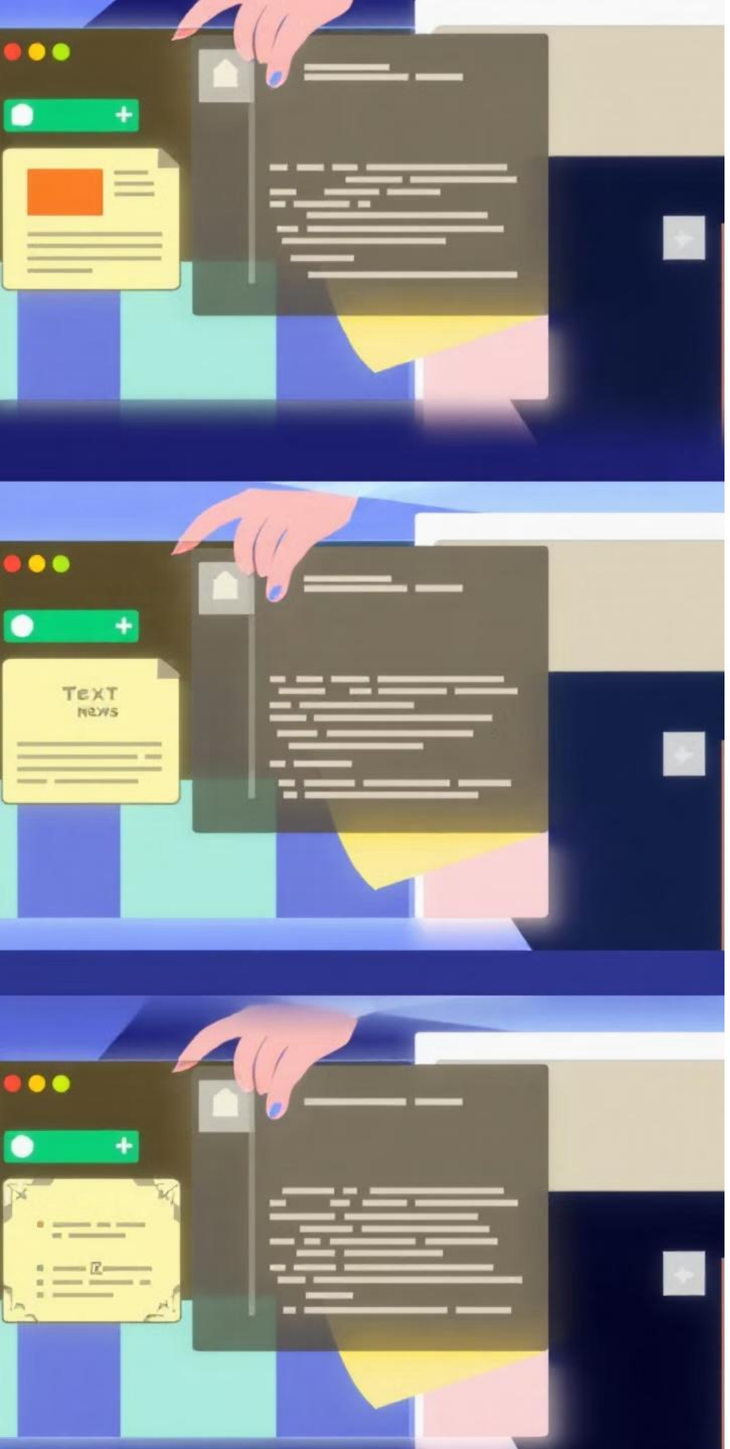
Sincronización y comunicación

Estudiaremos las técnicas de sincronización y comunicación entre los hilos.

Beneficios y desafíos

Identificaremos las ventajas y los retos de utilizar la programación multihilo.

Enlace a video introductorio



Ejemplos de la programación multihilo

Navegador web

Permite descargar imágenes y navegar simultáneamente sin interrupciones.



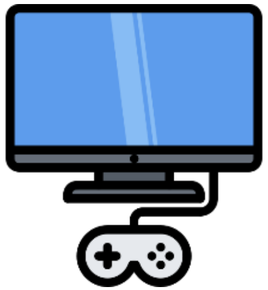
Editor de texto

Revisa la ortografía y gramática en tiempo real mientras se guarda el documento.



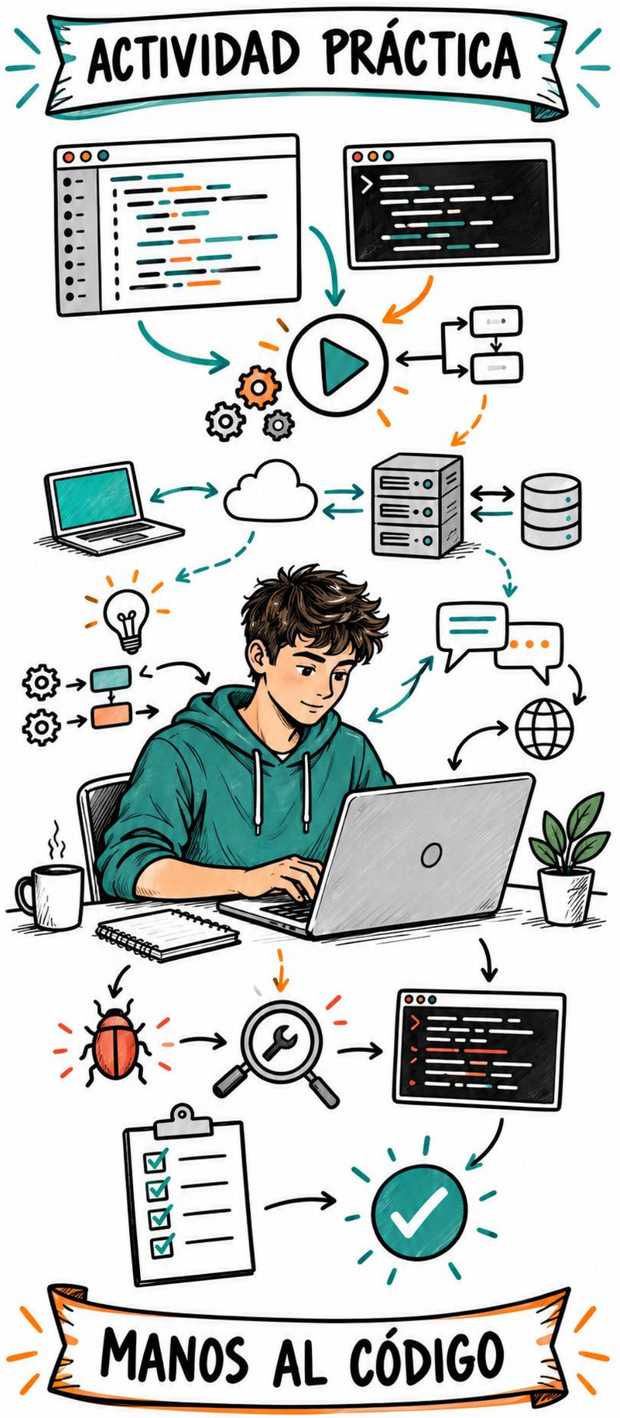
Videojuegos

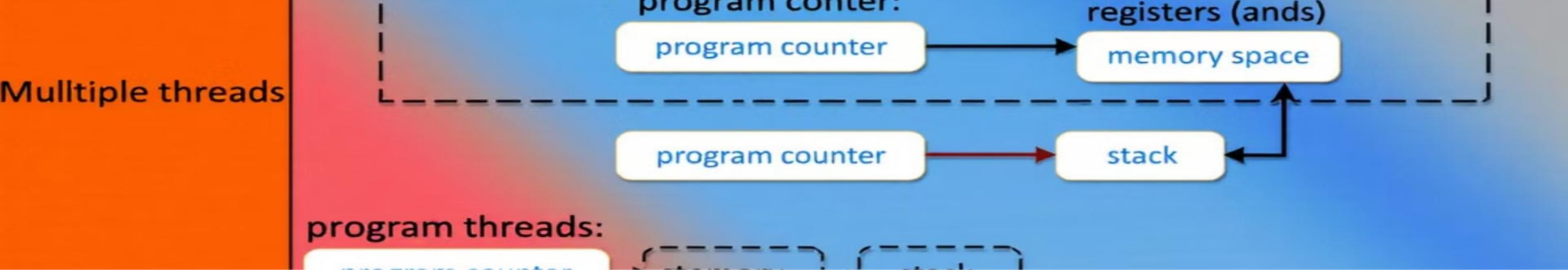
Gestiona de manera eficiente tareas como renderizado, IA y detección de colisiones.



ACTIVIDAD 1

Identificar situaciones donde resulte útil el uso de hilos





Desafíos de la programación multihilo

Condición de carrera

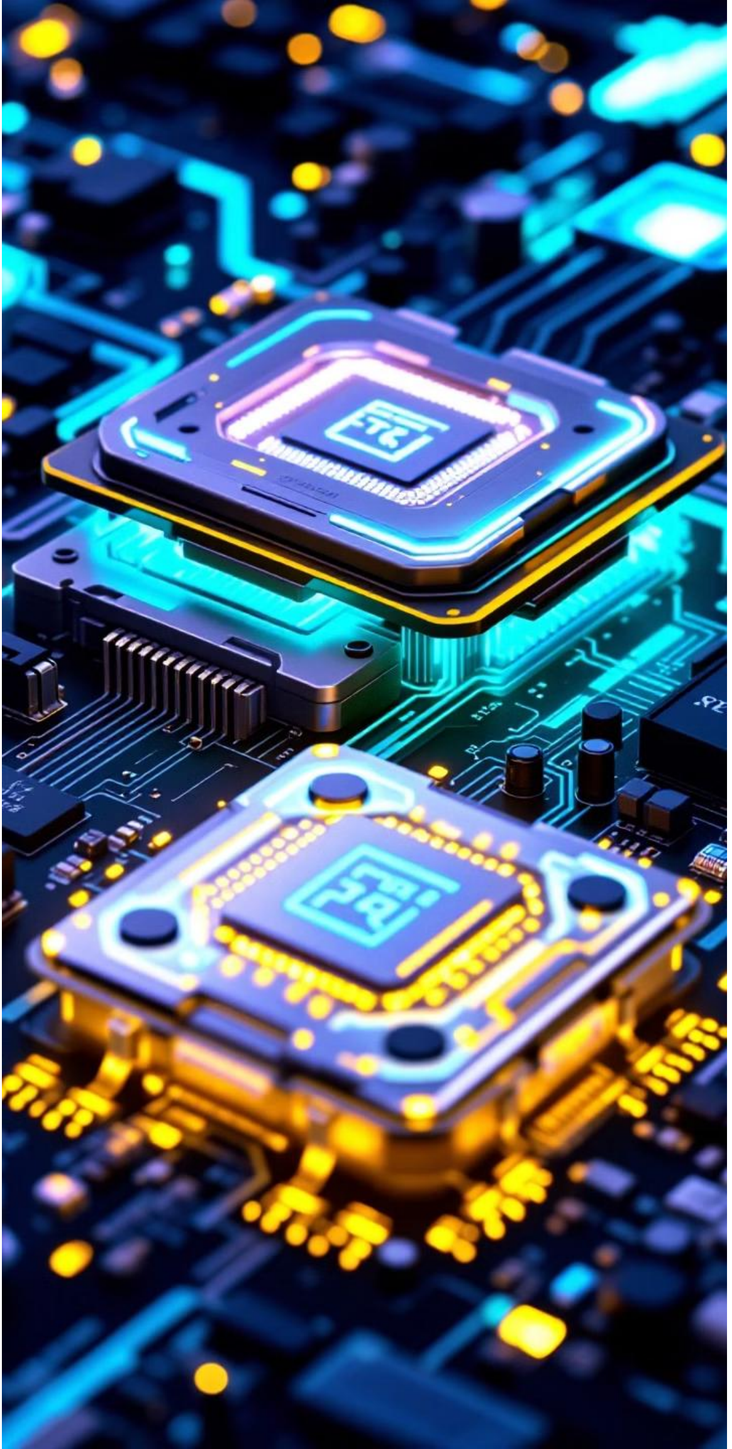
Cuando varios hilos acceden a la vez a datos compartidos, el resultado depende del orden en que se intercalan, lo que provoca errores difíciles de reproducir.

Interbloqueo y sección crítica

La ejecución de hilos debe controlarse cuidadosamente para evitar conflictos o bloqueos que paralicen el programa.
Dos o más hilos se pueden quedar esperando indefinidamente un recurso que tiene otro de ellos, y el programa se paraliza.

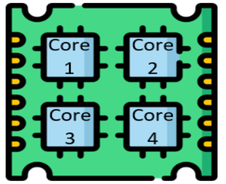
Mecanismos de sincronización

Objetos de bloqueo y métodos sincronizados ayudan a regular el acceso simultáneo a recursos compartidos.



Contexto actual de la programación multihilo

Tendencia en procesadores



Enfoque en añadir más núcleos en lugar de aumentar la velocidad de cada núcleo.

Consumo de energía

Los procesadores menos potentes consumen menos energía. Subir la frecuencia de un núcleo dispara el consumo y el calor. Varios núcleos a menor frecuencia dan más rendimiento con menos energía.

Paralelismo y concurrencia

Más núcleos permiten manejar más hilos en paralelo, aumentando las posibilidades de realizar tareas concurrentes.
Un programa con un solo hilo usa un único núcleo, aunque el equipo tenga ocho. Para aprovechar el resto, hay que programar con varios hilos.

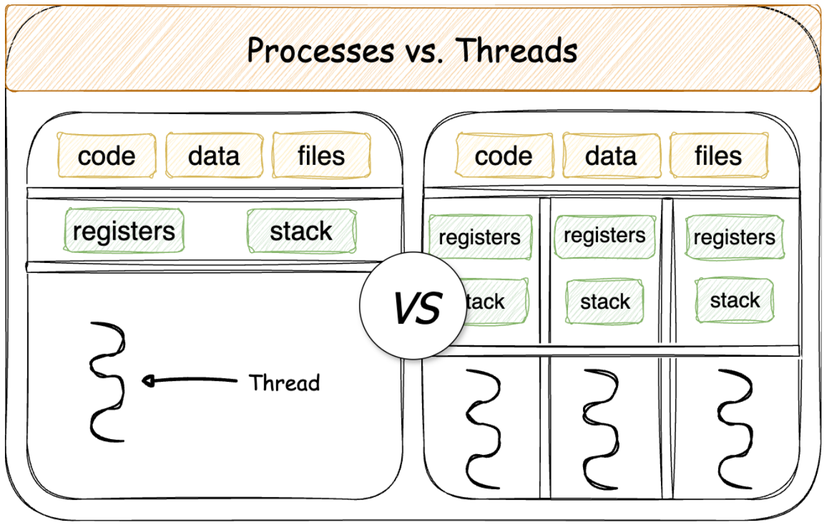
2. Conceptos fundamentales

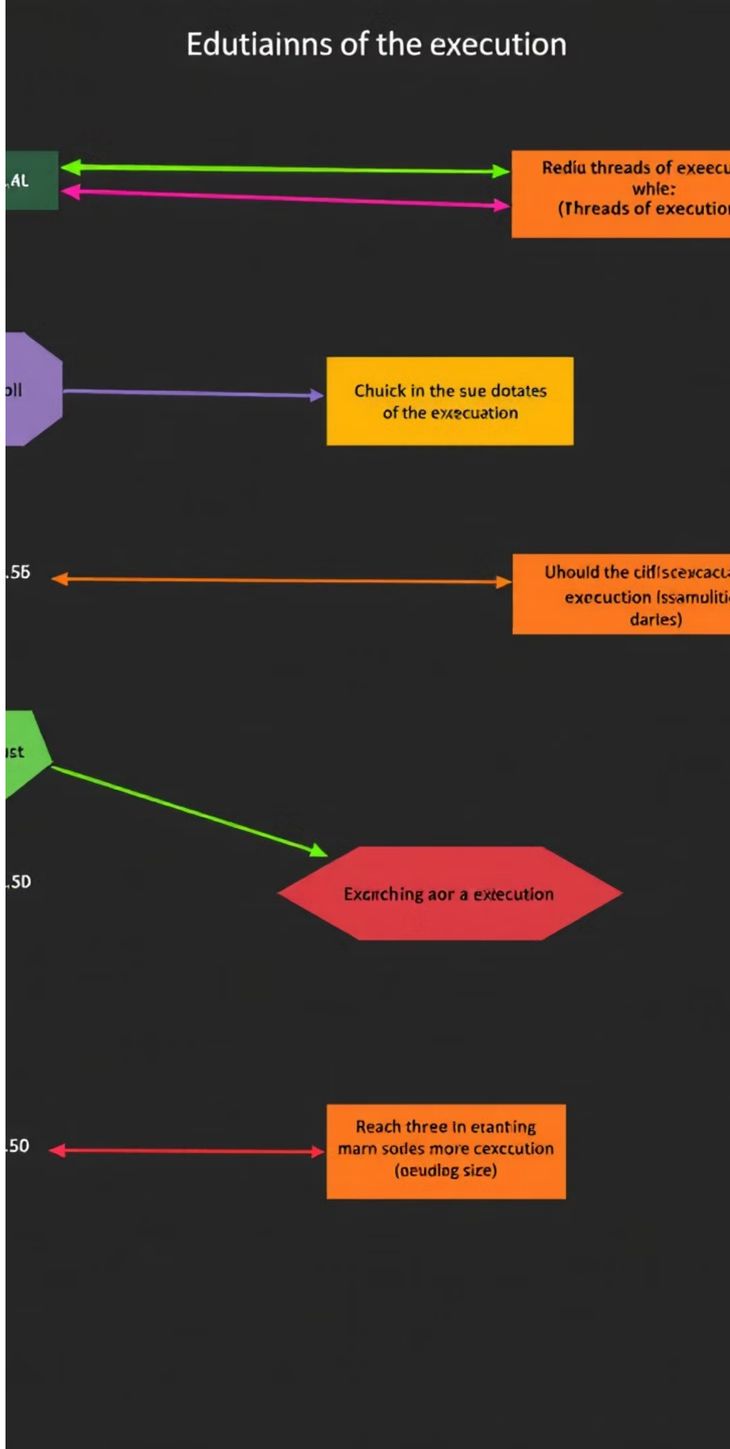
2.1. Hilos y procesos

- Un hilo (thread) es la **unidad básica de ejecución** que el sistema operativo planifica en la CPU.

- Un hilo **no está ligado a un núcleo**: el sistema operativo puede ejecutarlo en cualquiera y moverlo entre ellos.

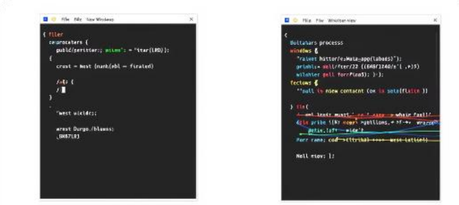
- Todo hilo **pertenece a un proceso y se ejecuta dentro de su contexto**. Un proceso tiene como mínimo un hilo (en Java, el hilo main) y puede crear más. El sistema operativo gestiona tanto los procesos como sus hilos.





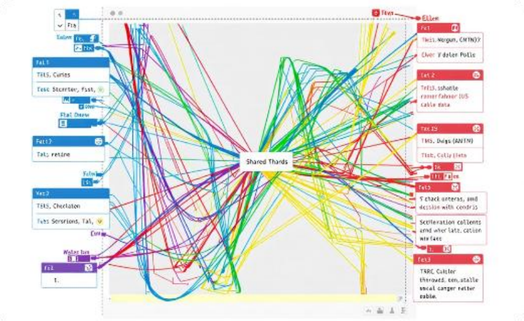
2.1. Hilos y procesos

Hilos vs Procesos



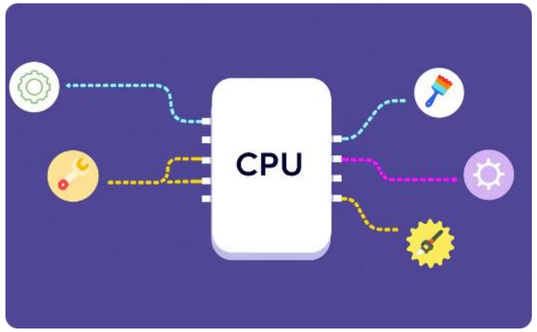
Procesos independientes

Los procesos son **entidades independientes** que cuentan con sus propios espacios de memoria.



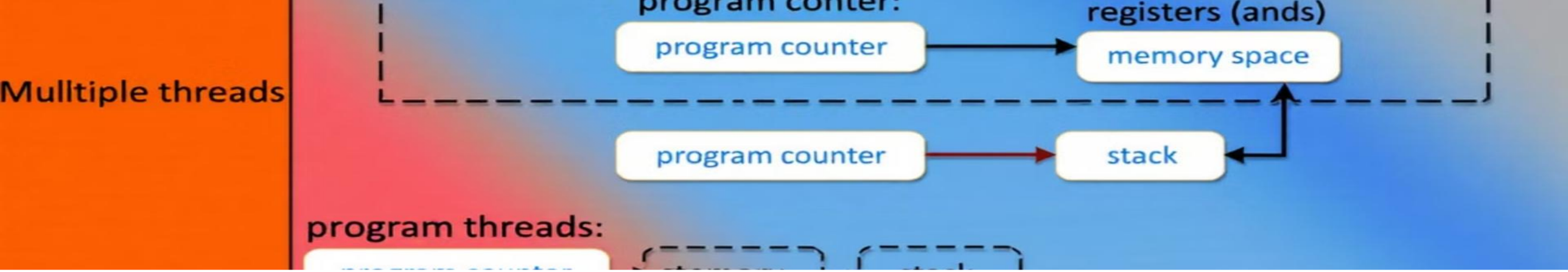
Hilos compartiendo recursos

Dentro de un mismo proceso, los hilos comparten la memoria y los recursos del proceso.



Ejecución concurrente

Los hilos permiten que un proceso avance en varias tareas a la vez: alternándose en un mismo núcleo (concurrencia) o simultáneamente en núcleos distintos (paralelismo).



2.2. Recursos NO compartidos por los hilos

Identificador único

Cada hilo tiene un identificador único que lo distingue dentro del proceso.

Contador de programa

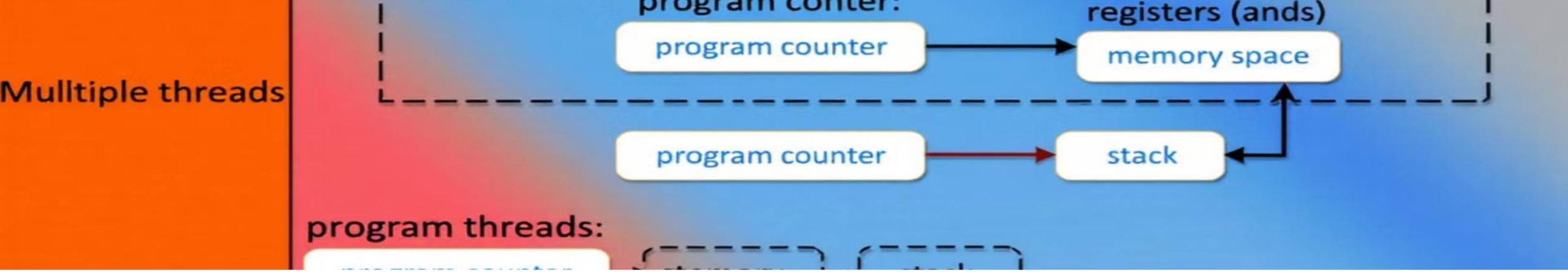
Cada hilo mantiene su propio contador de programa para controlar la ejecución.

Conjunto de registros

Los hilos disponen de su propio conjunto de registros para el procesamiento.

Pila de ejecución

Cada hilo tiene su propia pila para gestionar variables locales y controlar la ejecución. Sus variables locales y sus llamadas a métodos no son visibles para los demás hilos.



2.2. Recursos SI compartidos por los hilos



Código compartido

Los hilos de un mismo proceso comparten el código, lo que les permite acceder a las mismas secciones del programa.



Datos compartidos

Los hilos también comparten datos, lo que facilita la realización de tareas en paralelo. En concreto, comparten los objetos de la memoria del proceso (atributos y variables static). Así pueden intercambiar información directamente, sin los mecanismos que necesitan los procesos.



Recursos del sistema

Otros recursos del sistema operativo, como los archivos abiertos o las conexiones de red, también se comparten entre los hilos.

2.2. Recursos SI compartidos por los hilos



Riesgos de compartir

Aunque la compartición de recursos es útil, también supone riesgos, ya que un hilo puede modificar datos compartidos mientras otro hilo los está usando, afectando a los demás hilos.

1 Bloqueo de recursos

Los hilos deben utilizar mecanismos de bloqueo para controlar el acceso a los recursos compartidos y evitar problemas.

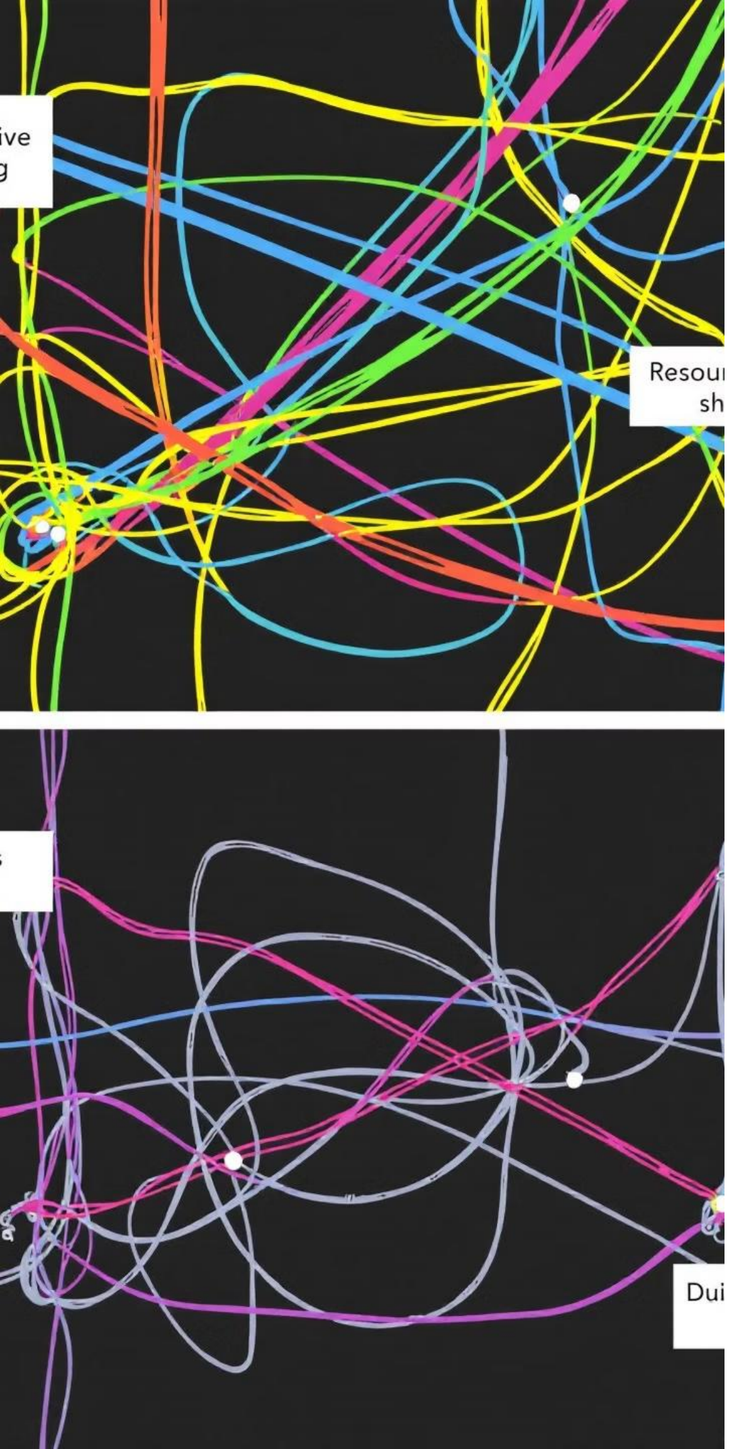
2 Comunicación entre hilos

Cuando un hilo necesita un dato que produce otro, ambos deben coordinarse para no leer un dato incompleto o antiguo.

3 Técnicas de gestión

Se profundizará en las técnicas necesarias para gestionar de forma eficiente los recursos compartidos en Java (apartados 4 y 5).





2.3. Ventajas en el uso de hilos

Capacidad de respuesta

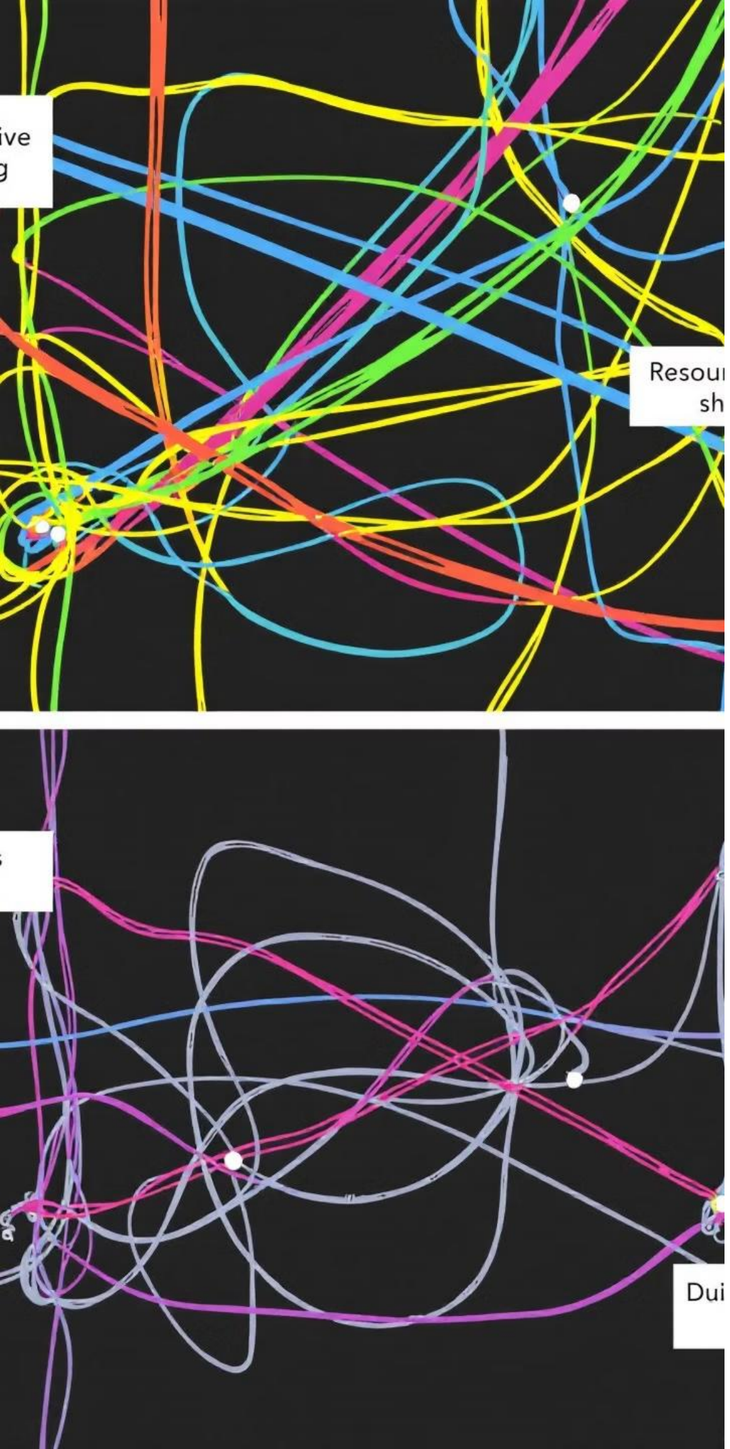
Los hilos permiten que un proceso continúe respondiendo a las solicitudes del usuario incluso cuando una tarea es prolongada.

Compartición de recursos

Los hilos de un mismo proceso comparten la memoria y los recursos, facilitando la comunicación entre ellos.

Mejor uso de la CPU

Mientras un hilo espera (disco, red o usuario), otros hilos del mismo proceso pueden seguir usando la CPU.



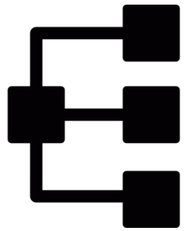
2.3. Ventajas en el uso de hilos

Eficiencia en el uso de memoria



Los hilos comparten la memoria del proceso al que pertenecen, lo que hace que su creación sea más económica en términos de uso de memoria y recursos en comparación con la creación de procesos independientes.
El cambio de contexto entre hilos también es más rápido que entre procesos.

Paralelismo real



En un procesador multinúcleo, varios hilos del mismo proceso se ejecutan a la vez en núcleos distintos. La mejora real depende de que la tarea pueda dividirse y de cuánto tengan que sincronizarse los hilos.

¿Cuándo utilizar hilos en una aplicación?



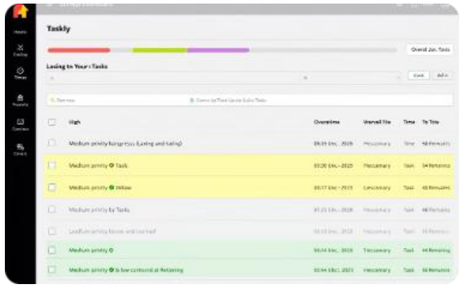
Manejo de entradas/salidas de varios dispositivos

Cuando la aplicación debe atender varias fuentes (usuario, red, disco) sin quedarse bloqueada esperando a una de ellas.



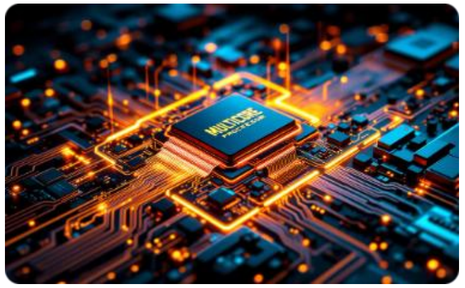
Realización de tareas en paralelo

Cuando la aplicación debe poder realizar diferentes tareas a la vez para mejorar la eficiencia.



Priorización de tareas

Cuando hay tareas críticas (como atender al usuario) que no deben quedar retrasadas por otras tareas de fondo.

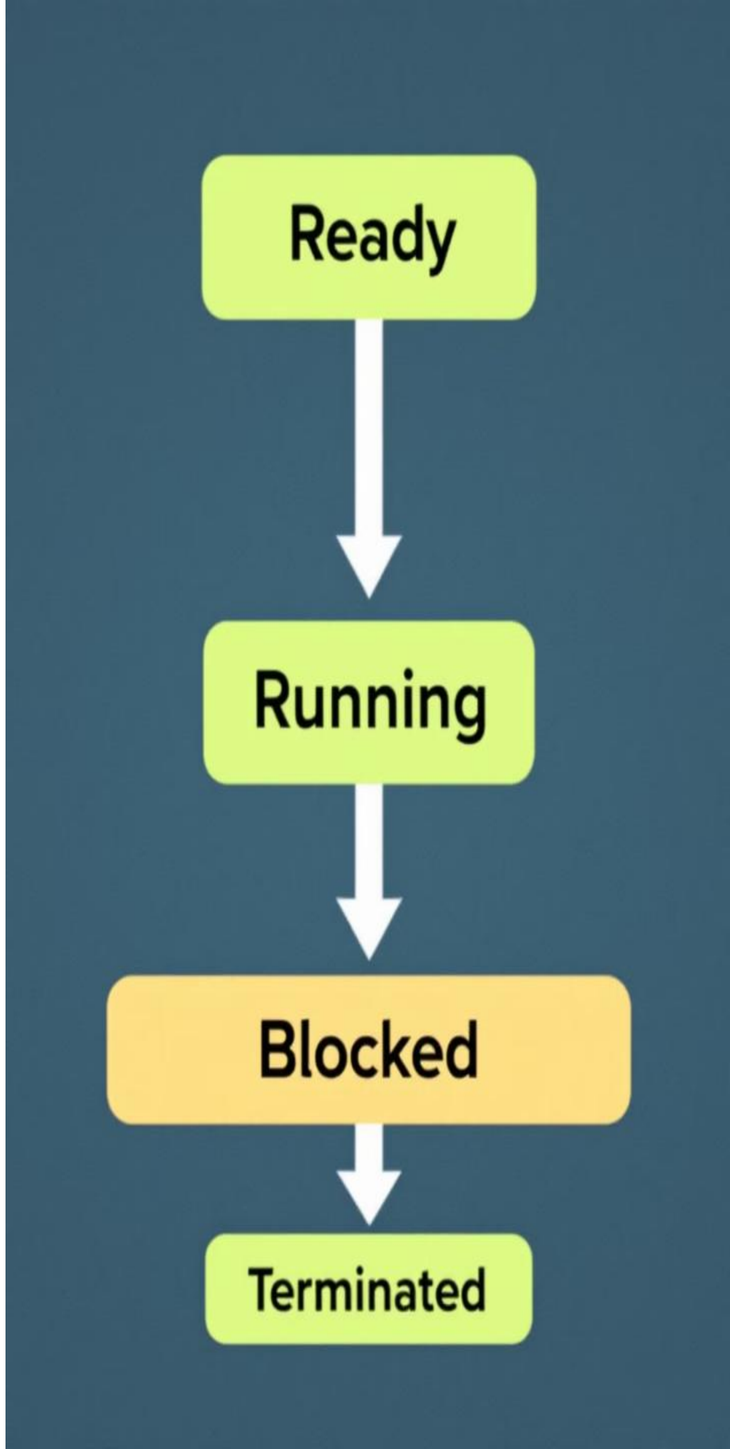


Entorno multiprocesador

Si la aplicación se ejecuta en un equipo con varios núcleos (hoy, prácticamente cualquiera) y la tarea se puede repartir entre ellos.

3. Estados de un hilo y su ciclo de vida

El ciclo de vida de un hilo incluye varios estados por los que pasa desde su creación hasta su finalización.
El hilo cambia de estado dependiendo de las condiciones del sistema y de la lógica de programación.

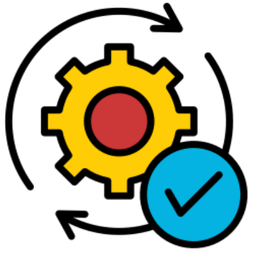


3.1. Estados principales de un Hilo



Nuevo

El hilo se ha creado (objeto Thread), pero aún no ha comenzado a ejecutarse (no se ha llamado a start()).



Ejecutable o en ejecución

Se ha llamado a start(). El hilo se está ejecutando o está listo, esperando a que el sistema operativo le asigne un núcleo.



No Ejecutable o Bloqueado

El hilo no puede ejecutarse temporalmente debido a una condición. Espera un bloqueo (synchronized), un aviso de otro hilo (wait(), join()) o a que pase un tiempo (sleep())



Terminado o Muerto

El hilo ha finalizado su ejecución y ya no puede volver a ejecutarse.

Transición entre estados

Estado nuevo

El hilo ha sido creado, pero no ha comenzado a ejecutarse. El hilo permanece en este estado hasta que se llama a su método start().

Estado ejecutable o en ejecución

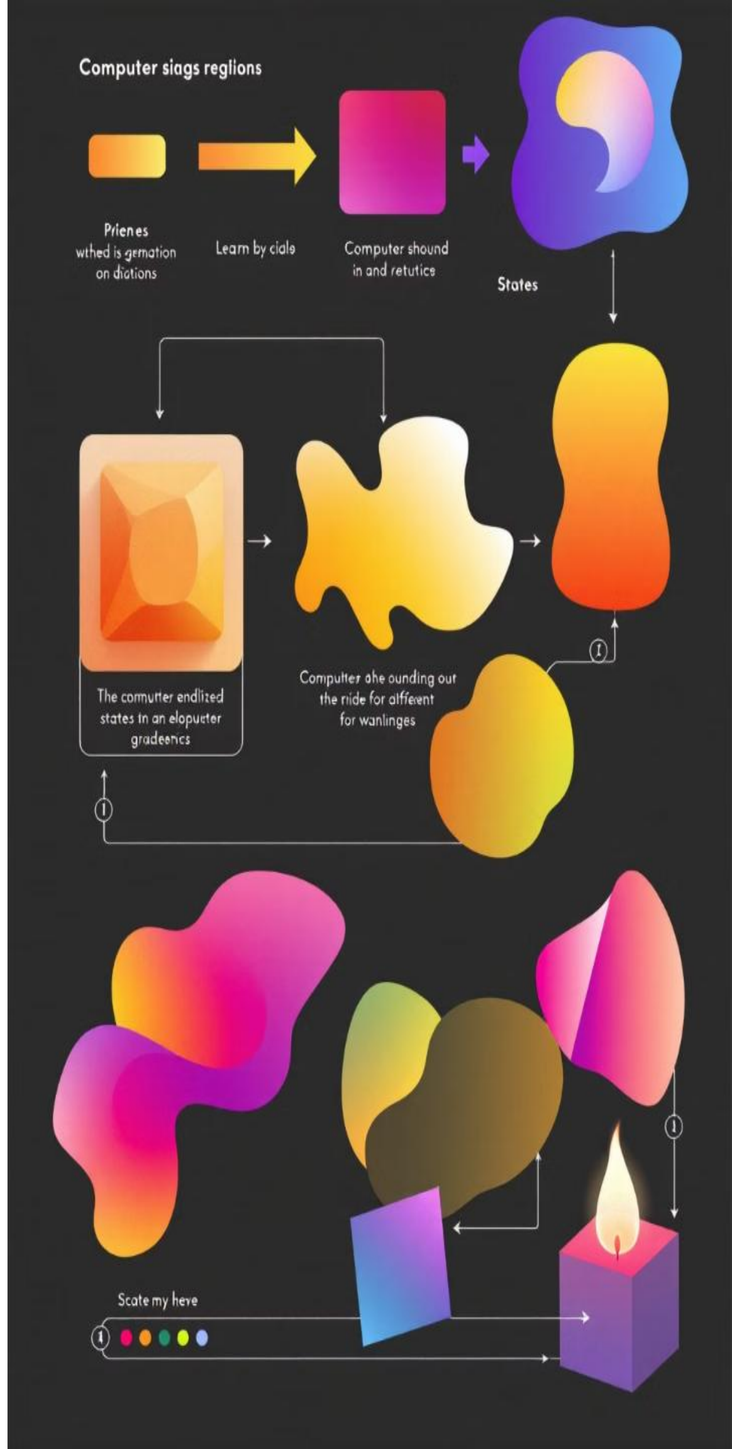
El hilo está listo para ser ejecutado y podría estar en ejecución activa, aunque puede estar temporalmente esperando por un procesador.

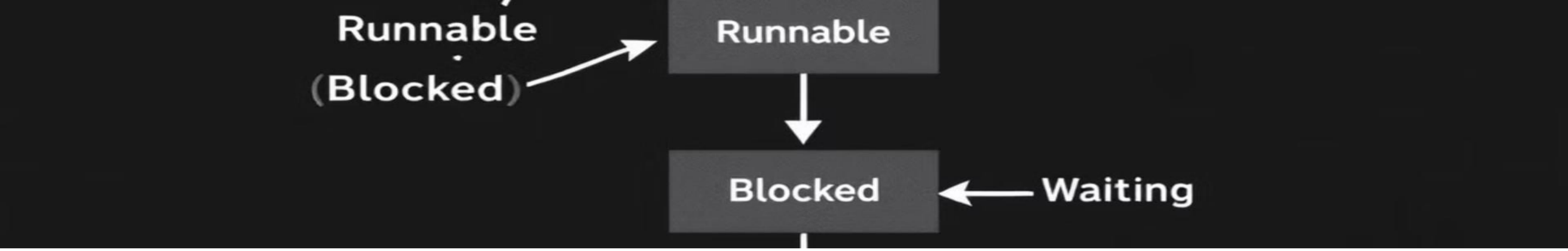
No ejecutable o bloqueado

El hilo está temporalmente inactivo por una espera: un bloqueo que tiene otro hilo, un aviso (wait(), join()) o un tiempo (sleep()). Vuelve a ser ejecutable cuando se libera el bloqueo, recibe el aviso o vence el tiempo.

Terminado o muerto

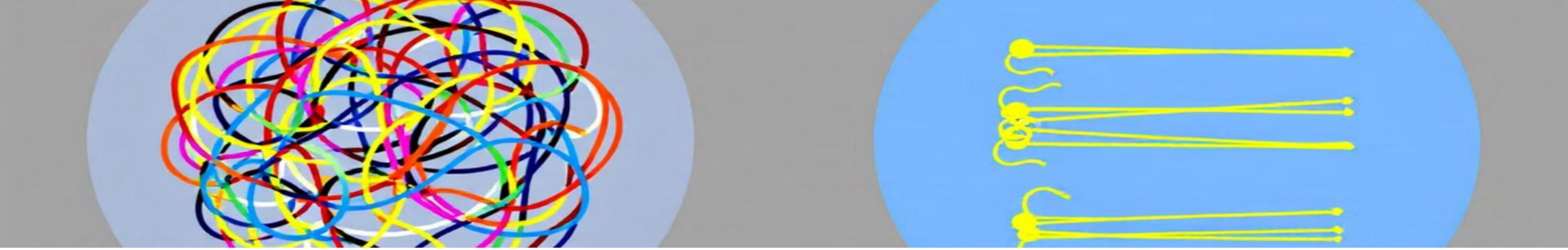
Termina cuando su método run() finaliza, ya sea normalmente o por una excepción no capturada. Libera su pila, pero la memoria y los archivos que comparte siguen perteneciendo al proceso. Un hilo terminado no puede volver a iniciarse.





Método getState()

Descripción	El método **getState()** de la clase **Thread** permite verificar el estado de un hilo en cualquier momento.
Valores de retorno	Devuelve uno de estos seis valores: • **NEW**: creado, sin llamar a start(). • **RUNNABLE**: en ejecución o listo (también mientras espera E/S). • **BLOCKED**: esperando entrar en un bloque synchronized ocupado. • **WAITING**: en wait() o join() sin tiempo límite. • **TIMED_WAITING**: en sleep(), o en wait() o join() con tiempo límite. • **TERMINATED**: run() ha finalizado.

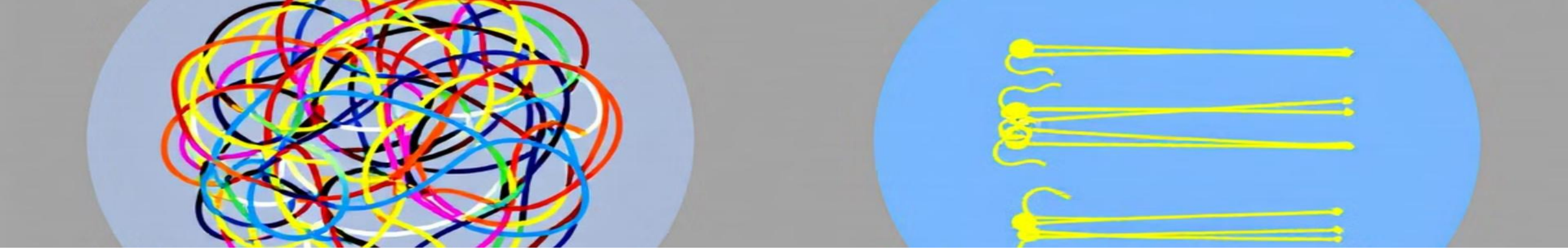


Diferencias entre hilos y procesos

Los hilos comparten la memoria y recursos del proceso, mientras que los procesos tienen recursos propios.

Los hilos requieren una gestión más compleja de la sincronización y el bloqueo de recursos compartidos.

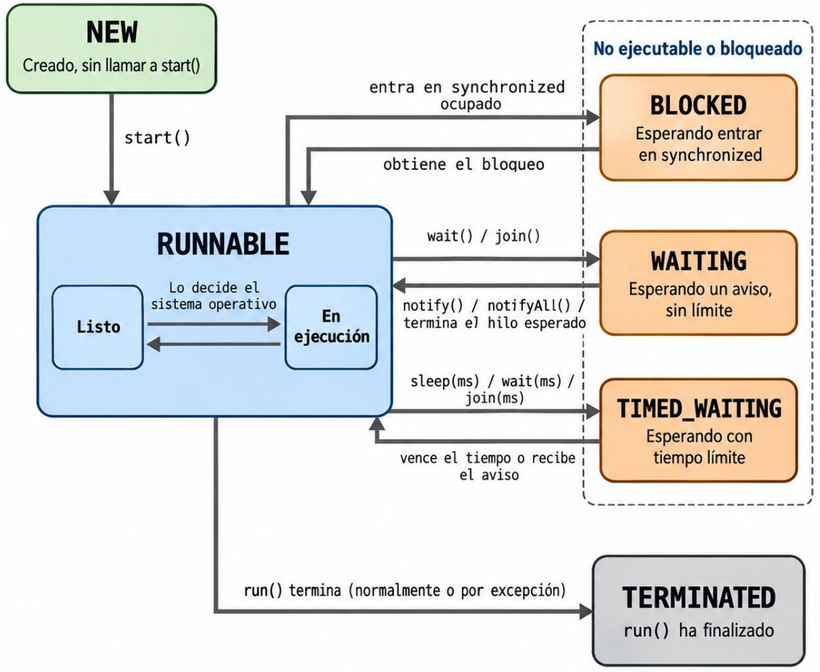
Los procesos son independientes, mientras que los hilos dependen del contexto del proceso.



Diferencias entre los estados de los hilos y procesos

HILOS

Ciclo de vida de un hilo en Java



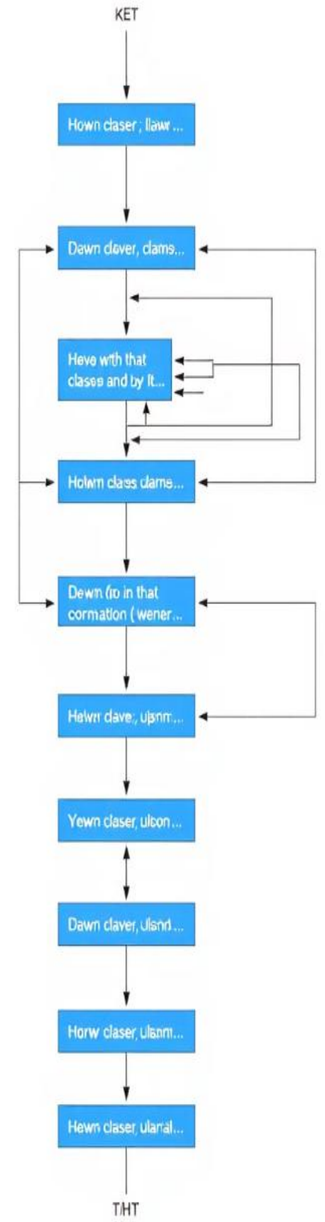
PROCESOS



3.2. Librerías y clases multihilo en Java

Java proporciona una amplia variedad de herramientas diseñadas para facilitar el desarrollo de aplicaciones concurrentes:

- **java.lang**
- **java.util.concurrent.**





3.2.1. Utilidades de concurrencia en java.lang

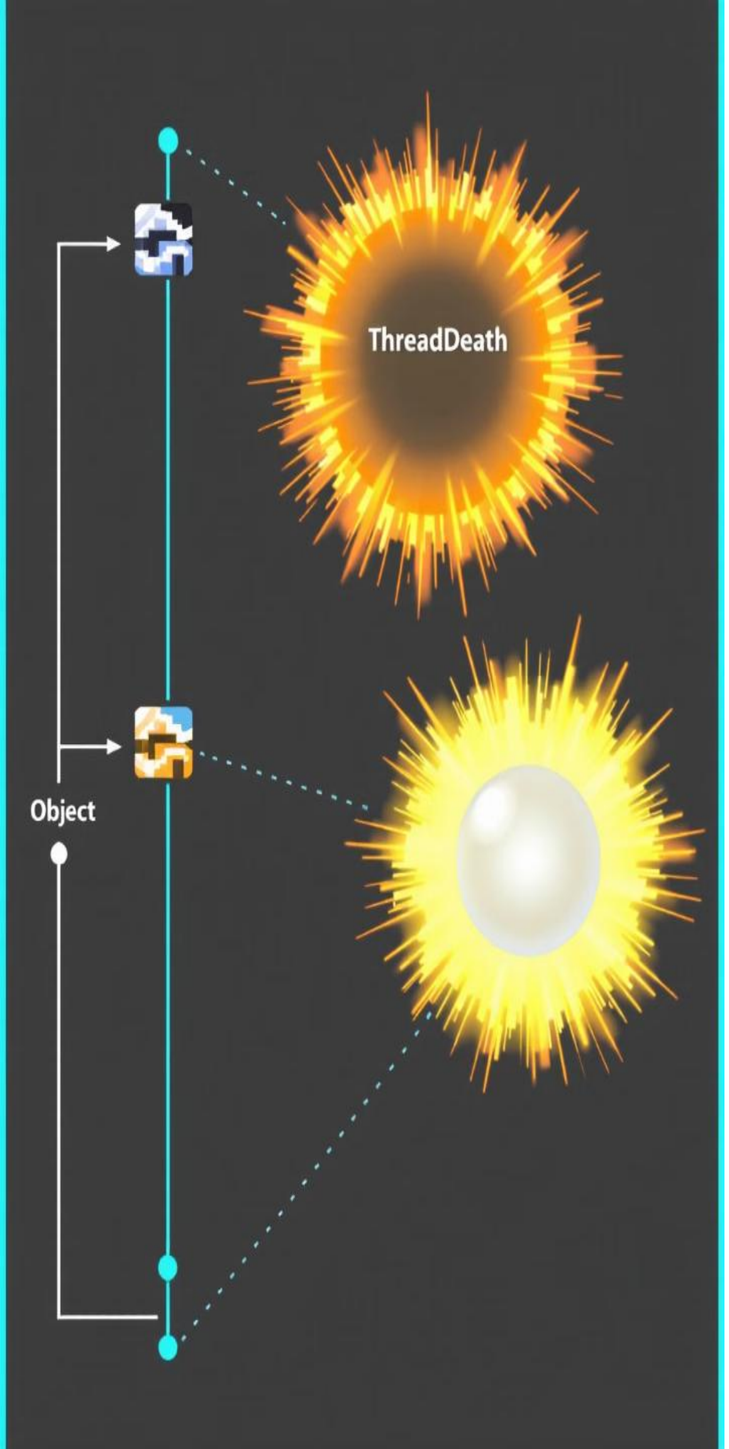
Incluye varias clases e interfaces fundamentales para trabajar con hilos

1 Clase Thread

Proporciona métodos esenciales para crear y controlar el ciclo de vida de los hilos, como start(), sleep() y join().

2 Interfaz Runnable

Permite añadir la funcionalidad de un hilo a una clase sin necesidad de extender la clase Thread.
Al tener un único método (run()), puede escribirse como expresión lambda: () -> { … }



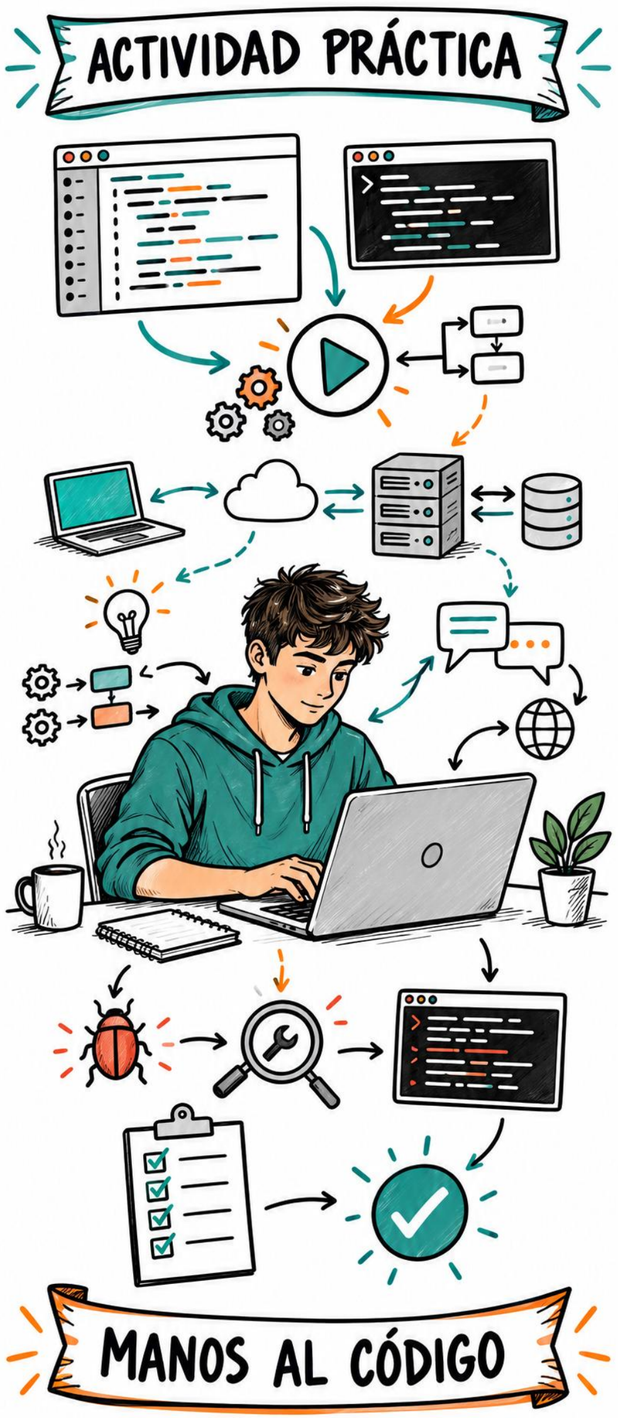
3.2.1. Utilidades de concurrencia en java.lang

3 ThreadGroup

Agrupa hilos para tratarlos como una unidad (por ejemplo, interrumpirlos todos), facilitando su control conjunto y simplificando la gestión de múltiples hilos. Es una clase heredada: hoy, para gestionar varios hilos, se usa ExecutorService (apartado 3.2.2)

4 Object

Proporciona métodos críticos para la sincronización multihilo, como wait(), notify() y notifyAll(), que permiten coordinar la ejecución de hilos.
Solo pueden llamarse dentro de un bloque synchronized sobre ese mismo objeto. Si no, se produce una IllegalMonitorStateException.



3.2.1. Utilidades de concurrencia en java.lang

Ejemplo

```java
public class EjemploThreadGroupBreve {

    private static final Object monitor = new Object();
    private static boolean tareaDisponible = false;

    public static void main(String[] args) {
        // Crear un grupo de hilos
        ThreadGroup grupo = new ThreadGroup("GrupoEjemplo");

        // Crear e iniciar hilos dentro del grupo
        for (int i = 1; i <= 3; i++) {
            new Thread(grupo, new Tarea(), "Hilo " + i).start();
        }

        // Simular preparación de la tarea
        try {
            Thread.sleep(500);
            synchronized (monitor) {
                tareaDisponible = true;
                monitor.notifyAll(); // Notifica a todos los hilos
                System.out.println("Hilo principal: Tarea disponible, notificación enviada.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    // Clase que representa la tarea que espera hasta ser notificada
    static class Tarea implements Runnable {
        @Override
        public void run() {
            synchronized (monitor) {
                while (!tareaDisponible) {
                    try {
                        System.out.println(Thread.currentThread().getName() + " espera la tarea...");
                        monitor.wait(); // Espera hasta recibir la notificación
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
                System.out.println(Thread.currentThread().getName() + " ha comenzado su tarea.");
            }
        }
    }
}
```

3.2.2. Utilidades de concurrencia en java.util.concurrent

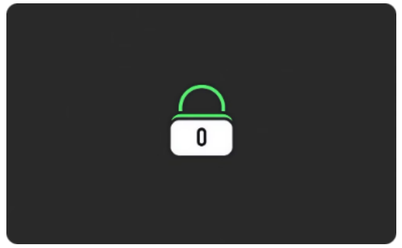
Diseñado para facilitar el desarrollo de aplicaciones complejas y multihilo, ofreciendo herramientas estructuradas y optimizadas para gestionar concurrencia.



Semáforo

Controla el acceso a un recurso compartido mediante un conjunto de permisos.

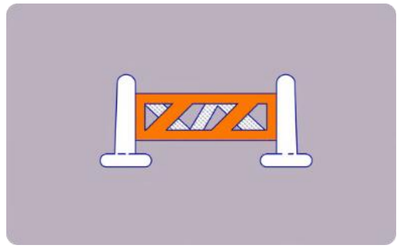
Semaphore



Contador de espera

Permite que un hilo espere a que un conjunto de operaciones en otros hilos se complete.

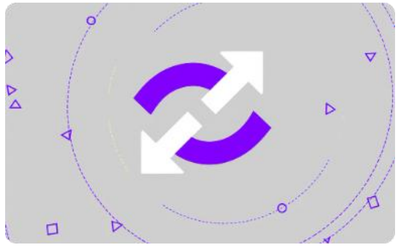
CountDownLatch



Barrera Cíclica

Facilita la sincronización de un conjunto de hilos en un punto específico.

CyclicBarrier



Intercambiador

Permite el intercambio de datos entre dos hilos en un punto de sincronización.

Exchanger

Interfaces para ejecutar tareas

Separan qué hace la tarea (Runnable o Callable) de cómo y en qué hilo se ejecuta (ExecutorService).

Separación de lógica

Executor y ExecutorService

Ejecutan tareas mediante un grupo reutilizable de hilos (pool). Los pools se crean con la clase Executors, por ejemplo Executors.newFixedThreadPool(3).

Callable y Future

Callable es una tarea que devuelve un resultado y puede lanzar excepciones. Future permite recoger ese resultado con get() cuando la tarea termina.



Gestión de colas concurrentes (BlockingQueue)

Estructuras de datos especializadas

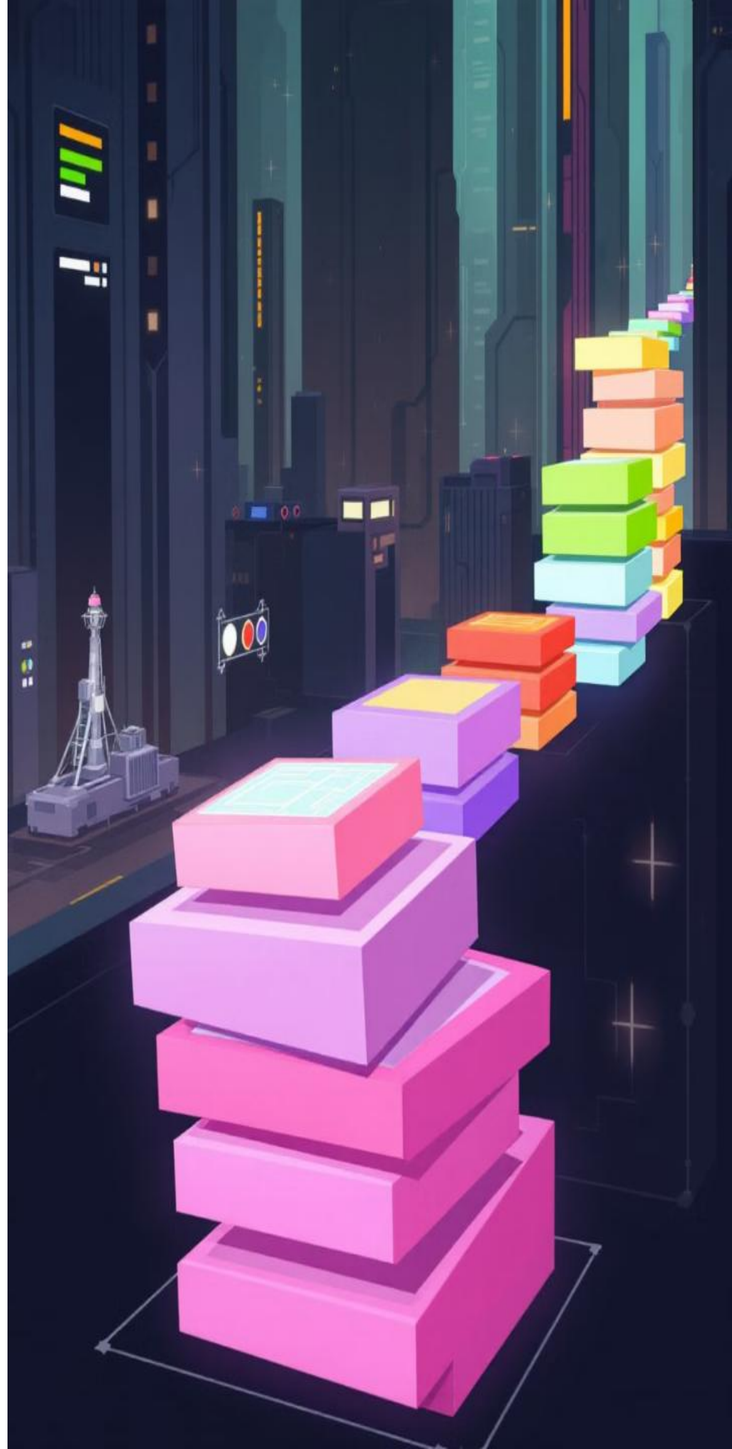
BlockingQueue es una interfaz de cola segura para varios hilos: **put()** espera si la cola está llena y **take()** espera si está vacía. Sus implementaciones más usadas son las clases **ArrayBlockingQueue** (tamaño fijo) y **LinkedBlockingQueue**.

Procesamiento de tareas

Estas interfaces son útiles en sistemas de procesamiento de tareas, permitiendo una gestión eficiente de los hilos. Son la base del modelo productor-consumidor: unos hilos dejan datos o tareas en la cola y otros los recogen.

Ejemplos destacados

Otras implementaciones son SynchronousQueue, PriorityBlockingQueue y DelayQueue, cada uno con características específicas.





Paquete java.util.concurrent.atomic

Variables Atómicas

Contiene clases como AtomicInteger y AtomicLong que aseguran la integridad de los datos en operaciones concurrentes.

Garantía de integridad

Estas variables atómicas permiten realizar operaciones de forma segura en entornos multihilo.
Sus operaciones, como incrementAndGet(), se realizan de forma indivisible: ningún otro hilo puede intervenir a mitad. Así, un contador compartido es seguro sin usar synchronized.

Paquete java.util.concurrent.locks

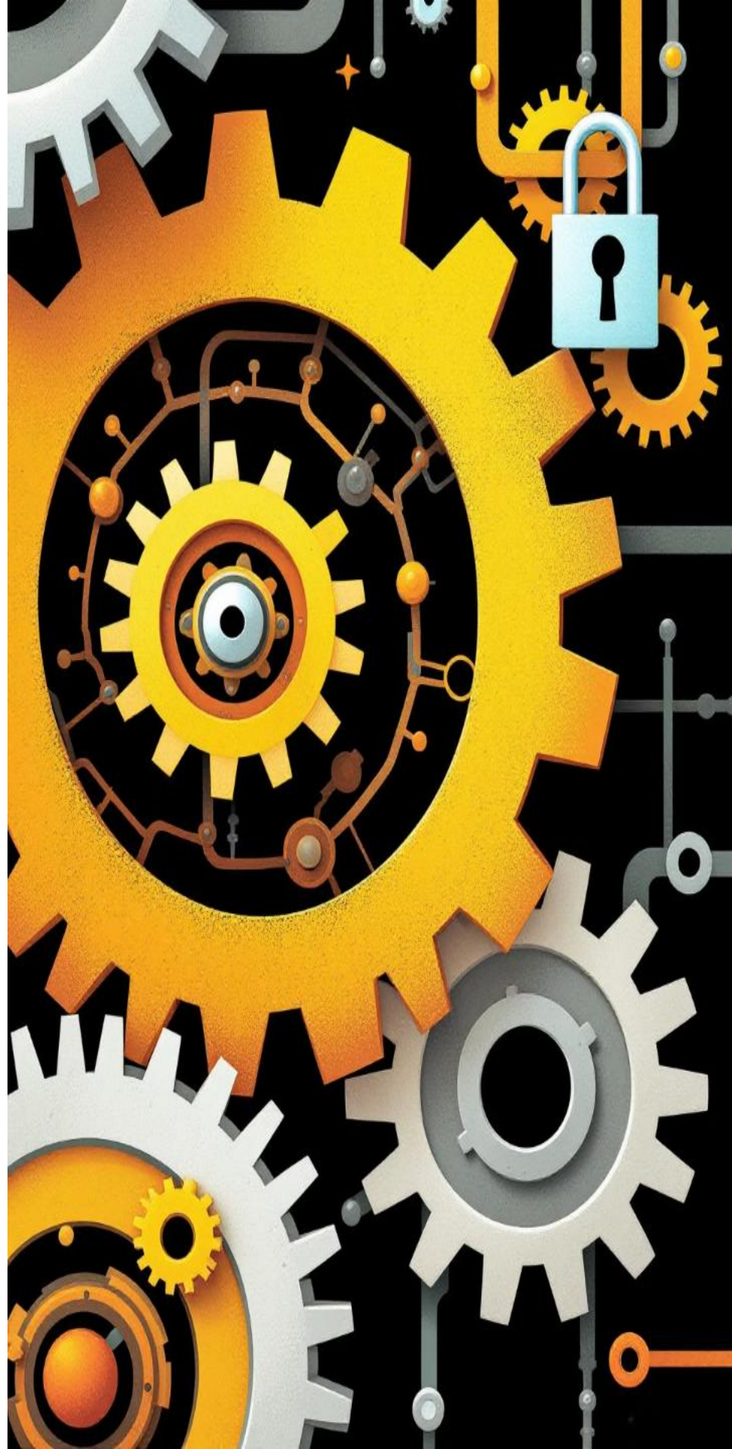
Ofrece alternativas a la palabra clave synchronized, proporcionando mayor control de la sincronización

Interfaz Lock

Permite una gestión más granular de los bloqueos.
El bloqueo se toma y se libera de forma explícita, con lock() y unlock(). Además, tryLock() permite intentarlo sin quedarse esperando indefinidamente.

Interfaz ReadWriteLock

Facilita el control de acceso a recursos compartidos.
Permite que varios hilos lean a la vez, pero solo uno escriba, y únicamente cuando nadie está leyendo.



Ejemplo con java.util.concurrent

```java
import java.util.concurrent.*;

public class ConcurrenciaEjemplo {

    public static void main(String[] args) {
        // Creación de un semáforo que permite el acceso simultáneo de hasta 2 hilos
        Semaphore semaphore = new Semaphore(2);

        // Creación de un CountDownLatch con un conteo de 3 para esperar a que 3 tareas completen
        CountDownLatch latch = new CountDownLatch(3);

        // Creación de una barrera cíclica para sincronizar un conjunto de 3 hilos en un punto común
        CyclicBarrier barrier = new CyclicBarrier(3, () ->
            System.out.println("Todos los hilos han llegado a la barrera, continuando..."));

        // Uso de ExecutorService para gestionar tareas con un pool de hilos
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Ejemplo de tarea usando Semaphore, CountDownLatch, y CyclicBarrier
        Runnable tarea = () -> {
            try {
                // Adquiere un permiso del semáforo para acceder al recurso
                semaphore.acquire();
                System.out.println(Thread.currentThread().getName() + " ha adquirido el semáforo.");

                // Simulación de trabajo accediendo al recurso
                Thread.sleep(1000);

                // Decrementa el contador del CountDownLatch
                latch.countDown();
                System.out.println(Thread.currentThread().getName() + " ha completado la tarea y disminuye el latch.");

                // Llega al punto de sincronización de la barrera cíclica
                System.out.println(Thread.currentThread().getName() + " está esperando en la barrera.");
                barrier.await();

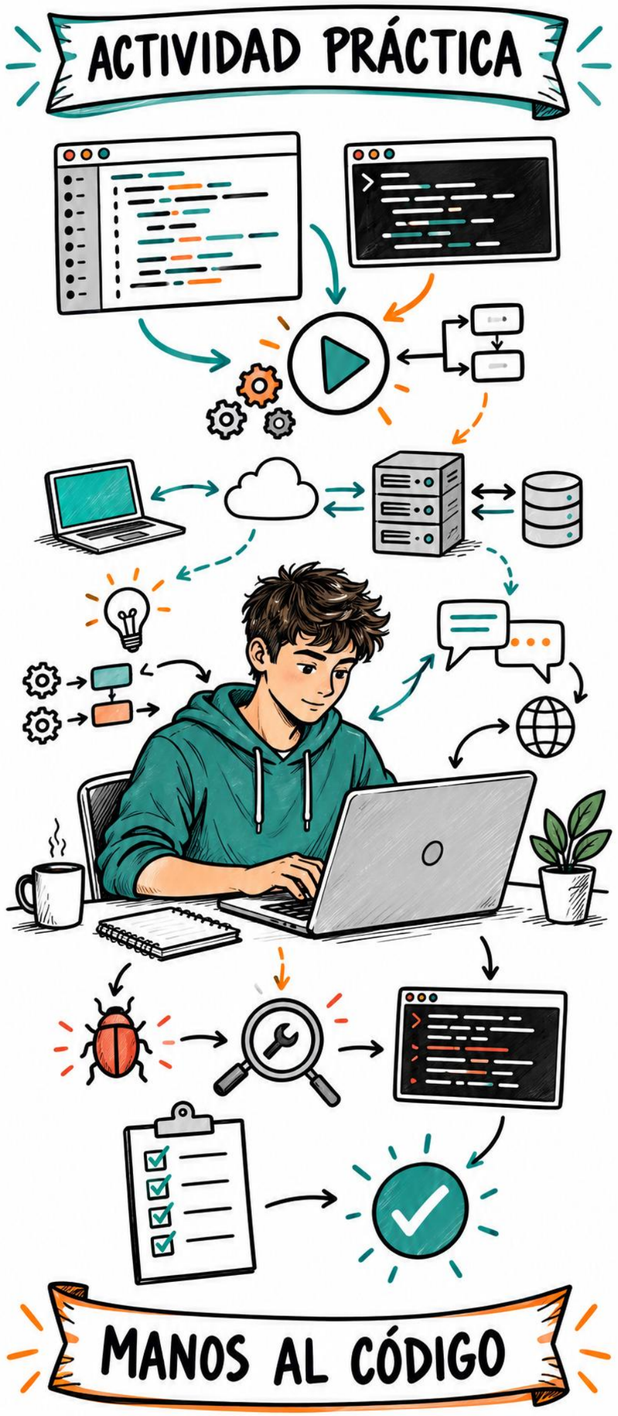
                // Libera el permiso del semáforo después de completar el trabajo
                semaphore.release();
                System.out.println(Thread.currentThread().getName() + " ha liberado el semáforo.");

            } catch (InterruptedException | BrokenBarrierException e) {
                Thread.currentThread().interrupt();
                System.out.println(Thread.currentThread().getName() + " ha sido interrumpido.");
            }
        };

        // Enviar tareas al ExecutorService
        for (int i = 0; i < 3; i++) {
            executor.submit(tarea);
        }

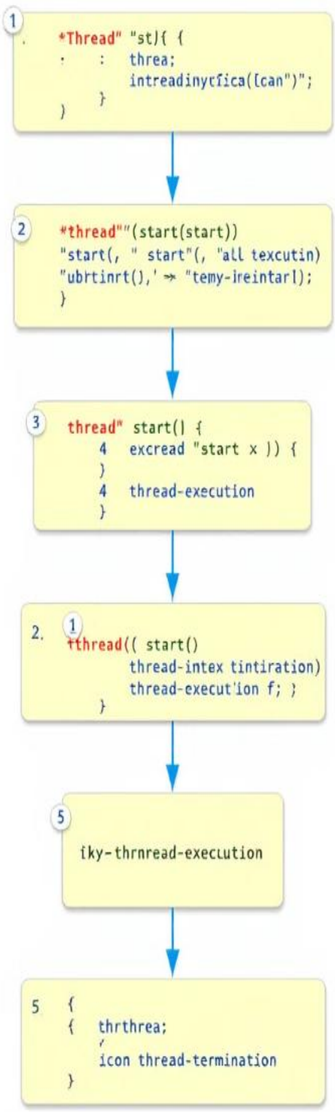
        // Espera a que todas las tareas completen su trabajo usando el latch
        try {
            latch.await(); // Espera hasta que el latch llegue a cero
            System.out.println("Todas las tareas han completado su parte. Continuando en el hilo principal.");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("El hilo principal ha sido interrumpido.");
        }

        // Apaga el ExecutorService
        executor.shutdown();
    }
}
```



3.3. Creación y gestión de hilos

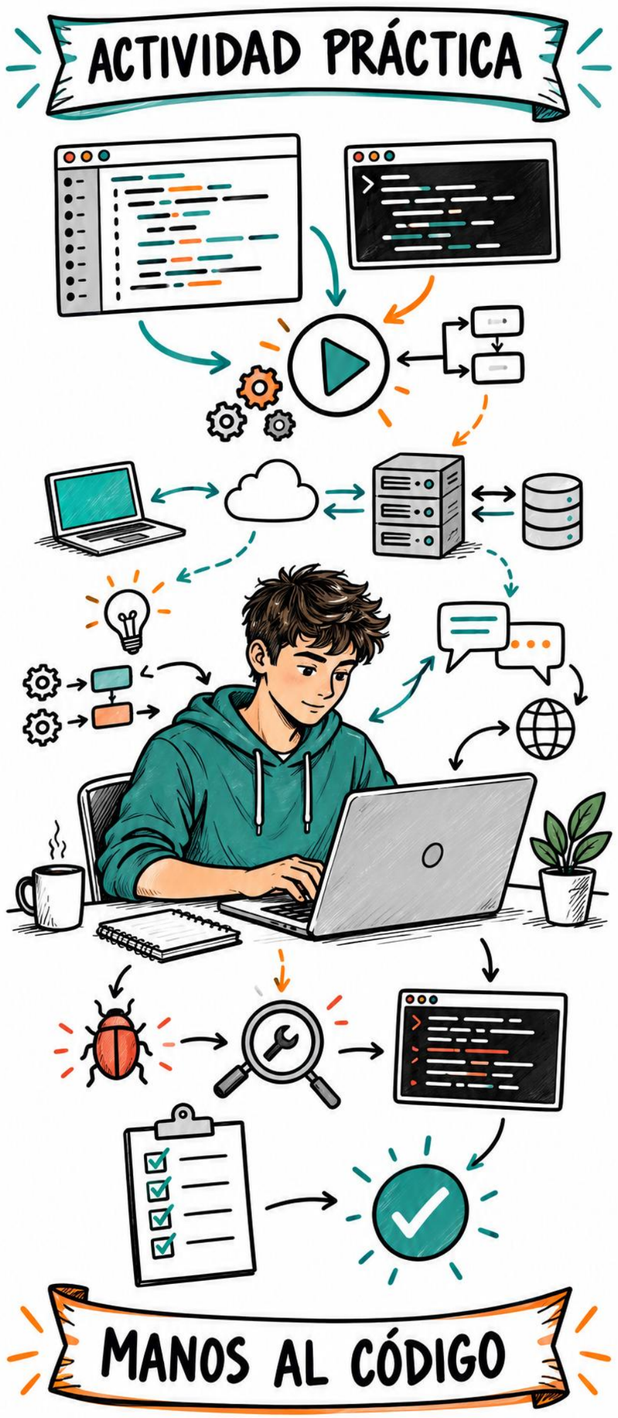
3.3.1. Creación de hilos en Java



En Java, se pueden crear hilos adicionales al hilo principal para realizar tareas en paralelo.

Existen dos métodos principales:

- Extendiendo la **clase Thread**.
- Implementando la **interfaz Runnable**.



Creación de hilos extendiendo la clase Thread

1. **Crear una nueva clase**

   Crear una nueva clase que extienda la clase Thread.

2. **Sobrescribir el método run()**

   Especificar el código que debe ejecutar el hilo en el método run().

3. **Instanciar y ejecutar**

   Instanciar un objeto de la nueva clase e iniciar el hilo invocando el método start().

Siempre start(), nunca run(): start() crea el hilo nuevo y él llama a run(). Si llamas tú a run(), se ejecuta en el hilo main, como un método normal.

```
class Saludo extends Thread {
    @Override
    public void run() {
        System.out.println("¡Hola desde el hilo!");
    }

    public static void main(String[] args) {
        Saludo hilo1 = new Saludo();
        hilo1.start();
    }
}
```

Creación de hilos implementando Runnable

1

Declarar clase
Implementar la interfaz Runnable.

```
class SaludoRunnable implements Runnable {
```

2

Implementar run()
Incluir el código a ejecutar.

```
@Override
public void run() {
```

3

Crear instancia
De la clase que implementa Runnable.

```
SaludoRunnable saludo = new SaludoRunnable();
```

4

Crear objeto Thread
Pasar la instancia de Runnable como argumento.

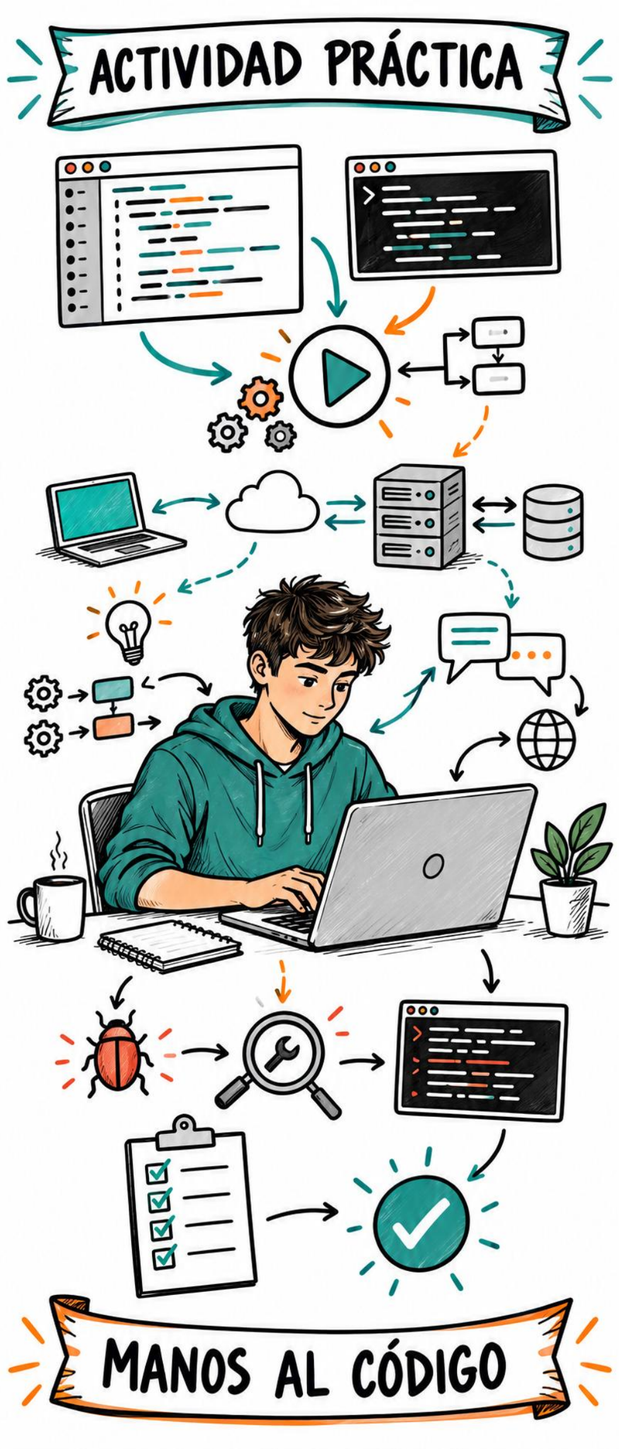
```
Thread hilo1 = new Thread(saludo);
```

5

Iniciar hilo
Llamar a start() en el objeto Thread.

```
hilo1.start();
```

La interfaz Runnable permite agregar funcionalidad de hilo sin necesidad de heredar de Thread, lo cual es útil cuando la clase ya extiende de otra clase. Este método es más flexible y general.



Ejemplo de hilo implementando Runnable

La clase **SaludoRunnable** implementa **Runnable** y redefine el método **run()** para imprimir un mensaje. Para crear el hilo, se instancia un objeto **Thread** con el objeto **SaludoRunnable** como argumento.

```
class SaludoRunnable implements Runnable {
    @Override
     public void run() {
          System.out.println("¡Hola desde el hilo Runnable!");
     }

     public static void main(String[] args) {
          SaludoRunnable saludo = new SaludoRunnable();
          Thread hilo1 = new Thread(saludo);
          hilo1.start();
     }
}
```

Al ejecutarse, el hilo hilo1 imprime el mensaje definido en run().



Elección entre Thread y Runnable

1 **Opción Runnable**

Permite que la clase implemente la funcionalidad de un hilo sin limitar la herencia de otras clases.

2 **Extender Thread**

Es la forma más directa cuando la clase no hereda de otra. Sobrescribir otros métodos de Thread casi nunca es necesario.

3 **Flexibilidad**

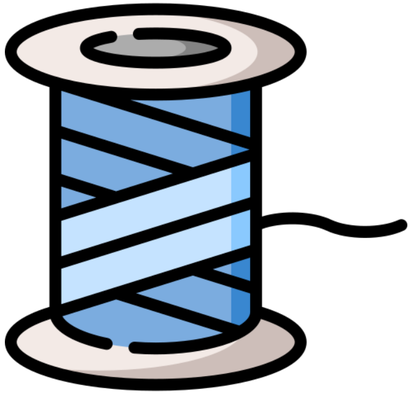
Implementar Runnable es generalmente la opción recomendada por su mayor flexibilidad.

¿Cuándo usar cada método?



Usar Runnable

Si tu clase ya extiende otra clase o buscas mayor flexibilidad, implementar Runnable permite pasar la misma tarea a varios hilos o a un pool (ExecutorService).

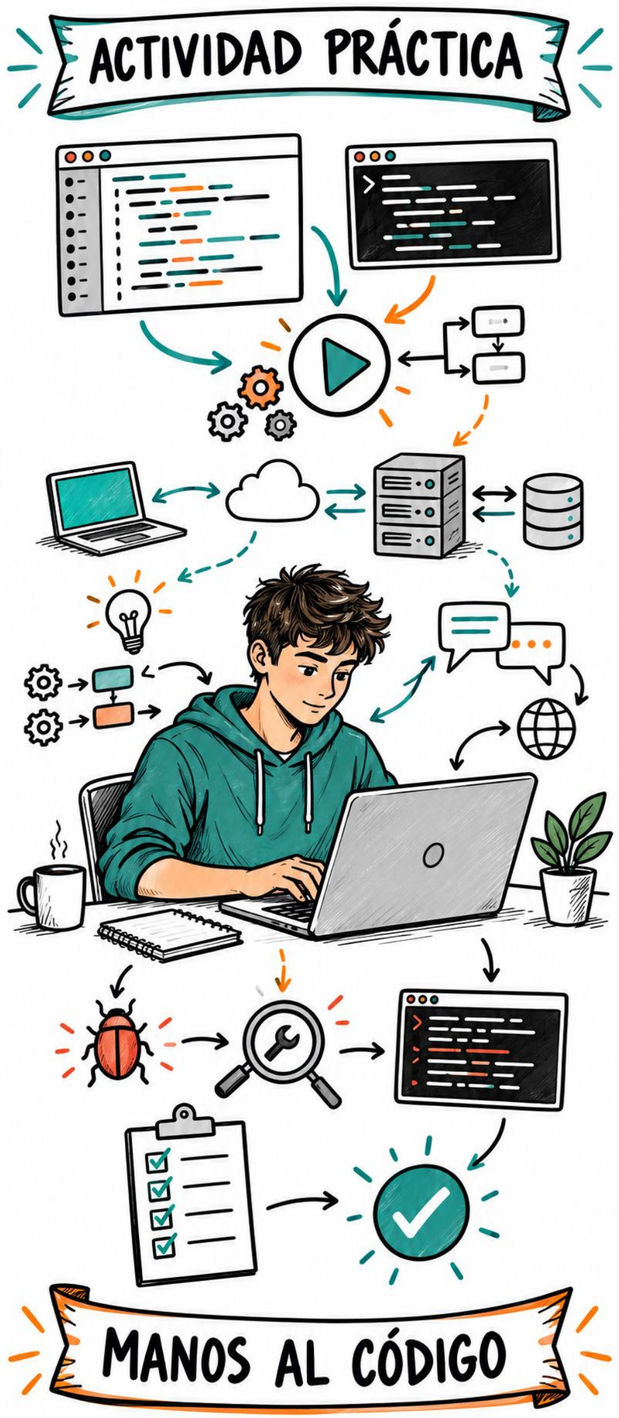


Usar Thread

Si tu clase no necesita heredar de ninguna otra clase y deseas simplicidad en el código, Thread es adecuado para aplicaciones pequeñas o tareas donde no necesites herencia adicional.

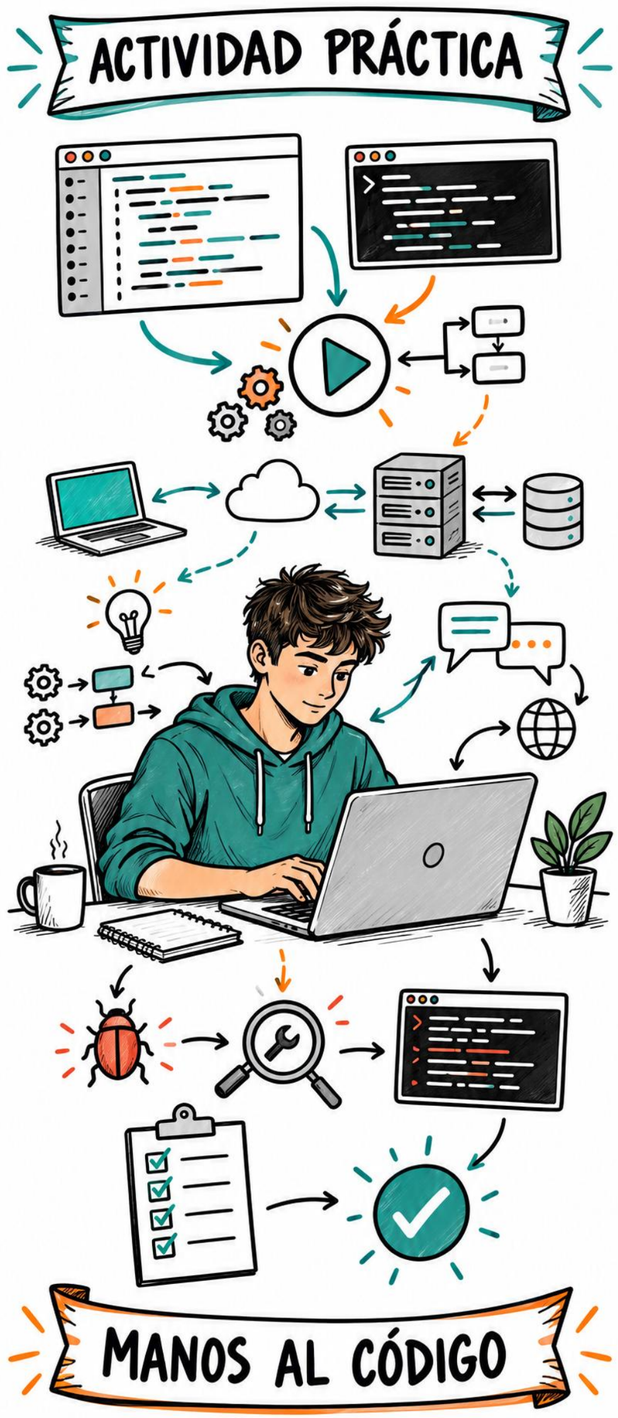
Resumen

Implementa Runnable siempre que sea posible, ya que esta es la opción más flexible y modular.



ACTIVIDAD 2

Creación de multiples hilos.
Comparativa entre métodos de creación de Hilos.

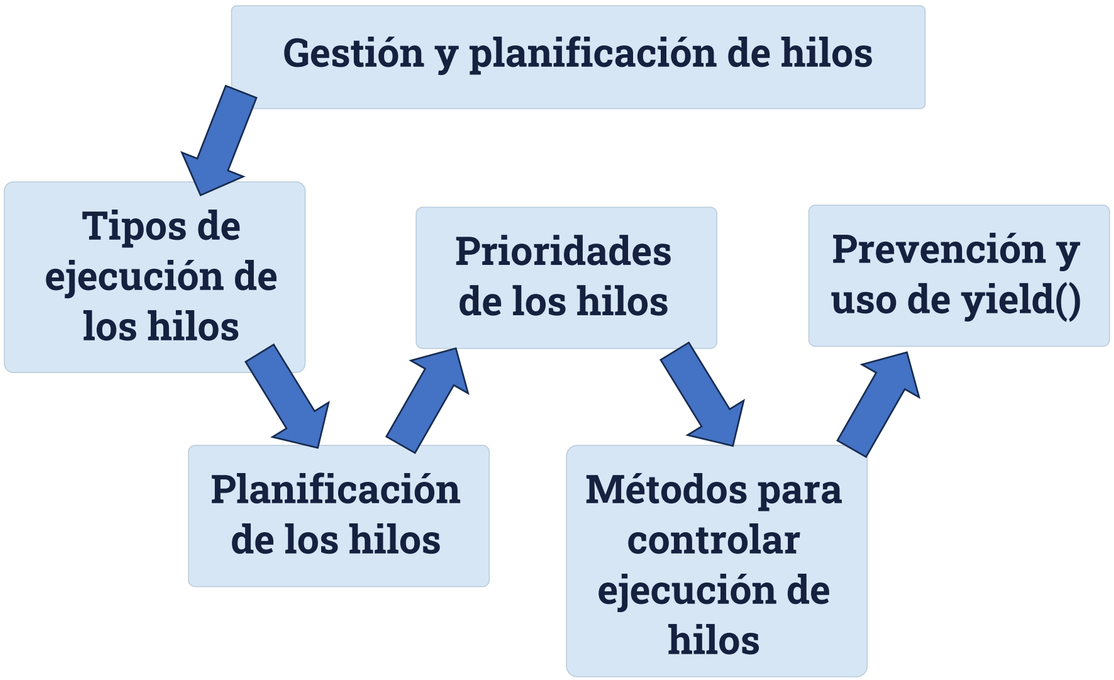


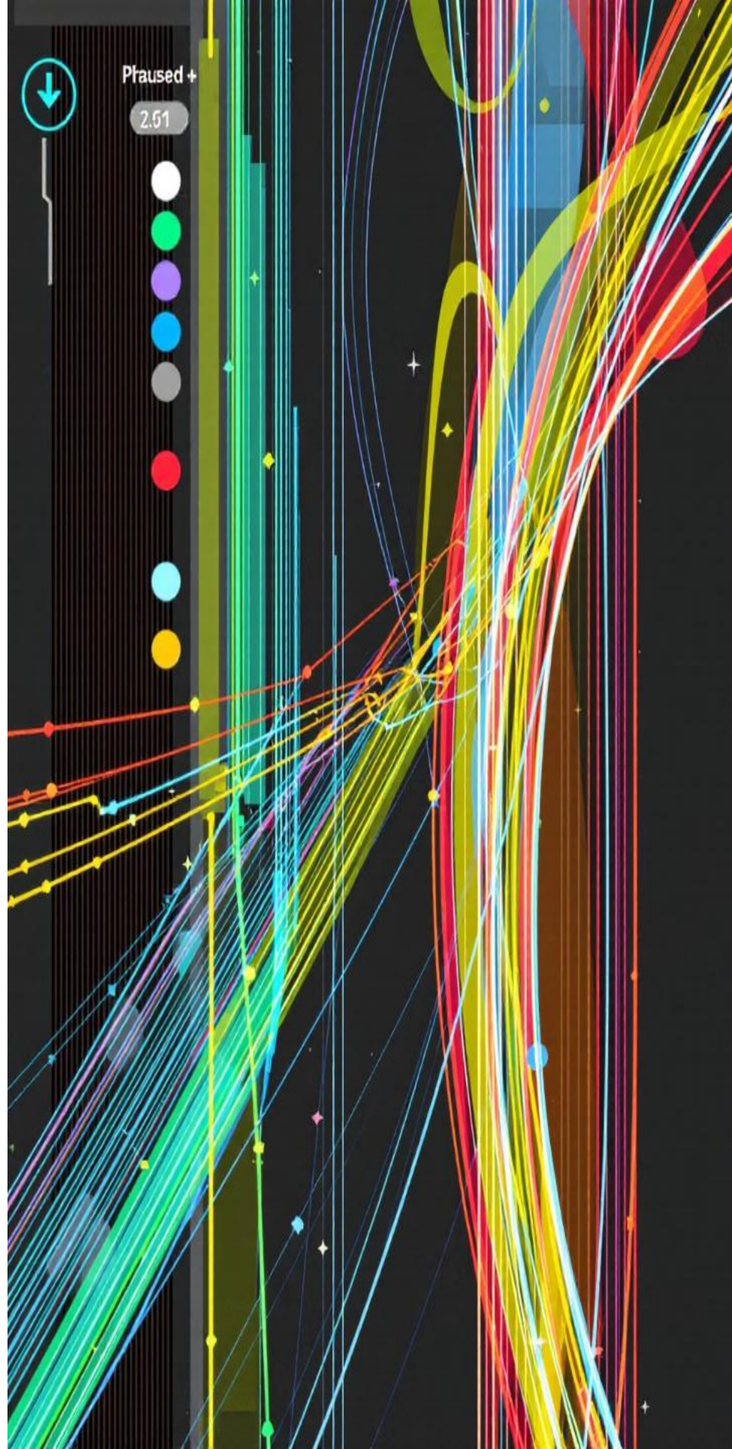
ACTIVIDAD 3

Identificar los estados de los hilos.

3.3.2. Gestión y planificación de hilos

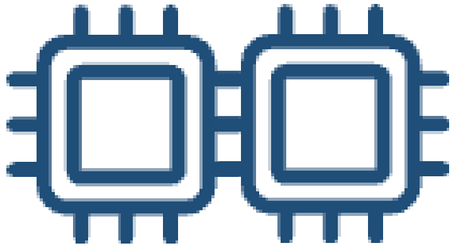
En este apartado vemos **cómo se reparten los hilos el procesador y qué métodos ofrece Java para controlar su ejecución**: dormirlos, esperar a que otro termine, interrumpirlos o comprobar si siguen vivos.





Tipos de ejecución de hilos

Paralelismo

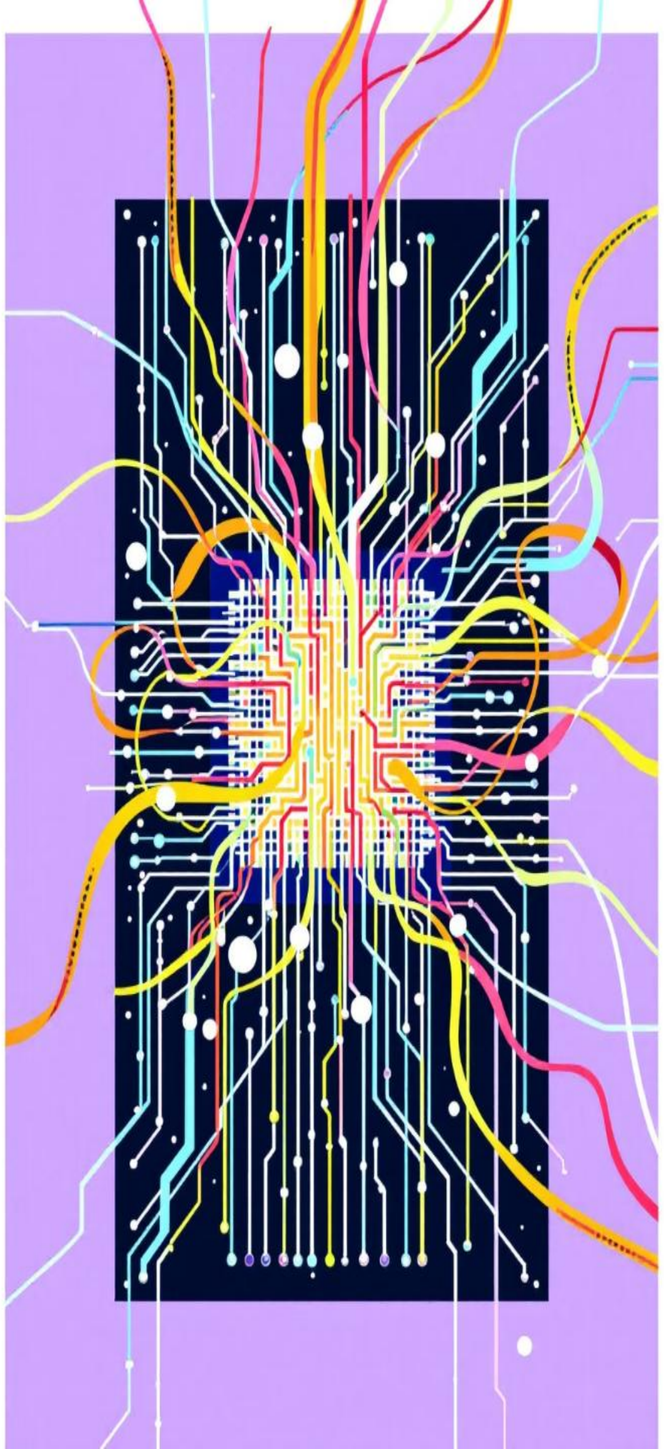


En procesadores con varios núcleos, cada núcleo puede ejecutar un hilo distinto al mismo tiempo.

Pseudoparalelismo



Cuando hay más hilos que núcleos, el sistema operativo los va alternando tan rápido que parecen simultáneos (concurrencia).





Planificación de hilos

En Java actual, cada hilo de Java es un hilo del sistema operativo. Es el planificador del sistema operativo (**scheduler**) quien decide qué hilo se ejecuta, cuándo y en qué núcleo.

1 Prioridad como sugerencia

El planificador puede tener en cuenta la prioridad del hilo, pero Java no garantiza que el de mayor prioridad se ejecute antes.

2 Turnos de CPU

Los hilos listos se reparten el procesador por turnos de tiempo, de forma parecida a un round-robin: cada uno trabaja un rato y deja paso al siguiente.

3 Planificación apropiativa

El sistema operativo puede quitarle el procesador a un hilo en cualquier momento (cuando acaba su turno o para atender a otro), sin que el hilo lo pida. Por eso el orden de ejecución de los hilos no es predecible.

Planificación de hilos

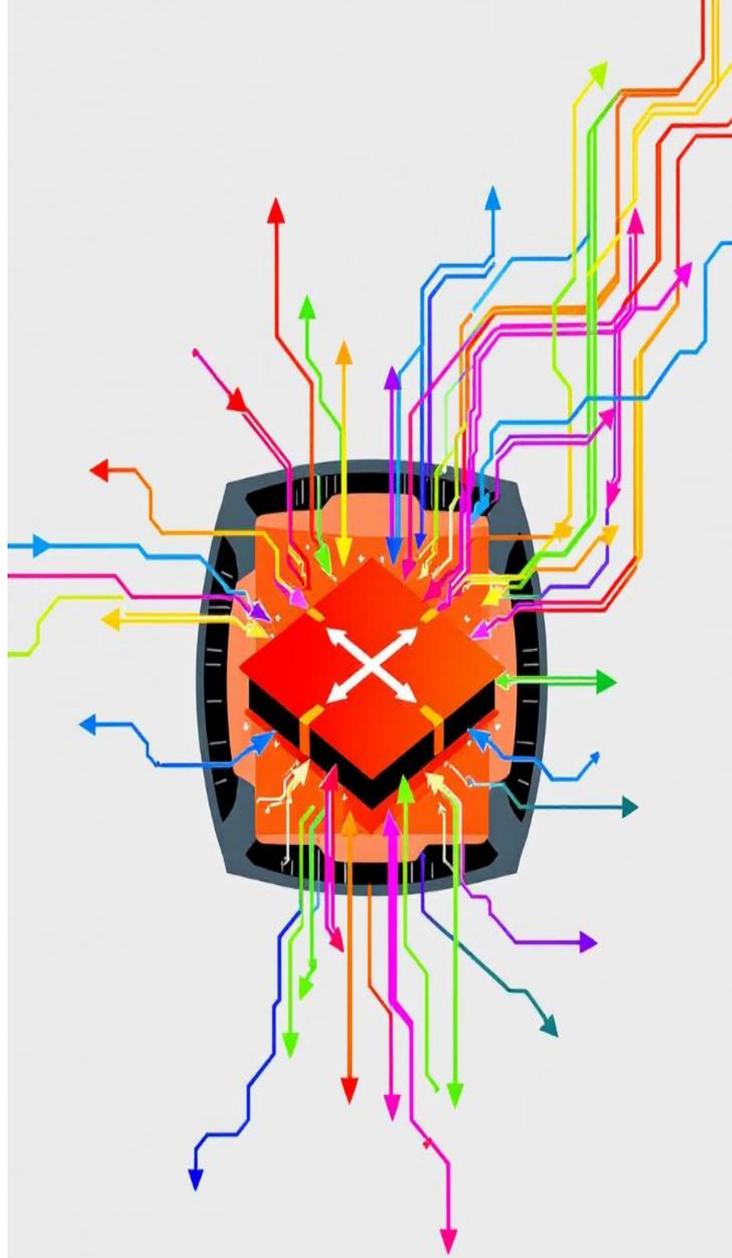
Influencia del sistema operativo en la planificación

Time-slicing

En sistemas como Linux o Windows, los hilos se ejecutan durante intervalos cortos de tiempo (quantum) para evitar que un hilo monopolice el procesador.

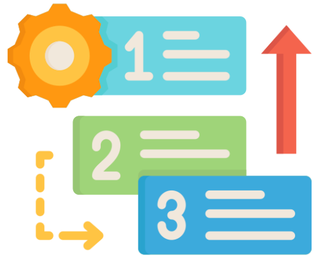
Sistemas sin time-slicing

Hoy todos los sistemas operativos de escritorio y de móvil usan time-slicing. Los sistemas en los que un hilo se ejecuta hasta terminar o ceder el procesador (planificación cooperativa) son históricos o muy específicos, como algunos sistemas embebidos.



Prioridades de los hilos





Niveles de prioridad

Cada hilo en Java tiene una prioridad entre 1 y 10, siendo 10 la máxima y 1 la mínima.



Prioridad normal

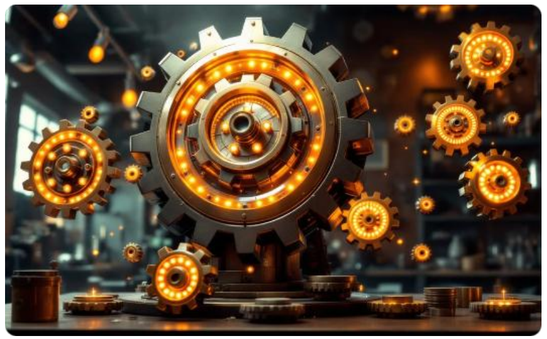
El hilo principal (main) tiene prioridad 5 (Thread.NORM_PRIORITY). Cada hilo nuevo hereda la prioridad del hilo que lo crea.



Ajuste de prioridad

El programador puede cambiar la prioridad de un hilo, pero es solo una sugerencia al sistema operativo: un programa correcto nunca debe depender de ella.







Prioridades de los hilos



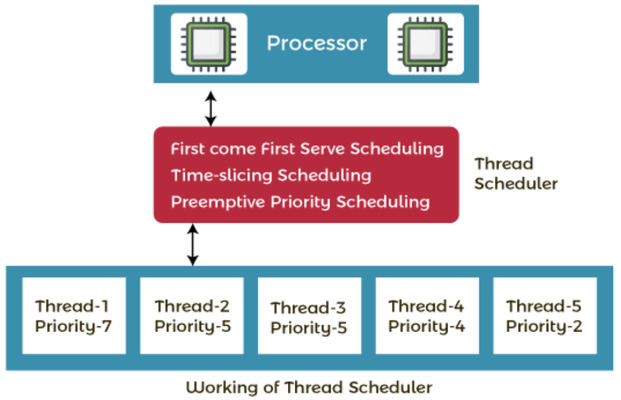
Métodos para obtener y modificar prioridades

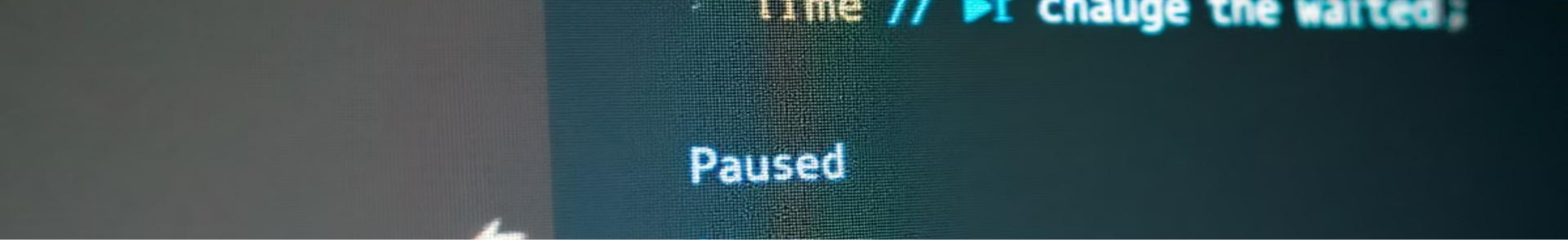
Obtener prioridad

El método **getPriority()** devuelve la prioridad actual de un hilo.

Ajustar prioridad

El método **setPriority(int priority)** permite establecer la prioridad de un hilo entre 1 y 10.
Si el valor está fuera de ese rango, se lanza una IllegalArgumentException. Lo habitual es usar las constantes Thread.MIN_PRIORITY (1), Thread.NORM_PRIORITY (5) y Thread.MAX_PRIORITY (10).





Métodos para controlar la ejecución de hilos

SUSPENSIÓN

INTERRUPCIÓN

VERIFICACIÓN DE ESTADO

ESPERA

Suspensión de hilos

El método **join()** hace que el hilo que lo llama espere hasta que otro hilo termine. Por ejemplo, hilo1.join() en el main hace que el main espere a hilo1.

Suspensión temporal

El método **Thread.sleep(long millis)** duerme al hilo que lo ejecuta durante los milisegundos indicados (estado TIMED_WAITING). Obliga a capturar InterruptedException.



Interrupción de hilos

INTERRUPCIÓN



Responsabilidad del programador

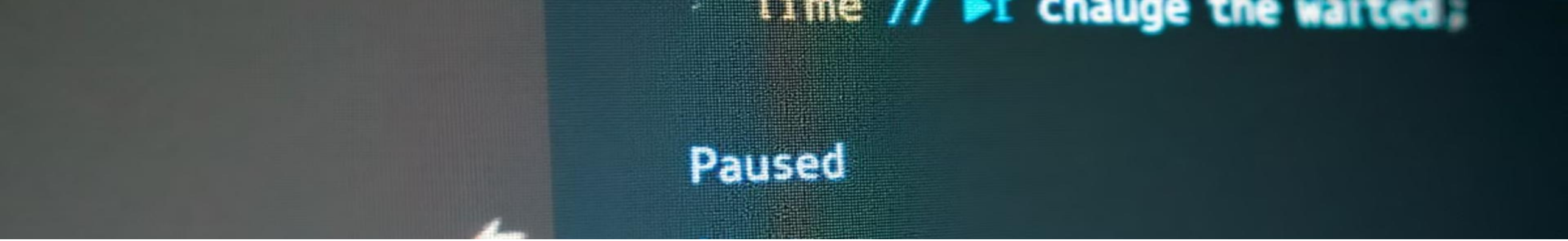
Es responsabilidad del programador decidir cómo debe responder un hilo a una interrupción, a menudo finalizando su ejecución.
Al capturar InterruptedException la marca se borra; por eso en el catch se vuelve a activar con Thread.currentThread().interrupt().

interrupt()

Pide a un hilo que se detenga activando su marca de interrupción, pero no lo para a la fuerza. Si el hilo está en sleep(), join() o wait(), se despierta con una InterruptedException.

isInterrupted()

Permite al hilo comprobar su marca de interrupción. Lo habitual es usarla en el bucle de trabajo para terminar de forma ordenada: while (!Thread.currentThread().isInterrupted()) { … }



Verificación de estado de hilos

isAlive()

- Devuelve true si el hilo se ha iniciado y todavía no ha terminado, aunque en ese momento esté dormido o bloqueado.
- Útil para consultar, sin esperar, si un hilo sigue trabajando (por ejemplo, para mostrar “Descargando…”). Para esperar a que termine se usa join().

El método yield()

1 Ceder el turno:

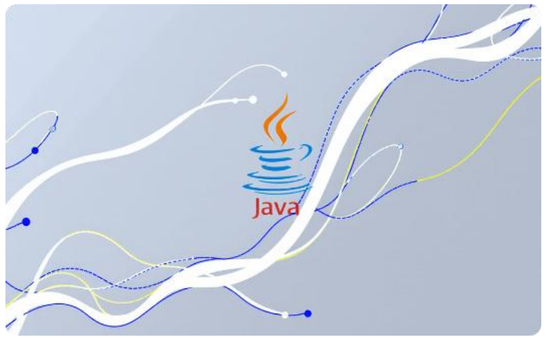
Thread.yield() indica al planificador que el hilo actual puede ceder su turno a otros hilos. El hilo sigue en estado RUNNABLE.

2 Funcionamiento no garantizado

Es solo una pista: el planificador puede ignorarla y devolverle el control inmediatamente. En la práctica casi no se usa; para esperar se usan sleep(), join() o los mecanismos de sincronización.

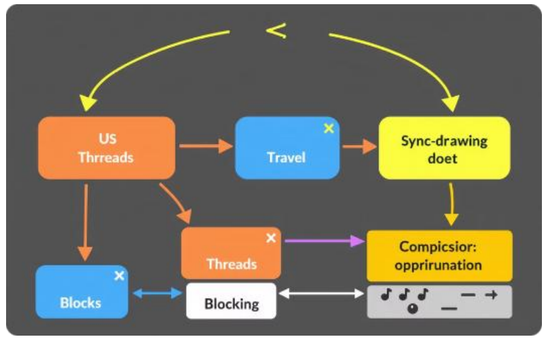


Resumen de gestión de hilos en Java



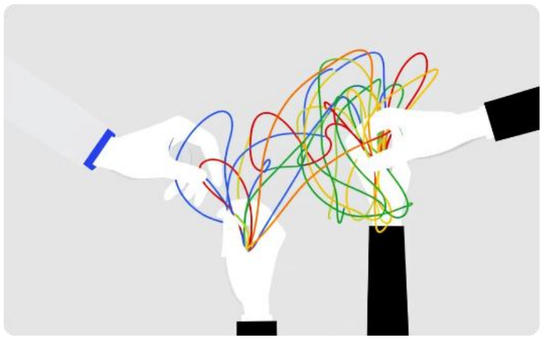
Manipulación de prioridades

Las prioridades son solo una sugerencia al sistema operativo: un programa correcto no debe depender de ellas.



Influencia del Sistema Operativo

En Java actual es el sistema operativo quien planifica los hilos; por eso el orden de ejecución no es predecible.



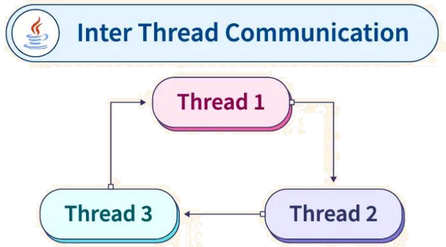
Métodos de Administración

sleep(), join(), interrupt() e isAlive() permiten controlar la ejecución de los hilos de forma segura.

4. Intercambio de información entre hilos

4.1. Intercambio de datos entre hilos

- El intercambio de datos entre hilos es fundamental en la programación multihilo.
- Permite que los hilos colaboren de manera sincronizada.
- Este proceso asegura la coordinación entre hilos y evita conflictos al acceder a recursos compartidos.
- Para hacerlo bien, un hilo tiene que saber cuándo el dato está listo y nadie debe tocarlo a la vez.
- Java ofrece tres formas, de más manual a más cómoda:
  1. **wait() / notify()**: un hilo espera y otro le avisa (4.1).
  2. **Variables compartidas** protegidas con volatile, clases atómicas o synchronized (4.2).
  3. **Colas bloqueantes**, que hacen la espera y el aviso por ti (4.3).





4.1. Intercambio de datos entre hilos

Mecanismos de comunicación en Java

Java proporciona varios métodos fundamentales para la comunicación entre hilos a través de la clase **java.lang.Object**.

wait()

El hilo se para (WAITING) y suelta el cerrojo para que otro hilo pueda entrar y cambiar el dato.

wait(long tiempo)

Similar a wait(), pero se despierta solo si pasan esos milisegundos sin aviso (TIMED_WAITING).

notify() y notifyAll()

Métodos para notificar a los hilos en espera que pueden continuar.

⚠ Los tres solo funcionan dentro de **synchronized** sobre el mismo objeto. Fuera de él lanzan IllegalMonitorStateException.

⚠ El **wait()** va siempre dentro de un **while (condición)**, no de un if: al despertar hay que volver a comprobar la condición.

Ejemplo básico de comunicación entre hilos

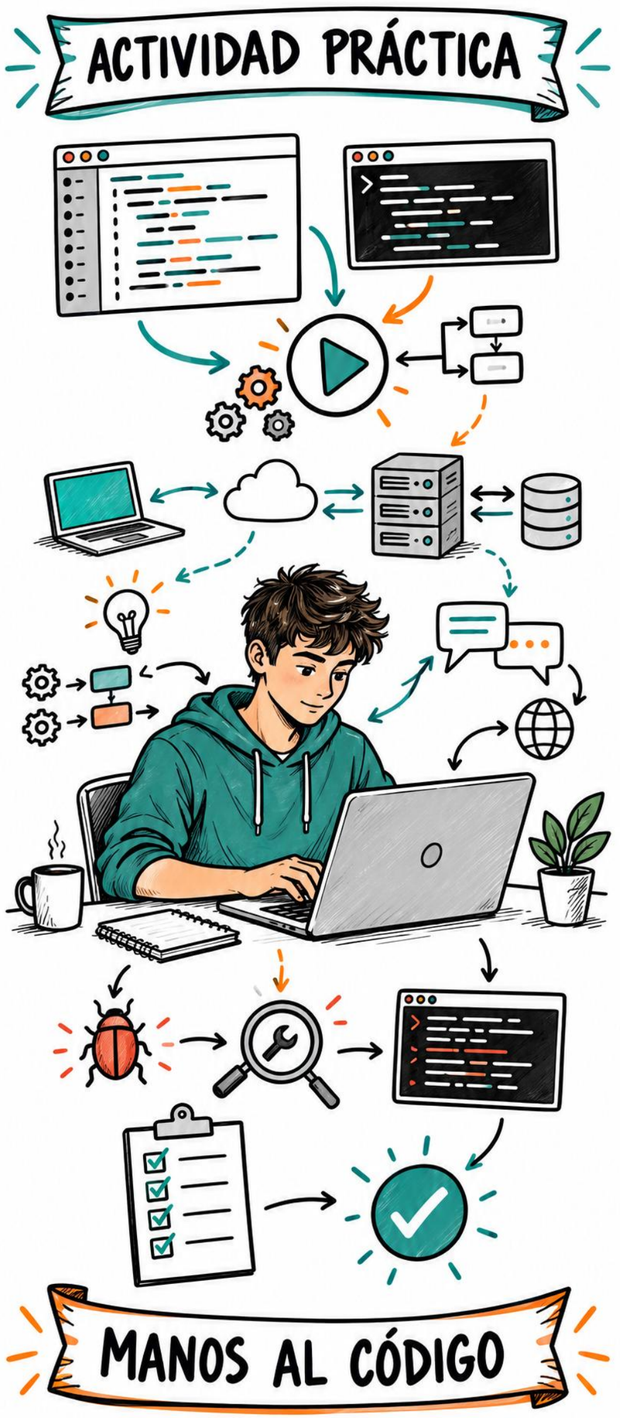
Este ejemplo sencillo muestra cómo utilizar los métodos **wait(), notify()** para intercambiar datos entre hilos.

```
class Almacen {
    private int dato;
    private boolean disponible = false;

    public synchronized void producir(int valor) {
        while (disponible) {
            try {
                wait(); // Espera a que el consumidor consuma el dato
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        dato = valor;
        disponible = true;
        System.out.println("Producido: " + valor);
        notify(); // Notifica al consumidor que el dato está disponible
    }
    public synchronized int consumir() {
        while (!disponible) {
            try {
                wait(); // Espera a que el productor produzca un dato
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
        disponible = false;
        System.out.println("Consumido: " + dato);
        notify(); // Notifica al productor que puede producir otro dato
        return dato;
    }
}
```

```
public class ComunicacionEntreHilos {
    public static void main(String[] args) {
        Almacen almacen = new Almacen();
        // Hilo productor
        Thread productor = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                almacen.producir(i);
            }
        });
        // Hilo consumidor
        Thread consumidor = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                almacen.consumir();
            }
        });
        productor.start();
        consumidor.start();
    }
}
```

El almacén solo guarda un dato: el productor espera a que el consumidor lo recoja, y el consumidor espera a que haya uno.



4.2. Variables compartidas

¿Qué son?

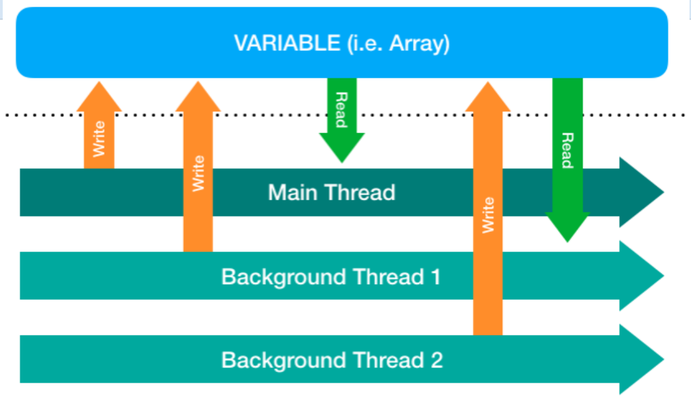
Variables accesibles por múltiples hilos en el mismo proceso, para comunicar información y coordinar tareas.

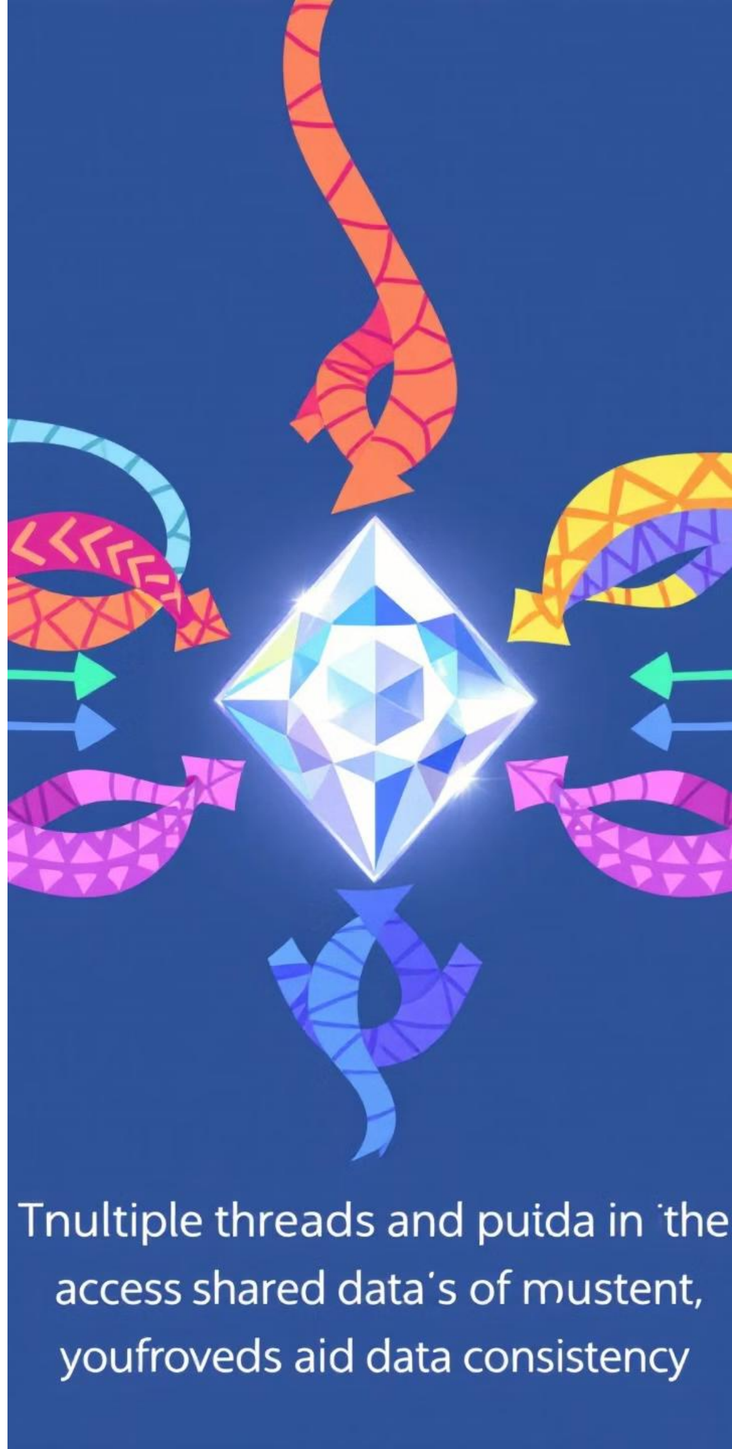
Desafíos

Condiciones de carrera (dos hilos hacen likes++ y se pierde uno) y problemas de visibilidad (un hilo cambia una variable y otro sigue viendo el valor antiguo).

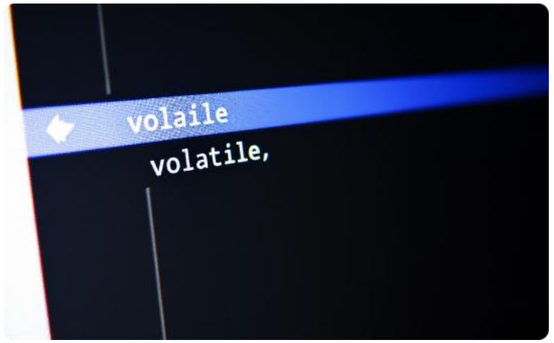
Importancia

Esencial garantizar que las operaciones con variables compartidas sean seguras y coherentes.





Soluciones para manejar variables compartidas



Palabra clave volatile

Garantiza que las actualizaciones de variables sean visibles inmediatamente para todos los hilos.



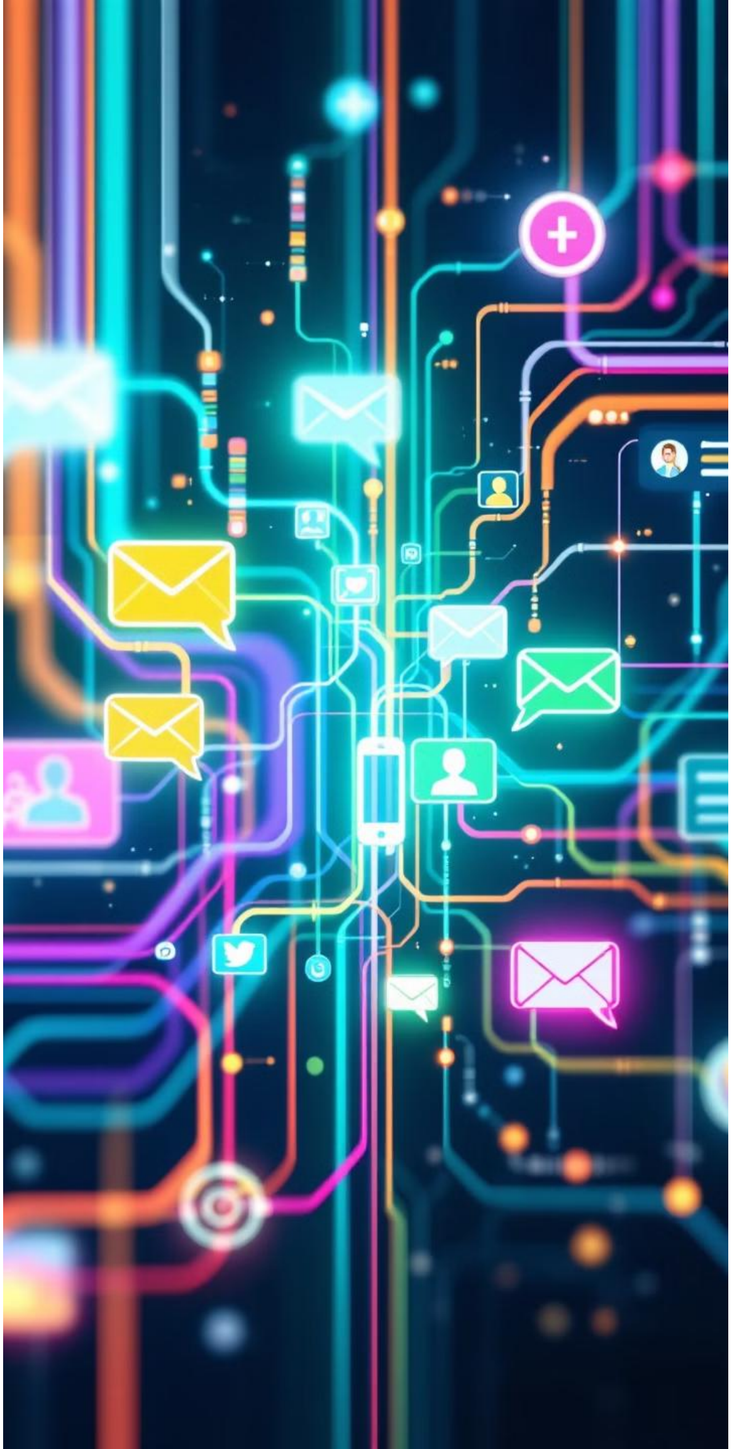
Clases atómicas

Proporcionan operaciones **seguras** para variables compartidas sin necesidad de sincronización.



Bloques sincronizados

Solo un hilo a la vez ejecuta el bloque. Es la opción cuando hay que actualizar **varias variables juntas** o hacer más de un paso seguido.



4.3. Canales de comunicación

1 Intercambio estructurado

Los canales de comunicación permiten que los hilos intercambien datos de manera organizada y eficiente.

2 Evitar problemas comunes

Proporcionan mecanismos seguros para la comunicación multihilo, evitando condiciones de carrera e inconsistencias de memoria.

3 Herramientas esenciales

Estas herramientas son fundamentales para la colaboración y el trabajo en paralelo entre hilos.

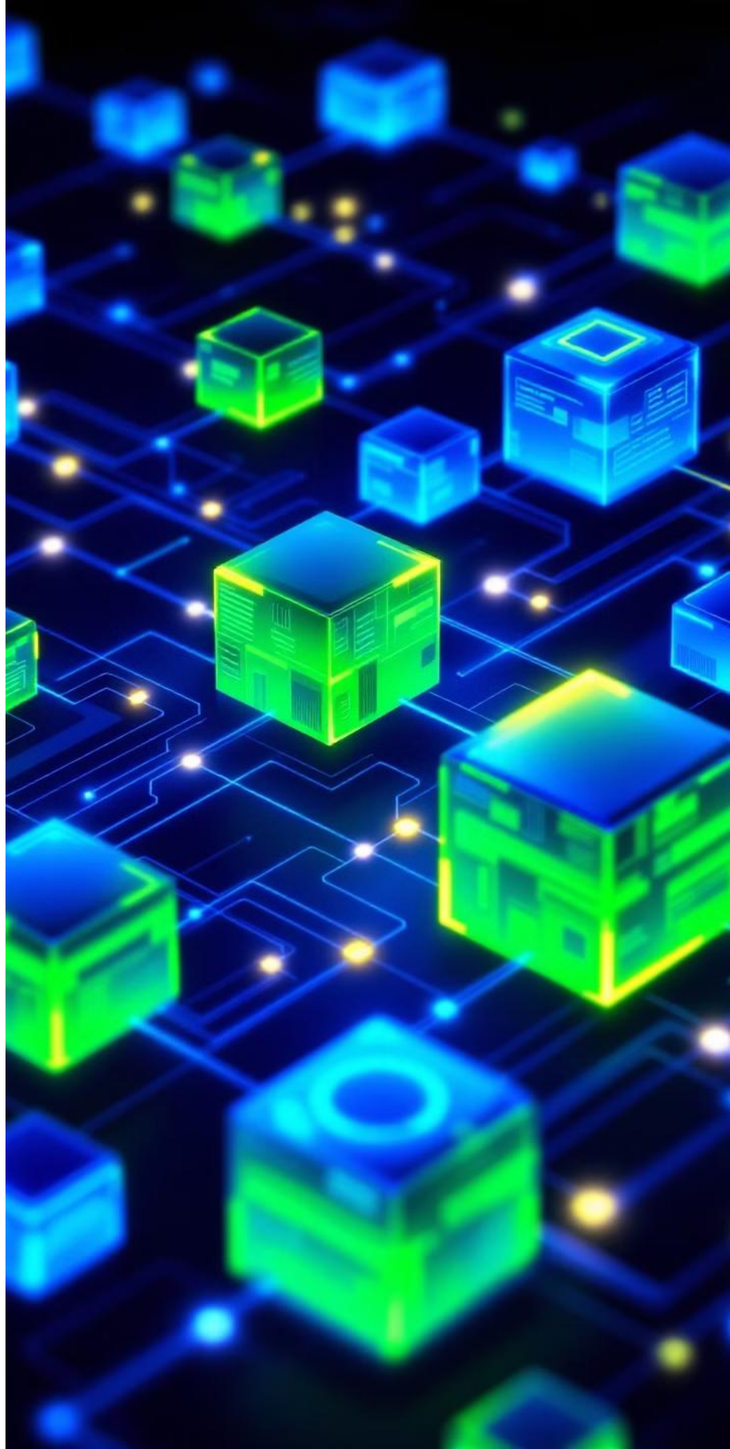
4.3. Canales de comunicación

Clases en Java para la comunicación entre hilos

Paquete java.util.concurrent

Proporciona clases diseñadas específicamente para facilitar la comunicación entre hilos.

- **BlockingQueue** (cola bloqueante): para productor y consumidor. *Es la que vamos a usar.*
- **Exchanger**: dos hilos se intercambian un objeto a la vez.
- **CyclicBarrier**: varios hilos esperan a estar todos en un punto antes de seguir.



4.3.1. Bloqueos y colas bloqueantes

Colas bloqueantes

Permiten a un hilo añadir elementos mientras otro los consume. Manejan automáticamente la sincronización entre hilos.

- **put(x)**: añade un elemento. Si la cola está llena, el hilo espera a que haya hueco.
- **take()**: saca un elemento. Si la cola está vacía, el hilo espera a que llegue uno.
- Por dentro hacen el wait() y el notify() que en el 4.1 escribíamos a mano.

Clases comunes

LinkedBlockingQueue (tamaño opcional), **ArrayBlockingQueue** (tamaño fijo) y **PriorityBlockingQueue** (saca primero el de mayor prioridad, y no tiene límite) son algunas implementaciones de colas bloqueantes.

Productor-Consumidor

Las colas bloqueantes son ideales para escenarios de productor-consumidor, donde se requiere sincronización entre hilos.

Ejemplo: Productor-Consumidor

Un ejemplo clásico de comunicación entre hilos es el problema del Productor-Consumidor, donde un hilo produce elementos y los coloca en una **cola bloqueante**, mientras que otro hilo los consume de la cola.

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class ProductorConsumidorCola {
    public static void main(String[] args) {
        BlockingQueue<Integer> cola = new ArrayBlockingQueue<>(2);   // caben 2

        Thread productor = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    cola.put(i);             // espera si la cola está llena
                    System.out.println("Productor: dejo " + i + "  (en cola: " + cola.size() + ")");
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

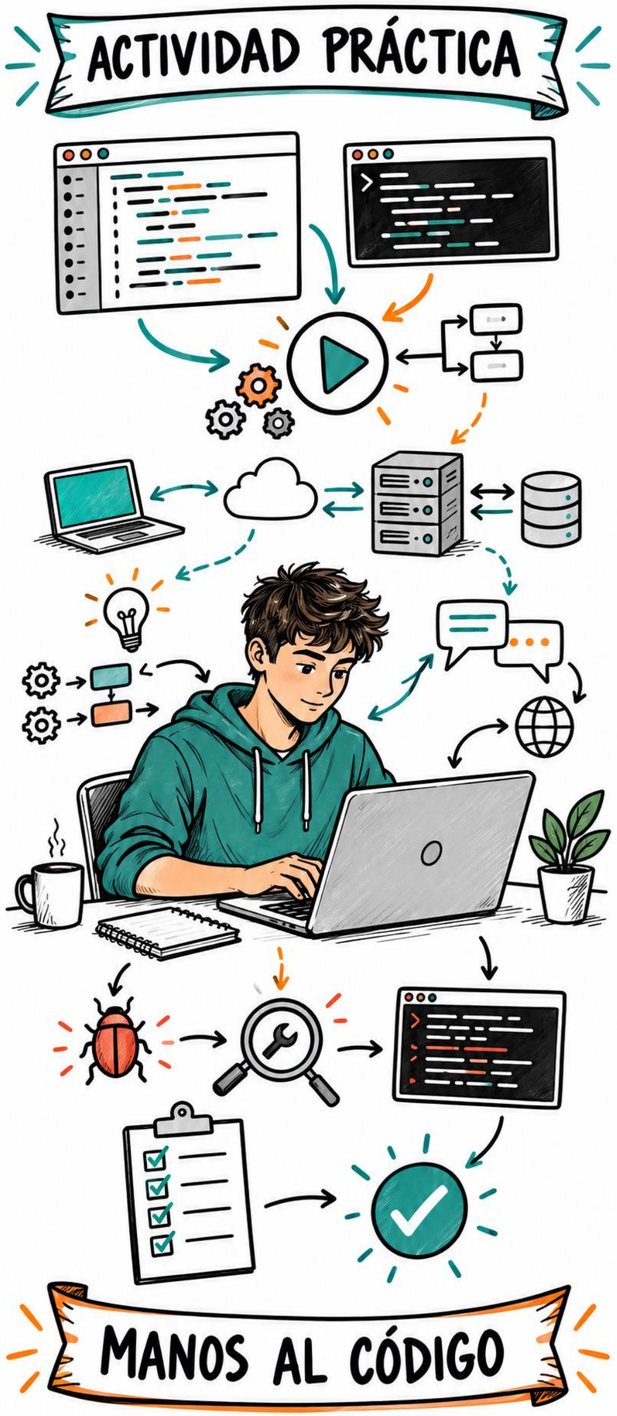
        Thread consumidor = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    Thread.sleep(1000);       // el consumidor es más lento
                    int valor = cola.take();   // espera si la cola está vacía
                    System.out.println("    Consumidor: recojo " + valor);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        productor.start();
        consumidor.start();
    }
}
```



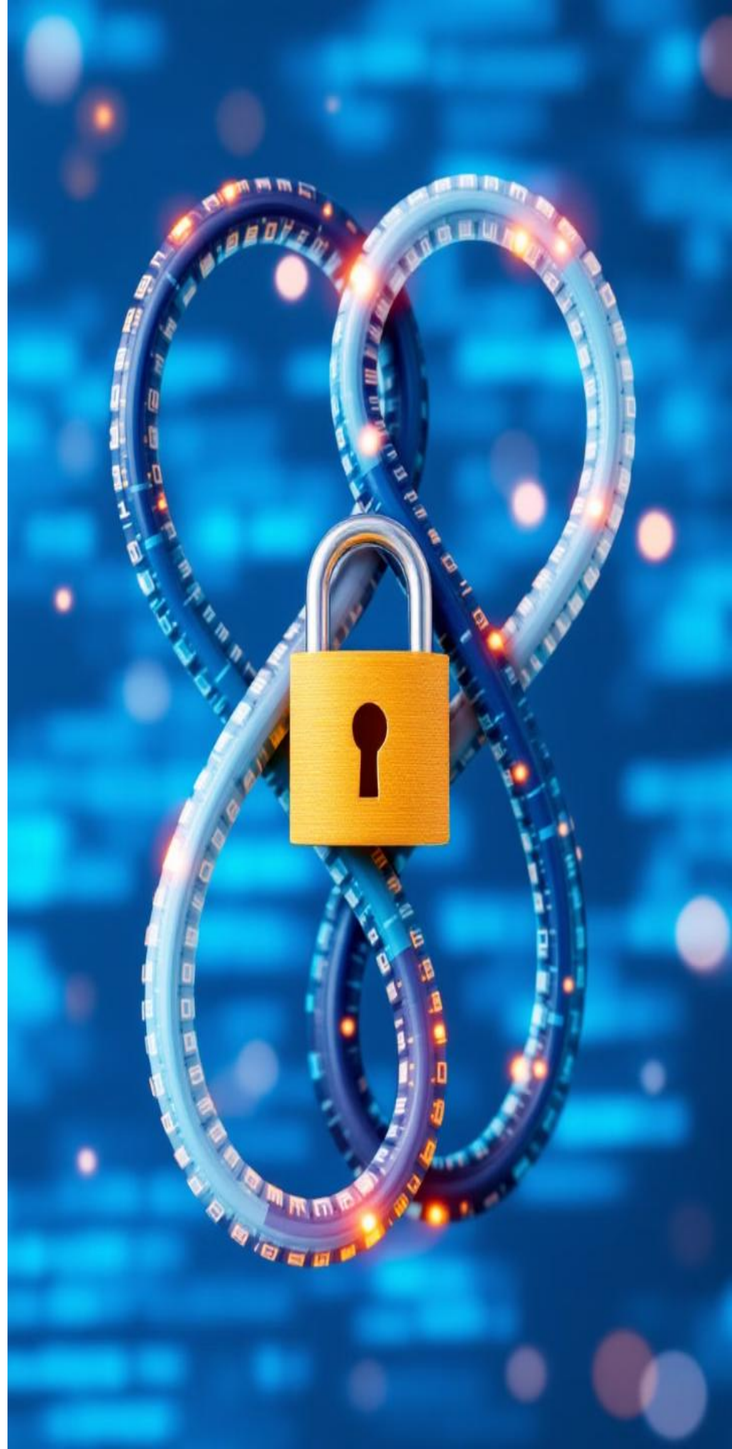
ACTIVIDAD 4

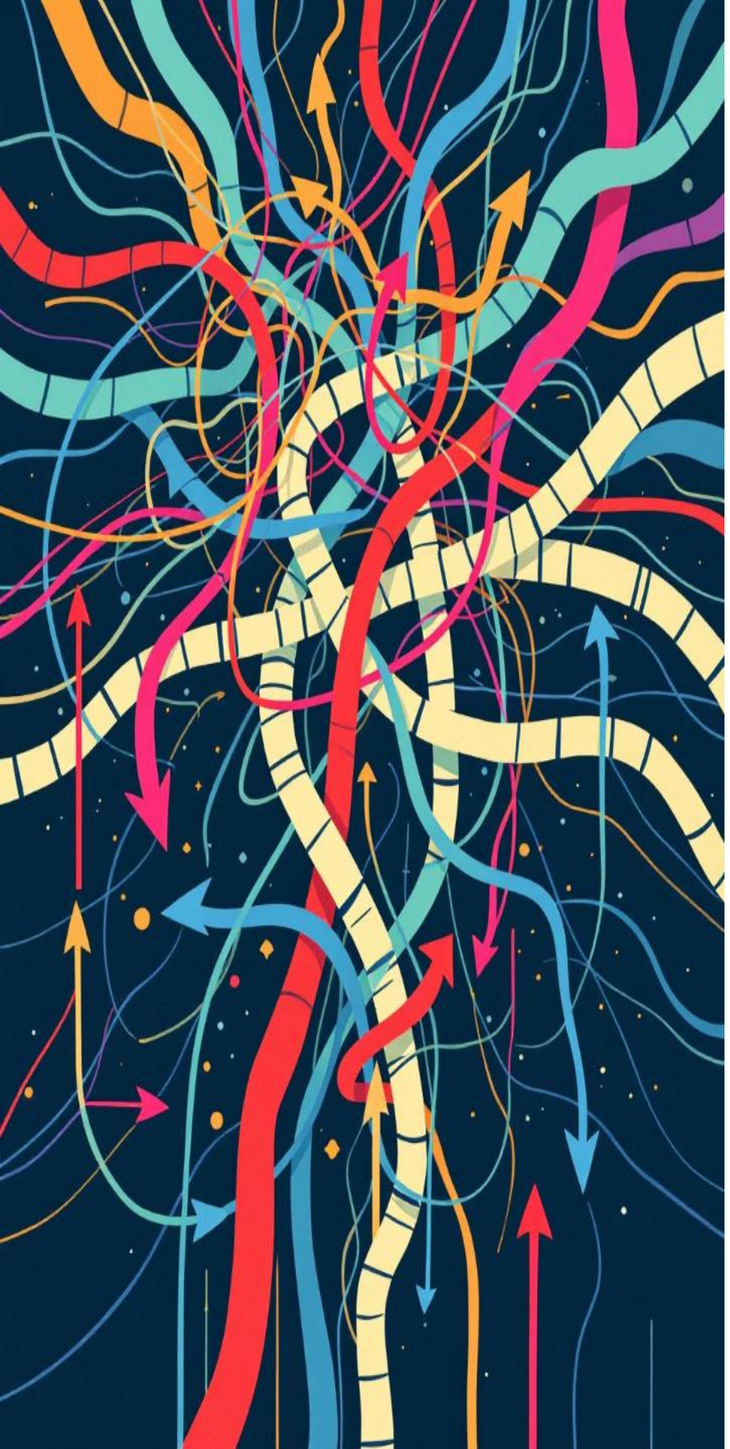
Comunicación entre hilos y Cola bloqueante



5. Sincronización de hilos

- Los hilos se comunican mediante el intercambio de información a través de variables y objetos en memoria.
- Cuando varios hilos manipulan a la vez objetos compartidos, pueden aparecer resultados erróneos o el programa puede quedarse paralizado.
- **La solución es la sincronización.**





5.1. Problemas comunes en la sincronización de hilos

Condición de carrera 

Cuando el resultado depende del orden de acceso a memoria compartida por varios hilos.

Inconsistencia de memoria 

Cuando un hilo cambia una variable y otro sigue viendo el valor antiguo.

Inanición 

Cuando un hilo nunca consigue el recurso, aunque este se libere, porque siempre se lo lleva otro hilo.

Interbloqueo 

Cuando dos o más hilos se bloquean mutuamente esperando recursos.

Bloqueo activo 

Como el interbloqueo, ningún hilo avanza, pero aquí no están parados: no dejan de reaccionar el uno al otro y gastan CPU sin progresar.

Condición de carrera

- Ocurre cuando el resultado de la ejecución de un programa depende del orden en que los hilos acceden a la memoria compartida.
- Sucede cuando varios hilos acceden a la vez a la misma variable y al menos uno la modifica. Ejemplo: dos hilos hacen likes++ sobre 100 y el resultado es 101, porque likes++ son tres pasos (leer, sumar, escribir) y uno se cuela en medio del otro.
- Este tipo de problema se presenta especialmente en **secciones críticas** del código que requieren un acceso exclusivo (**exclusión mutua**) para evitar conflictos entre hilos.



Inconsistencia de memoria

- Un hilo cambia una variable y otro hilo **sigue viendo el valor antiguo**.

- Ocurre porque cada núcleo trabaja con su propia **caché** y el compilador puede reordenar instrucciones: el cambio no siempre llega a la memoria compartida a tiempo.

- Ejemplo: un hilo pone parar = true y el otro hilo sigue en su while (!parar) para siempre.

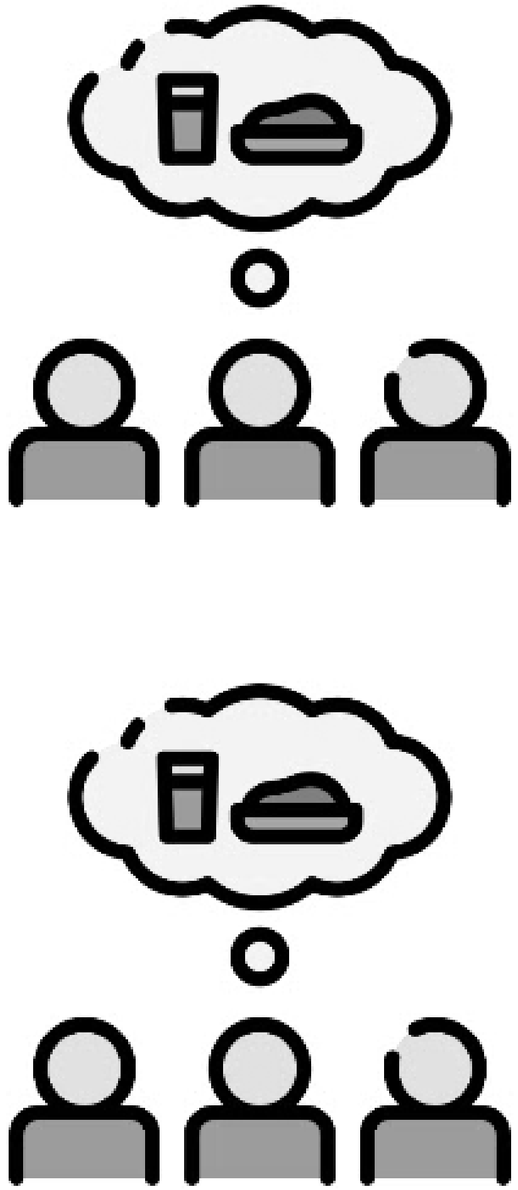
- Solución: **volatile** (si un hilo escribe y otros solo leen) o **synchronized** (que también garantiza que se vean los cambios).





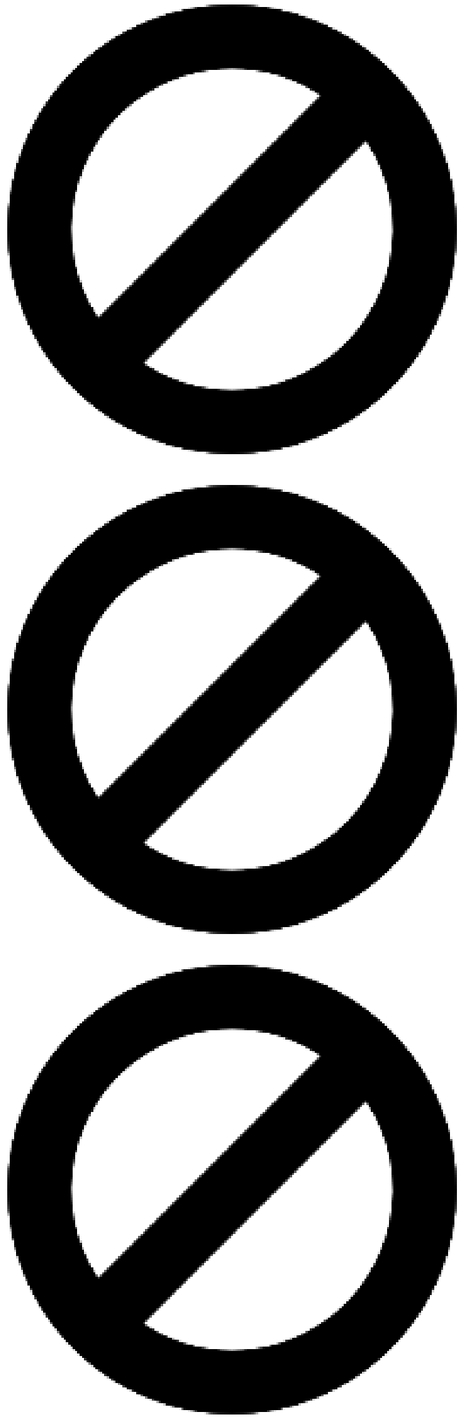
Inanición

- Ocurre cuando un hilo se ve constantemente **privado de acceder a un recurso compartido.**
- Ocurre porque el reparto no es justo: con synchronized, cuando se libera el cerrojo puede volver a cogerlo el mismo hilo una y otra vez, y otro hilo nunca entra. Es como la cola de una discoteca en la que siempre pasan otros antes que tú.
- Este problema es difícil de identificar y requiere atención especial en el diseño del programa para asegurar oportunidades justas de acceso a los recursos.



Interbloqueo (deadlock)

- Es un problema crítico que aparece cuando dos o más hilos se encuentran en un estado de espera indefinida porque están bloqueados en espera de un recurso que está en posesión de otro hilo, formando un ciclo de dependencia que impide que cualquiera de ellos avance.
- Este problema puede no presentarse en todas las ejecuciones, pero cuando ocurre, los hilos quedan completamente bloqueados sin posibilidad de reanudar su actividad.
- Cómo evitarlo: que **todos los hilos pidan los recursos en el mismo orden**, **retener los recursos el menor tiempo posible y no esperar a otros hilos** mientras se tiene un recurso que ellos necesitan.



Bloqueo activo

- El bloqueo activo es similar al interbloqueo, pero los hilos afectados no están completamente bloqueados.
- Cada hilo cede para dejar pasar al otro, y el otro hace lo mismo a la vez. Es como dos personas en un pasillo que se apartan siempre hacia el mismo lado: se mueven todo el rato, pero ninguna pasa.
- Ningún hilo progresa, igual que en el interbloqueo. La diferencia es que aquí los hilos **no están parados**: siguen ejecutándose (RUNNABLE) y **gastan CPU** sin hacer nada útil.



5.2. Mecanismos de sincronización de hilos



Evitar problemas

Permiten evitar problemas de sincronización como condiciones de carrera e inconsistencias de memoria.



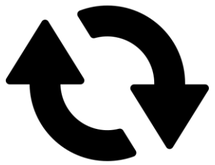
Controlar el acceso

Mecanismos que garantizan la **exclusión mutua**: solo un hilo a la vez dentro de la sección crítica.



Ejecución asíncrona

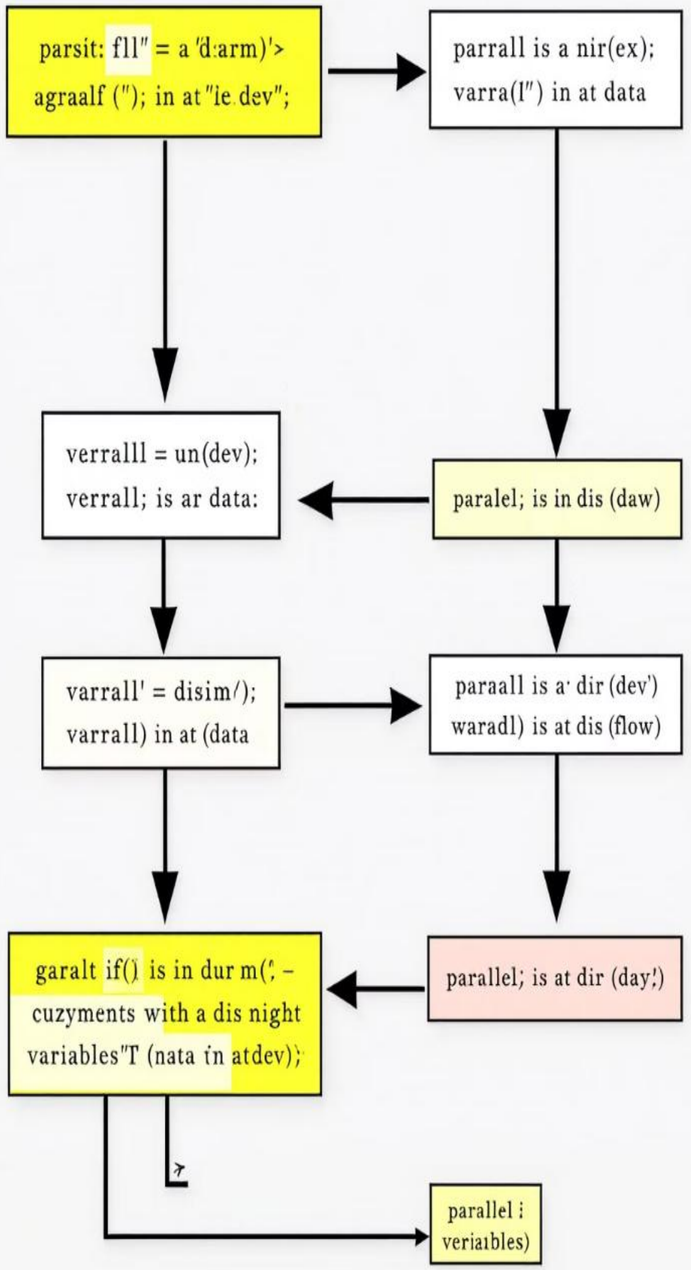
Los hilos pueden trabajar en paralelo cuando no comparten recursos críticos.



Ejecución síncrona

Los hilos deben sincronizar su ejecución cuando acceden a datos compartidos.

En Java: **synchronized, wait()/notify(), Lock, Semaphore, clases atómicas** y las clases de **java.util.concurrent**.



5.2.1. Condiciones de Bernstein

Los segmentos que cumplen estas condiciones pueden ejecutarse de forma asíncrona. Para aquellos que no las cumplen, se deben usar mecanismos de sincronización.

1 Dependencia de flujo

Las variables de entrada del segmento j deben ser distintas de las variables de salida del segmento i (evitas que j dependa de i).

2 Antidependencia

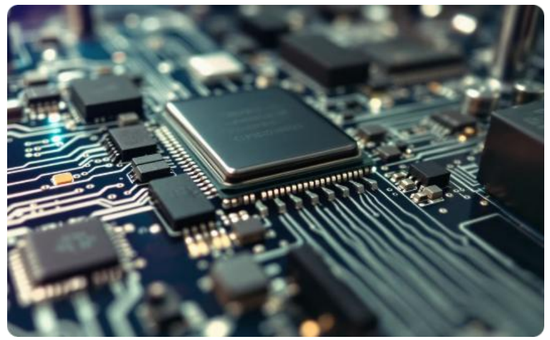
Las variables de entrada del segmento i deben ser diferentes de las de salida de j (aseguras que i no depende de resultados de j).

3 Dependencia de salida

Las variables de salida de i deben ser diferentes de las de salida de j (así el resultado no depende del último segmento que se ejecute).

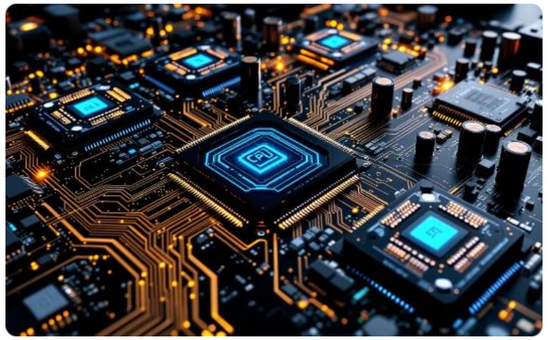
5.2.2. Operación atómica

Es una operación que se realiza en su totalidad sin ser interrumpida



Atomicidad en sistemas monoprocesador

En sistemas de un solo procesador, una operación que se ejecuta en una sola instrucción de CPU se considera atómica.
Una sola línea de Java puede ser varias instrucciones de CPU: x++ no es atómica ni siquiera con un solo núcleo.



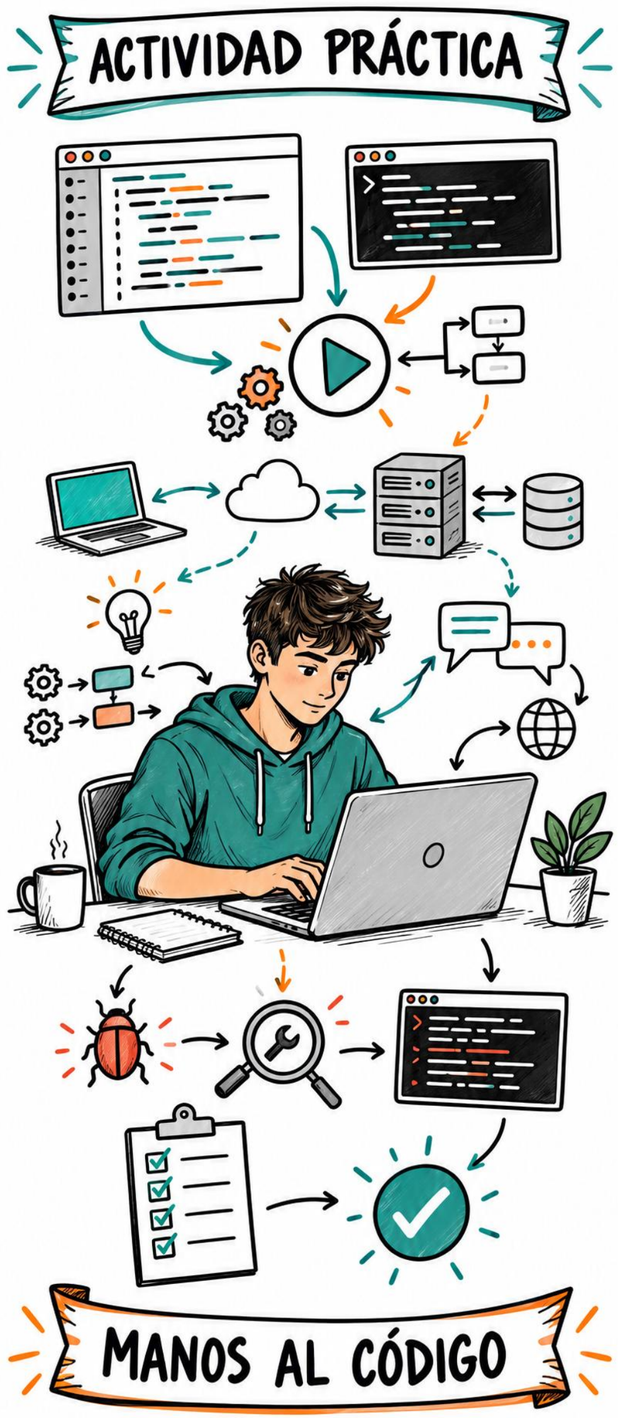
Desafíos en sistemas multiprocesador

En sistemas multiprocesador, asegurar la atomicidad es más complicado debido al acceso simultáneo de múltiples hilos a los mismos datos.



Atomicidad en Java

Leer o escribir un int, un boolean o una referencia es atómico. Pero contador++ no lo es, porque son tres pasos, aunque la variable sea volatile. Para operaciones atómicas se usan AtomicInteger, AtomicLong… o synchronized.



ACTIVIDAD 5

Sincronización de hilos
ATOMIC

5.2.3. Sección crítica

Es un fragmento de código en el que se accede de manera controlada a variables o recursos compartidos.

Requisitos para resolver el problema:

Exclusión mutua

Solo un hilo puede acceder a la sección crítica a la vez.

Progreso

Si nadie está en la sección crítica y algún hilo quiere entrar, alguno tiene que poder entrar: no se puede dejar la puerta cerrada sin motivo.

Espera limitada

Un hilo que pide entrar no puede quedarse esperando para siempre: hay un límite de veces que otros pueden colarse antes que él.

Java proporciona monitores y semáforos



5.2.4. Semáforos

Definición

Los semáforos son una herramienta de sincronización que controla el acceso a recursos con un número limitado de instancias.

Operaciones

Un semáforo es un contador de permisos con dos operaciones. **Coger un permiso**: si el contador es 0, el hilo espera; si no, lo resta y pasa. **Soltar un permiso**: suma 1 y, si había alguien esperando, lo deja pasar. En Java son **acquire()** y **release()**. No hay que confundirlas con el wait() de Object.

Implementación en Java

La clase **Semaphore** en java.util.concurrent permite implementar semáforos, incluyendo semáforos binarios (mutexes).
Con un solo permiso, new Semaphore(1), funciona como un cerrojo: solo pasa un hilo.





5.2.4. Clase Semáforo (métodos)

Método	Devuelve	Qué hace
`new Semaphore(int permisos)`	*(constructor)*	Crea el semáforo con ese número de permisos.
`acquire()`	`void`	Coge un permiso. Si no queda ninguno, el hilo **espera**.
`release()`	`void`	Devuelve un permiso y deja pasar a un hilo que esperaba en `acquire()`.
`tryAcquire()`	`boolean`	Intenta coger un permiso **sin esperar**: devuelve `true` si lo consigue y `false` si no.
`availablePermits()`	`int`	Indica cuántos permisos quedan libres.

5.2.4. Semáforos

Ejemplo

```java
import java.util.concurrent.Semaphore;

public class SemaforoEjemplo {

    // Creación de un semáforo con 2 permisos (solo 2 hilos pueden acceder al recurso a la vez)
    private static Semaphore semaforo = new Semaphore(2);

    public static void main(String[] args) {
        // Crear e iniciar 5 hilos que intentarán acceder al recurso limitado
        for (int i = 1; i <= 5; i++) {
            new Thread(new Tarea(i)).start();
        }
    }

    // Clase que representa una tarea que intentará acceder al recurso
    static class Tarea implements Runnable {
        private int id;

        public Tarea(int id) {
            this.id = id;
        }

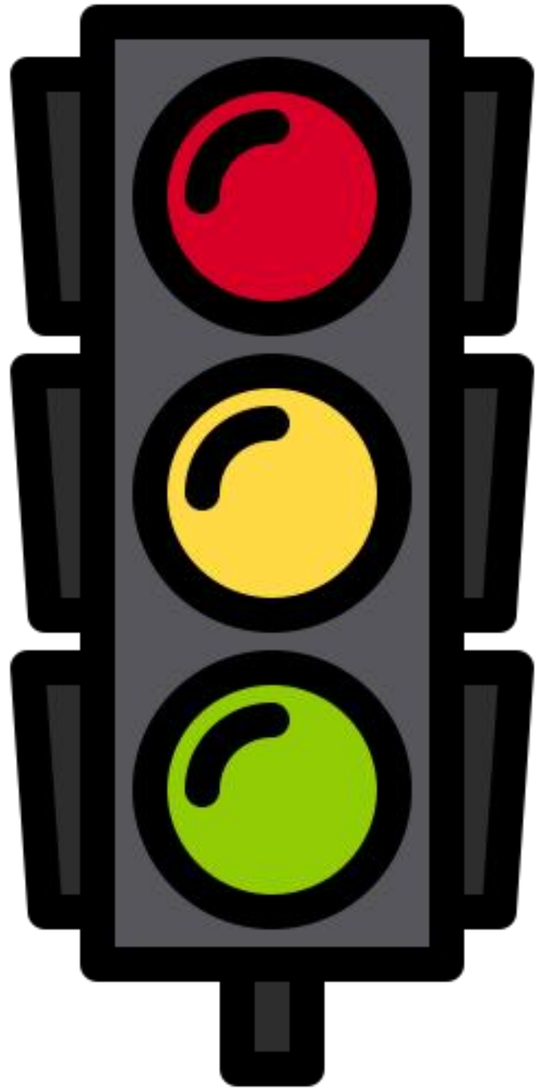
        @Override
        public void run() {
            try {
                System.out.println("Hilo " + id + " intenta acceder al recurso.");

                // Adquiere un permiso del semáforo para acceder al recurso
                semaforo.acquire();
                System.out.println("Hilo " + id + " ha adquirido acceso al recurso.");

                // Simulación del uso del recurso por un tiempo
                Thread.sleep(2000);

                // Libera el permiso para que otros hilos puedan acceder
                System.out.println("Hilo " + id + " libera el recurso.");
                semaforo.release();

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```



5.2.5. Monitores



Exclusión mutua

Los monitores permiten que solo un hilo ejecute un método dentro de ellos a la vez, garantizando la exclusión mutua.



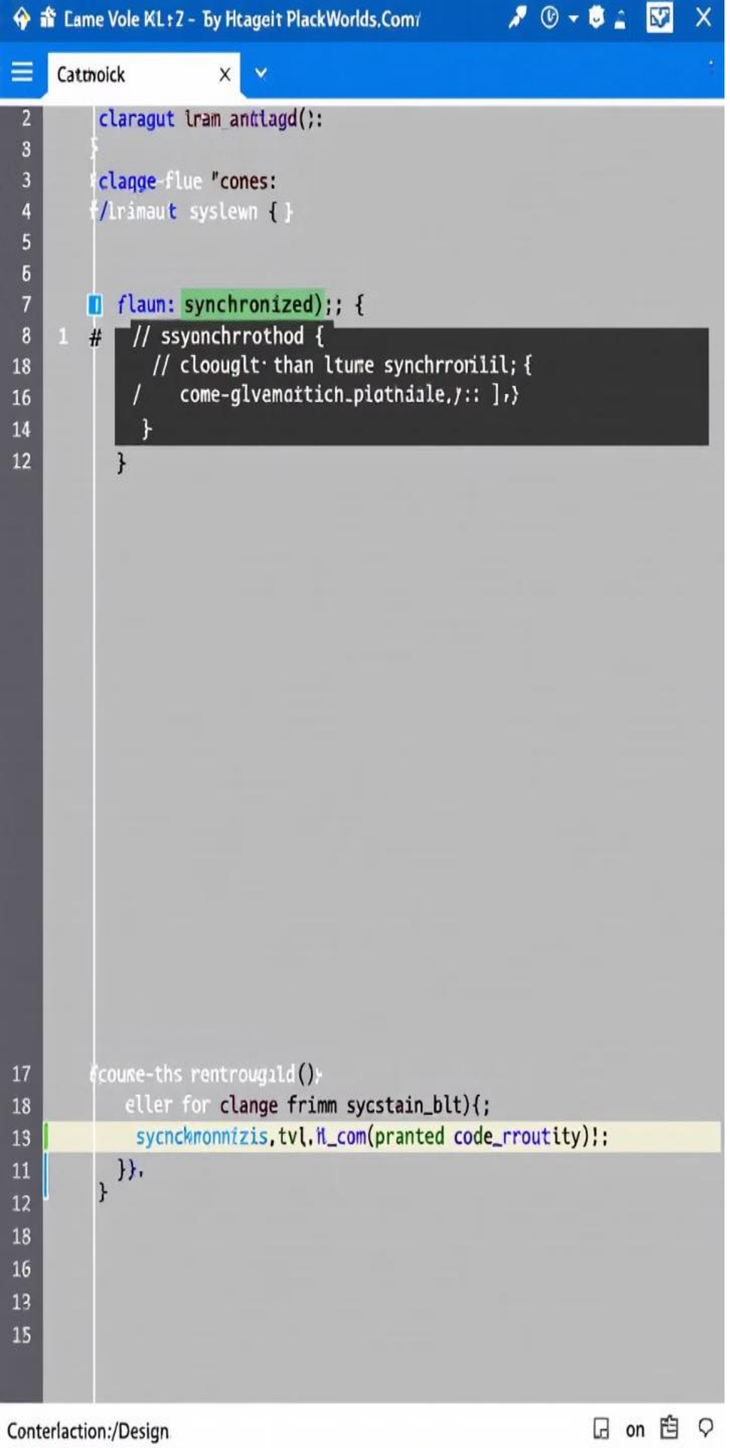
Implementación en Java

En Java, los monitores se implementan fácilmente usando la palabra clave synchronized. **Cada objeto de Java tiene un monitor** (un cerrojo interno). **synchronized** lo usa: un método synchronized bloquea el objeto (this) y un bloque synchronized (obj) bloquea obj. Además, el monitor trae de serie wait() y notify().



Simplifica la Sincronización

Los monitores permiten a los programadores declarar secciones críticas sin preocuparse por la implementación a nivel de sistema operativo.



Formas de utilizar monitores en Java

Métodos sincronizados

Declarar un método como synchronized asegura que solo un hilo pueda ejecutarlo a la vez.

```
public synchronized void incrementar() {
    cuenta++;
}
```

Bloques de código sincronizados

Sincronizar solo una sección específica de código en lugar de todo el método.

```
public void incrementar() {
    // código que no necesita protección
    synchronized (this) { cuenta++; }
}
```







5.2.5. Monitores

Ejemplo

```java
public class MonitorEjemplo {

    // Clase Contador con métodos sincronizados
    static class Contador {
        private int cuenta = 0;

        // Método sincronizado para incrementar el contador
        public synchronized void incrementar() {
            cuenta++;
            System.out.println(Thread.currentThread().getName() + " ha incrementado el contador a " + cuenta);
        }
        // Método para obtener el valor actual del contador
        public synchronized int getCuenta() {
            return cuenta;
        }
    }
    // Clase que representa una tarea que incrementa el contador
    static class IncrementarTarea implements Runnable {
        private Contador contador;

        public IncrementarTarea(Contador contador) {
            this.contador = contador;
        }
        @Override
        public void run() {
            for (int i = 0; i < 5; i++) {
                // ya está sincronizado por dentro
                       contador.incrementar();

                try {
                    Thread.sleep(100); // Simula trabajo y permite cambio de contexto
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }
    }

    public static void main(String[] args) {
        Contador contador = new Contador();
        // Crear e iniciar dos hilos que incrementan el contador
        Thread hilo1 = new Thread(new IncrementarTarea(contador), "Hilo 1");
        Thread hilo2 = new Thread(new IncrementarTarea(contador), "Hilo 2");

        hilo1.start();
        hilo2.start();

        try {
            hilo1.join();
            hilo2.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("Valor final del contador: " + contador.getCuenta());
    }
}
```

5.2.6. Condiciones de espera y notificación

Repaso del 4.1: **wait() y notify() son parte del monitor**.

1 wait()

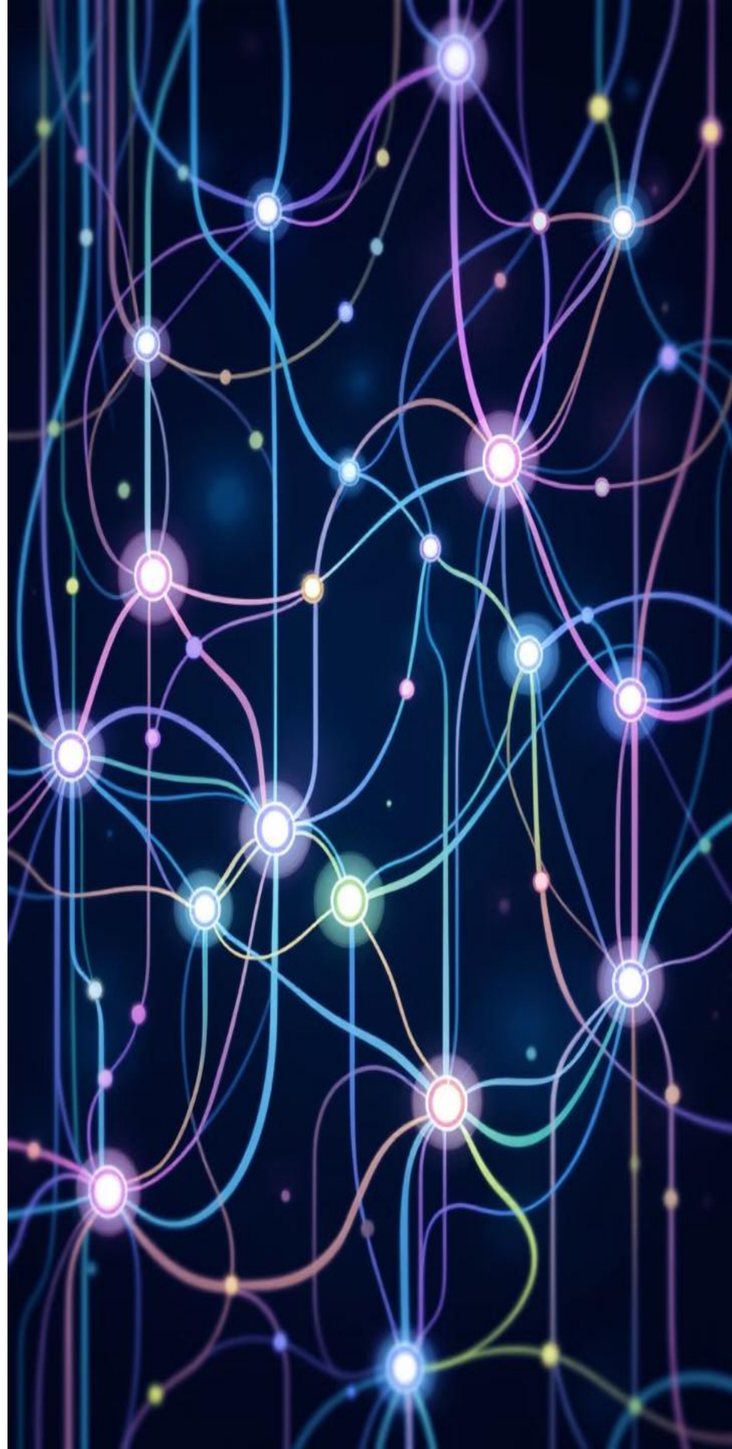
Suspende un hilo dentro de la sección crítica hasta que reciba una notificación de otro hilo.

2 notify()

Despierta a **uno** de los hilos que esperan en ese objeto (notifyAll() despierta a todos). El hilo despertado vuelve a coger el cerrojo y **comprueba otra vez la condición** en su while.

3 Uso en bloques sincronizados

Deben usarse dentro de synchronized sobre el mismo objeto: si no, Java lanza IllegalMonitorStateException. Además, wait() suelta el cerrojo mientras espera, para que otro hilo pueda entrar y cambiar la condición.



5.2.6. Condiciones de espera y notificación

Ejemplo

```java
public class CondicionesEsperaNotificacion {

    private static final Object monitor = new Object();
    private static boolean datoListo = false;

    public static void main(String[] args) {
        // Crear el hilo Consumidor
        Thread consumidor = new Thread(() -> {
            synchronized (monitor) {
                while (!datoListo) {
                    try {
                        System.out.println("Consumidor espera a que el dato esté listo...");
                        monitor.wait(); // Espera hasta recibir una notificación
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
                System.out.println("Consumidor ha recibido el dato y está procesándolo.");
            }
        });
        // Crear el hilo Productor
        Thread productor = new Thread(() -> {
            synchronized (monitor) {
                System.out.println("Productor está generando el dato...");
                datoListo = true;
                monitor.notify(); // Notifica al consumidor de que el dato está listo
                System.out.println("Productor ha notificado al consumidor.");
            }
        });
        consumidor.start();
        productor.start();
    }
}
```



5.3. Mecanismos de sincronización avanzados

Java proporciona varias clases en el paquete **java.util.concurrent** para manejar sincronización avanzada en aplicaciones multihilo, como controlar el acceso a recursos compartidos, coordinar el intercambio de datos o asegurar que varios hilos alcancen un estado común.





5.3.1. Clase Semáforo

Apartado 5.2.4

5.3.2. Clase Exchanger

Intercambio sincronizado

Permite sincronizar dos hilos e intercambiar objetos entre ellos.

Métodos principales

exchange(V x) y exchange(V x, long timeout, TimeUnit unit) para intercambiar datos.

Ejemplo de Uso

Un productor genera datos y los intercambia con un consumidor que los procesa.



5.3.2. Clase Exchanger

Ejemplo de Uso

Un productor genera datos y los intercambia con un consumidor que los procesa.

```java
import java.util.concurrent.Exchanger;

public class Intercambio {
    public static void main(String[] args) {
        Exchanger<String> exchanger = new Exchanger<>();

        Thread productor = new Thread(() -> {
            try {
                String recibido = exchanger.exchange("Datos generados");
                System.out.println("Productor recibe: " + recibido);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumidor = new Thread(() -> {
            try {
                String recibido = exchanger.exchange("Recibido, gracias");
                System.out.println("Consumidor recibe: " + recibido);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        productor.start();
        consumidor.start();
    }
}
```



5.3.3. Clase CountDownLatch

Descripción

Permite que un hilo o grupo de hilos esperen hasta que otro conjunto de hilos haya completado sus tareas.

Funcionamiento

Se crea con un contador, por ejemplo new CountDownLatch(3). Cada countDown() le resta 1, y cuando llega a 0 se abre y los que esperaban en await() continúan.
Es de un solo uso: una vez abierto, no se puede volver a cerrar… (es lo que lo diferencia de CyclicBarrier)

Métodos principales

- **await()**: Bloquea el hilo hasta que el contador llega a cero.
- **countDown()**: Decrementa el contador en uno.

5.3.3. Clase CountDownLatch

Ejemplo de Uso

Un proceso principal espera a que varios subprocesos completen tareas de inicialización antes de proceder.

```java
import java.util.concurrent.CountDownLatch;

public class ServiceInitialization {
    private static final int NUM_SERVICES = 3;
    private static final CountDownLatch latch = new CountDownLatch(NUM_SERVICES);

    static class Service implements Runnable {
        @Override
        public void run() {
            try {
                System.out.println(Thread.currentThread().getName() + " inicializando...");
                Thread.sleep(1000); // Simulación de trabajo
                System.out.println(Thread.currentThread().getName() + " listo.");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                latch.countDown(); // Servicio completo. Se avisa siempre, aunque falle
            }
        }
    }

    public static void main(String[] args) {
        for (int i = 0; i < NUM_SERVICES; i++) {
            new Thread(new Service()).start();
        }

        try {
            latch.await(); // Espera a que todos los servicios estén listos
            System.out.println("Todos los servicios están inicializados. Iniciando aplicación principal.");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

5.3.4. Clase CyclicBarrier

Permite que varios hilos esperen hasta que todos alcancen un punto común

Sincronización repetida

Permite que varios hilos se sincronicen repetidamente en un punto común, conocido como barrera.

Método await()

Hace que el hilo actual espere hasta que todos los hilos alcanzan la barrera.

Reutilizable

Cuando pasan todos, la barrera **se reinicia sola** y sirve para el siguiente paso (por eso es «cíclica»). Al crearla se le puede dar una tarea que se ejecuta cada vez que se abre. **reset()** solo se usa para forzar el reinicio en caso de error.



5.3.4. Clase CyclicBarrier

Ejemplo

Aplicación que realiza una serie de pasos en paralelo y, en cada paso, necesita que todos los hilos completen su tarea antes de pasar al siguiente.

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class TareasPorPasos {
    private static final int HILOS = 3;
    private static final CyclicBarrier barrera = new CyclicBarrier(HILOS,
            () -> System.out.println("--- Todos han llegado. Siguiente paso ---"));

    public static void main(String[] args) {
        for (int i = 1; i <= HILOS; i++) {
            new Thread(() -> {
                try {
                    for (int paso = 1; paso <= 3; paso++) {
                        System.out.println(Thread.currentThread().getName() + " termina el paso " + paso);
                        barrera.await();       // espera a los demás; la barrera se reinicia sola
                    }
                } catch (InterruptedException | BrokenBarrierException e) {
                    Thread.currentThread().interrupt();
                }
            }, "Hilo-" + i).start();
        }
    }
}
```



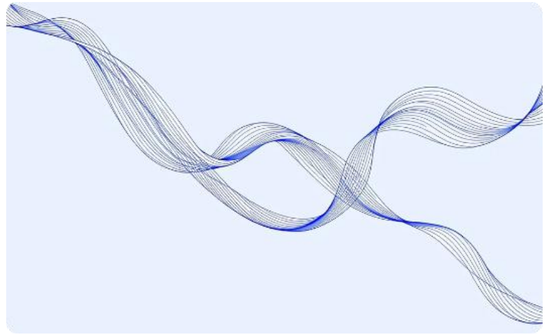
5.4. Buenas prácticas y estrategias para la sincronización de hilos

- Si bien la sincronización es esencial para evitar problemas de concurrencia, **su uso excesivo puede reducir el rendimiento y causar problemas de bloqueo.**

- Es importante encontrar un equilibrio y aplicar estrategias eficientes.

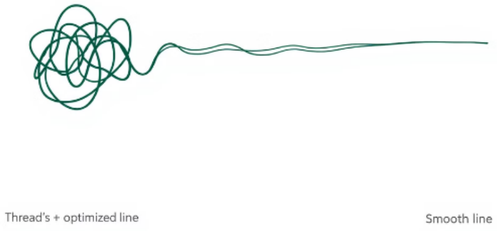


5.4.1. Minimizar el uso de sincronización



Reducir sobrecarga

La sincronización añade sobrecarga a la ejecución, por lo que es importante minimizar su uso para optimizar el rendimiento.



Evitar bloqueos

Reducir la sincronización ayuda a prevenir problemas como interbloqueos e inanición que pueden afectar la estabilidad del sistema.

Mejorar escalabilidad

Al minimizar la sincronización, los sistemas pueden escalar mejor y manejar una mayor carga de trabajo de manera eficiente.

Estrategias para minimizar la sincronización



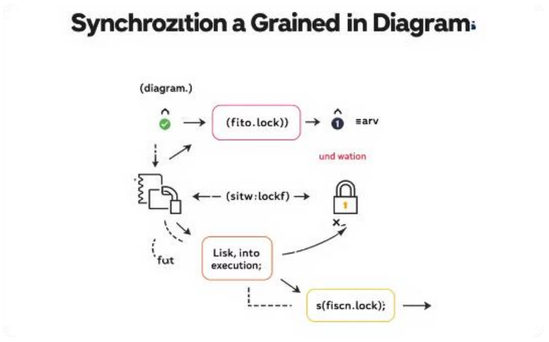
Evitar sincronización innecesaria

Reducir el uso de secciones críticas al mínimo necesario. Si un recurso no es compartido, no requiere sincronización.



Estructuras de datos seguras para hilos

Utilizar colecciones concurrentes de Java como ConcurrentHashMap y CopyOnWriteArrayList para simplificar la sincronización.



Sincronización de grano fino

Dividir las secciones que requieren sincronización en partes más pequeñas para permitir mayor paralelismo.

5.4.2. Estrategias para evitar interbloqueo

1 Orden predefinido

Establecer un orden estricto para adquirir bloqueos y evitar que dos hilos intenten acceder a los mismos recursos en orden diferente.

2 No esperar con recursos cogidos

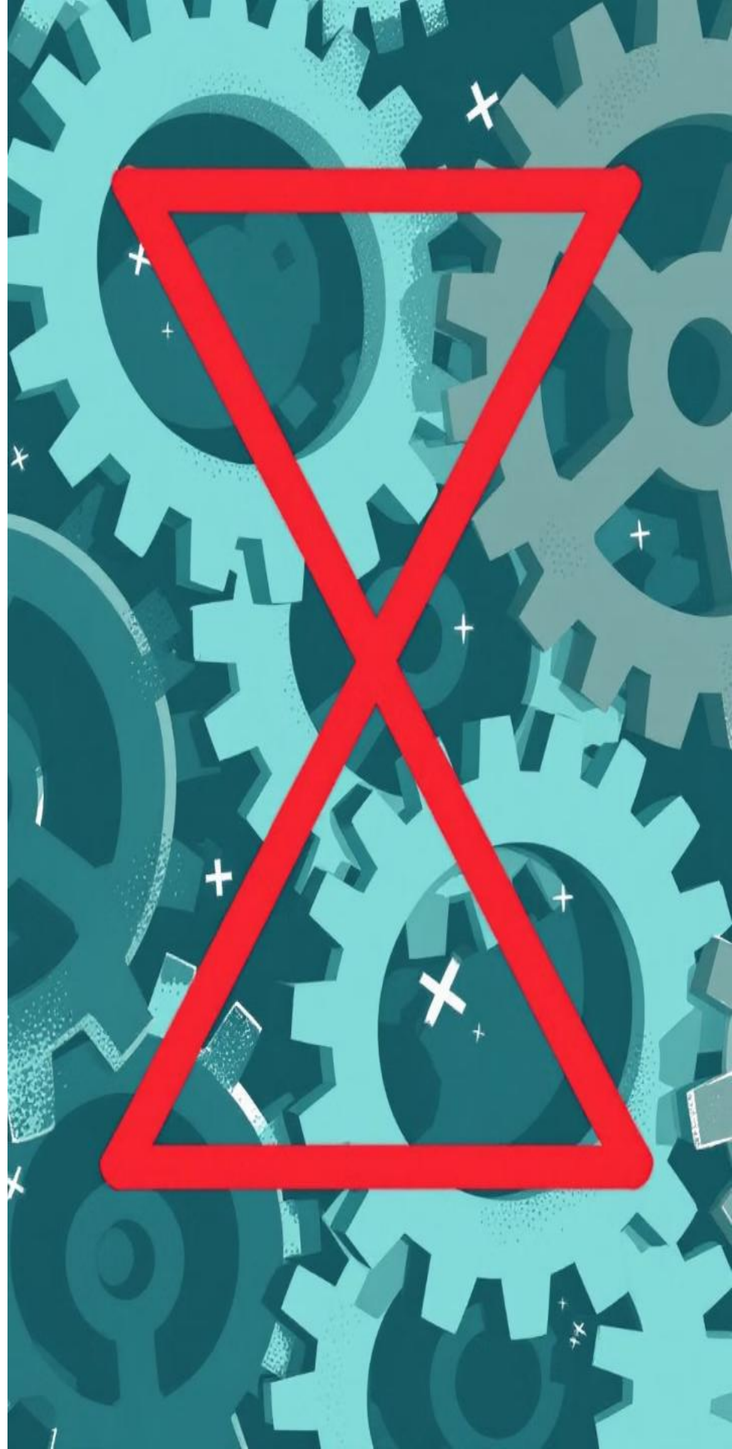
No esperar a otros hilos (await, join, acquire…) mientras tienes un recurso que ellos necesitan. Primero lo sueltas y después esperas

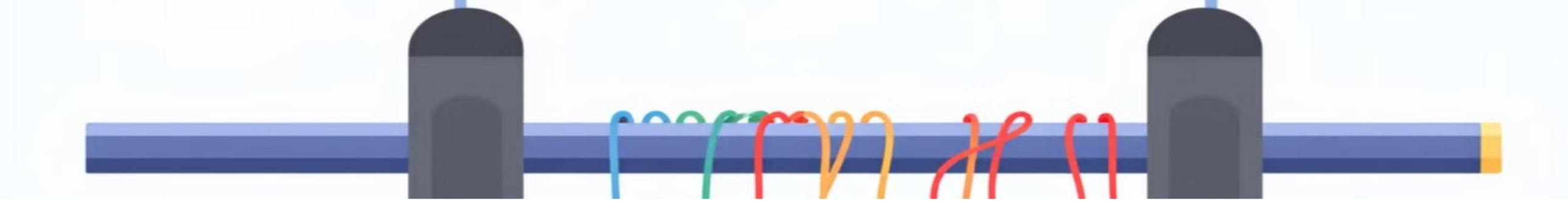
3 Minimizar tiempo de bloqueo

Mantener los bloqueos solo el tiempo estrictamente necesario para que otros hilos puedan acceder a los recursos más rápidamente.

4 Usar temporizadores

Utilizar métodos de sincronización con tiempos de espera, como tryLock, para evitar que los hilos esperen indefinidamente y generen interbloqueos prolongados.





5.4.3. Estrategias para evitar inanición

Prioridades equilibradas

No confiar en las prioridades para repartir recursos: en Java son solo una sugerencia al sistema operativo y no garantizan un reparto justo.

Mecanismos de sincronización justos

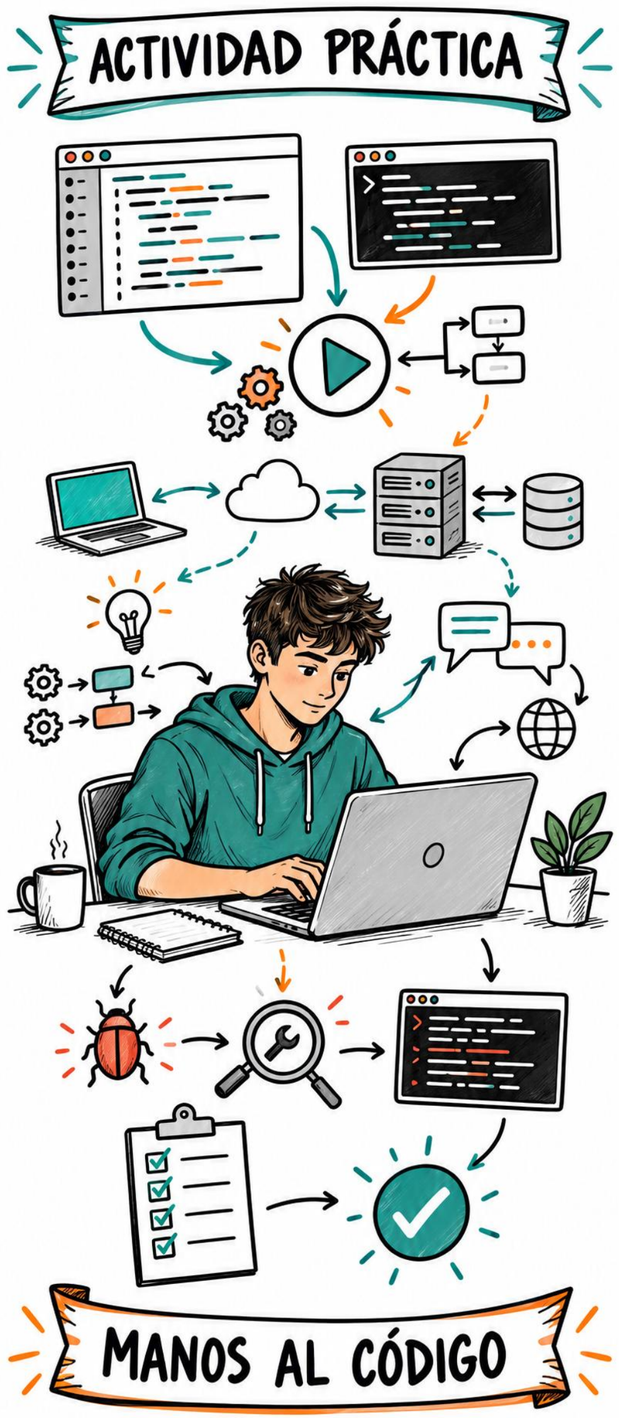
Utilizar modos "justos" (fair mode) en mecanismos como ReentrantLock para otorgar acceso a recursos en el orden en que se solicitaron.

Retroceso y reintento

Si un hilo no consigue el recurso, espera un tiempo aleatorio antes de reintentar. Es también la solución al bloqueo activo: así las dos personas del pasillo no se apartan siempre a la vez.

Semáforos justos

Crear el semáforo en modo justo, **new Semaphore(2, true)**, para que los permisos se den por orden de llegada, como en una cola.



ACTIVIDAD 6

Sincronización de hilos
SEMAPHORE

PRIORIDADES DE LOS HILOS

UNA SUGERENCIA, NO UNA ORDEN

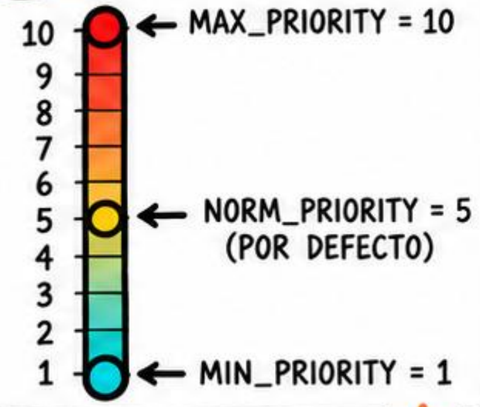
1 ¿QUÉ ES LA PRIORIDAD?

UNA SUGERENCIA AL SISTEMA OPERATIVO SOBRE LA IMPORTANCIA DE UN HILO

EL PLANIFICADOR PUEDE TENERLA EN CUENTA... O NO



2 NIVELES EN JAVA



3 SE HEREDA

MAIN NACE CON 5

CADA HILO HEREDA LA PRIORIDAD DE QUIEN LO CREA



4 MÉTODOS

```
hilo.getPriority();
hilo.setPriority(Thread.MAX_PRIORITY);
```

FUERA DE 1–10 → IllegalArgumentException

5 ¿CUÁNDO SE NOTA?



NÚCLEOS LIBRES: LA PRIORIDAD NO INFLUYE

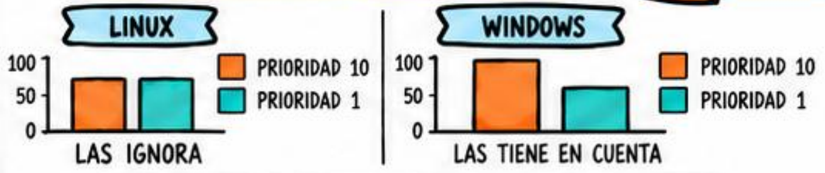
CPU SATURADA (MÁS HILOS QUE NÚCLEOS): SÍ PUEDE INFLUIR

NUNCA GARANTIZA UN ORDEN

6 EL EXPERIMENTO

DOBLE DE HILOS QUE NÚCLEOS · MITAD PRIORIDAD 10 · MITAD PRIORIDAD 1 · 3 SEGUNDOS

¿QUIÉN TRABAJA MÁS?



MISMO PROGRAMA, RESULTADO DISTINTO

7 BUENAS PRÁCTICAS

- ✓ EL PROGRAMA DEBE FUNCIONAR EN CUALQUIER ORDEN
- ✓ SUBIR LO QUE EL USUARIO NOTA (INTERFAZ, AUDIO)
- ✓ BAJAR LO QUE PUEDE ESPERAR (COPIAS DE SEGURIDAD)
- ✓ INANICIÓN → MECANISMOS JUSTOS, NO MÁS PRIORIDAD

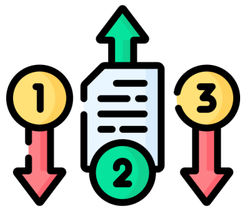
8 ERRORES QUE EVITAR

- ✗ USAR LA PRIORIDAD PARA CONTROLAR EL ORDEN → USA join() Y SINCRONIZACIÓN
- ✗ CREER QUE ES PORTABLE: CADA SISTEMA LA APLICA A SU MANERA
- ✗ CONTAR CON ELLA PARA CUMPLIR PLAZOS: JAVA NO ES DE TIEMPO REAL

UN PROGRAMA CORRECTO NUNCA DEPENDE DE LAS PRIORIDADES

6. Prioridades de los hilos

La prioridad es una **sugerencia** que damos al sistema operativo sobre la importancia de cada hilo. El planificador **puede tenerla en cuenta** para decidir qué hilo se ejecuta cuando hay más hilos listos que núcleos libres, pero no está obligado a hacerlo.



Niveles de prioridad en Java

Prioridad mínima

Thread.MIN_PRIORITY (1) representa la prioridad más baja.

Prioridad normal

Thread.NORM_PRIORITY (5) es la prioridad por defecto.

Prioridad máxima

Thread.MAX_PRIORITY (10) representa la prioridad más alta.

El hilo principal (**main**) arranca con **prioridad 5**. Cada hilo nuevo **hereda la prioridad del hilo que lo crea**: si un hilo de prioridad 8 crea otro, el nuevo nace con 8.



Establecimiento y modificación de prioridades

Consulta de prioridad

Puedes obtener la prioridad actual de un hilo utilizando el método: **getPriority()**.

Establecer nueva prioridad

Puedes establecer una nueva prioridad para un hilo usando el método: **setPriority(int priority)**, con valores entre 1 y 10.

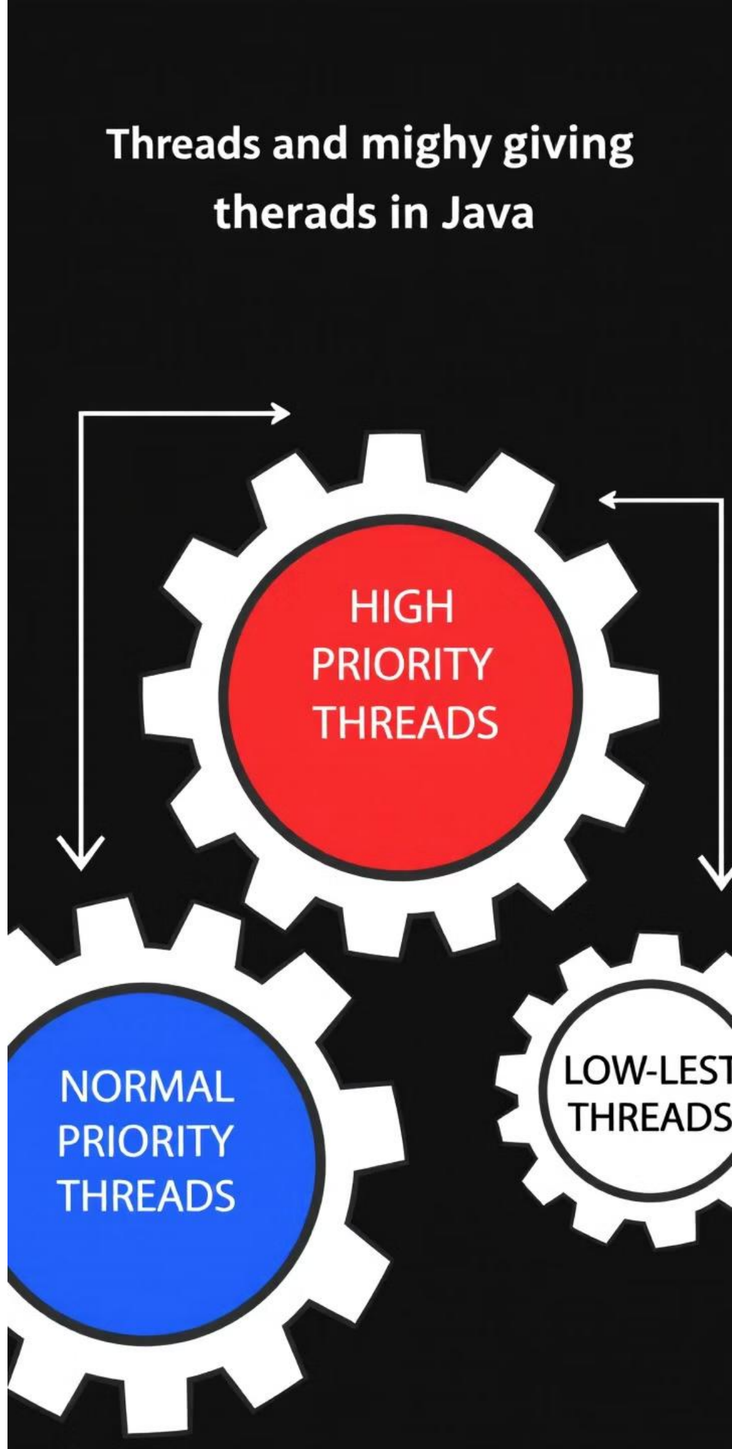
Con un valor fuera de ese rango (0 u 11, por ejemplo), lanza IllegalArgumentException.

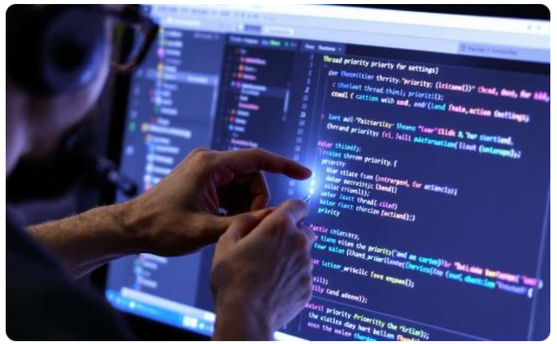
Ejemplo básico

```
public class PrioridadHilos {
    public static void main(String[] args) {
        Thread hilo1 = new Thread(() -> System.out.println("Hilo 1 ejecutándose"));
        Thread hilo2 = new Thread(() -> System.out.println("Hilo 2 ejecutándose"));

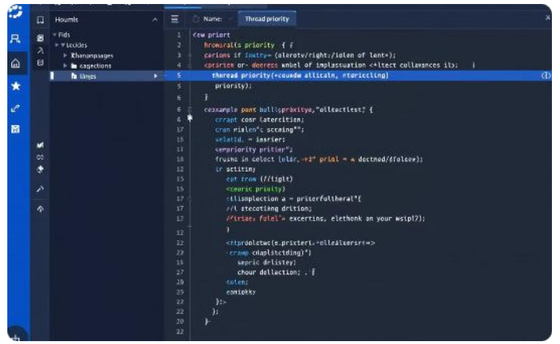
        // Establecer prioridades
        hilo1.setPriority(Thread.MIN_PRIORITY); // Prioridad baja
        hilo2.setPriority(Thread.MAX_PRIORITY); // Prioridad alta

        // Iniciar los hilos
        hilo1.start();
        hilo2.start();
    }
}
```









Efectos de las prioridades

Prioridades no son estrictas

Los hilos con mayor prioridad tienen más probabilidad de ejecutarse antes, **pero el sistema operativo puede ignorar estas prioridades** dependiendo de su planificador.

La prioridad solo se nota cuando la **CPU está saturada**, es decir, cuando hay más hilos listos que núcleos. Si sobran núcleos, cada hilo tiene el suyo y la prioridad no influye.

Ejecución no garantizada

En un programa con hilos de diferentes prioridades, los de mayor prioridad **tienden a completar sus tareas antes**, aunque no siempre de manera estricta.

Aunque la CPU esté saturada, la prioridad solo hace más probable que un hilo reciba más tiempo de CPU. **Nunca garantiza un orden.**

Ejemplo práctico

Lanzamos el doble de hilos que núcleos, la mitad con prioridad 10 y la otra mitad con prioridad 1, y los ponemos a trabajar 3 segundos. ¿Quién ha trabajado más?

```
public class CarreraPrioridades {
    public static void main(String[] args) throws InterruptedException {
        int nucleos = Runtime.getRuntime().availableProcessors();
        int numHilos = nucleos * 2;             // más hilos que núcleos: CPU saturada
        long[] vueltas = new long[numHilos];
        Thread[] hilos = new Thread[numHilos];

        for (int i = 0; i < numHilos; i++) {
            int id = i;
            hilos[i] = new Thread(() -> {
                long fin = System.currentTimeMillis() + 3000;   // trabaja 3 segundos
                long contador = 0;
                while (System.currentTimeMillis() < fin) {
                    contador++;
                }
                vueltas[id] = contador;
            });
            // la mitad con prioridad máxima y la otra mitad con mínima
            hilos[i].setPriority(i % 2 == 0 ? Thread.MAX_PRIORITY : Thread.MIN_PRIORITY);
        }

        for (Thread h : hilos) h.start();
        for (Thread h : hilos) h.join();

        long alta = 0, baja = 0;
        for (int i = 0; i < numHilos; i++) {
            if (i % 2 == 0) alta += vueltas[i]; else baja += vueltas[i];
        }
        System.out.println("Núcleos: " + nucleos + "  ·  Hilos: " + numHilos);
        System.out.printf("Trabajo de los hilos con prioridad 10: %,d vueltas%n", alta);
        System.out.printf("Trabajo de los hilos con prioridad 1 : %,d vueltas%n", baja);
    }
}
```



Buenas prácticas en el uso de prioridades



Evitar dependencias estrictas

Las prioridades no garantizan el orden de ejecución, por lo que los programas deben funcionar correctamente independientemente del orden de los hilos.



Usar prioridades solo cuando sea necesario

Subir un poco la prioridad de lo que el usuario nota (la interfaz, la reproducción de audio) y bajar la de lo que puede esperar (copias de seguridad, indexado). Pero nunca contar con ello para cumplir un plazo: Java estándar no es de tiempo real.



Considerar la portabilidad

Cada sistema operativo traduce las prioridades de Java a su manera: Windows agrupa los 10 niveles en 7 y Linux, por defecto, las ignora. El mismo programa se comporta distinto según dónde se ejecute. Usar las constantes (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY) hace el código más legible, pero no lo hace portable.

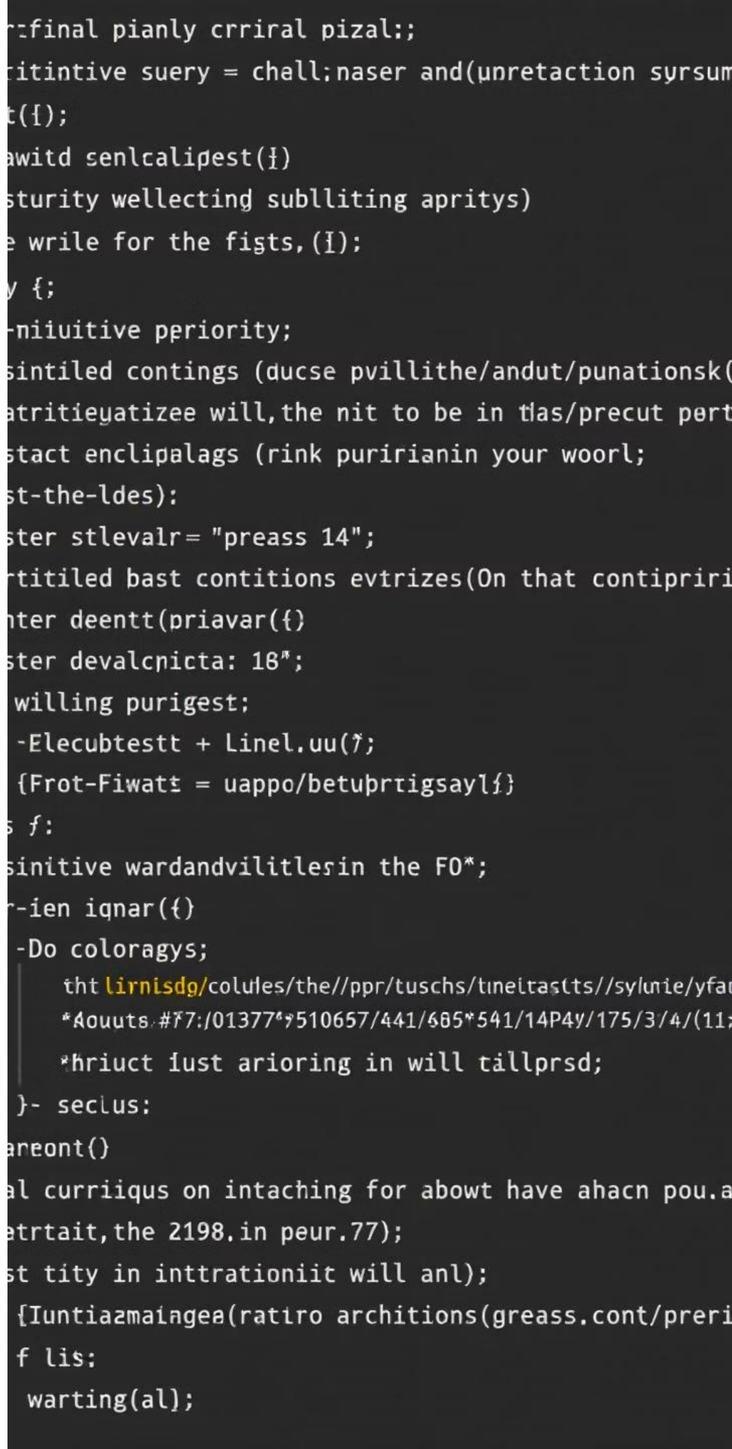


Evitar problemas de inanición

Ajustar las prioridades de manera razonable para evitar que hilos de prioridad baja queden sin ejecutar.
Si un hilo no consigue un recurso, la solución no es subirle la prioridad, sino usar mecanismos justos (5.4.3).

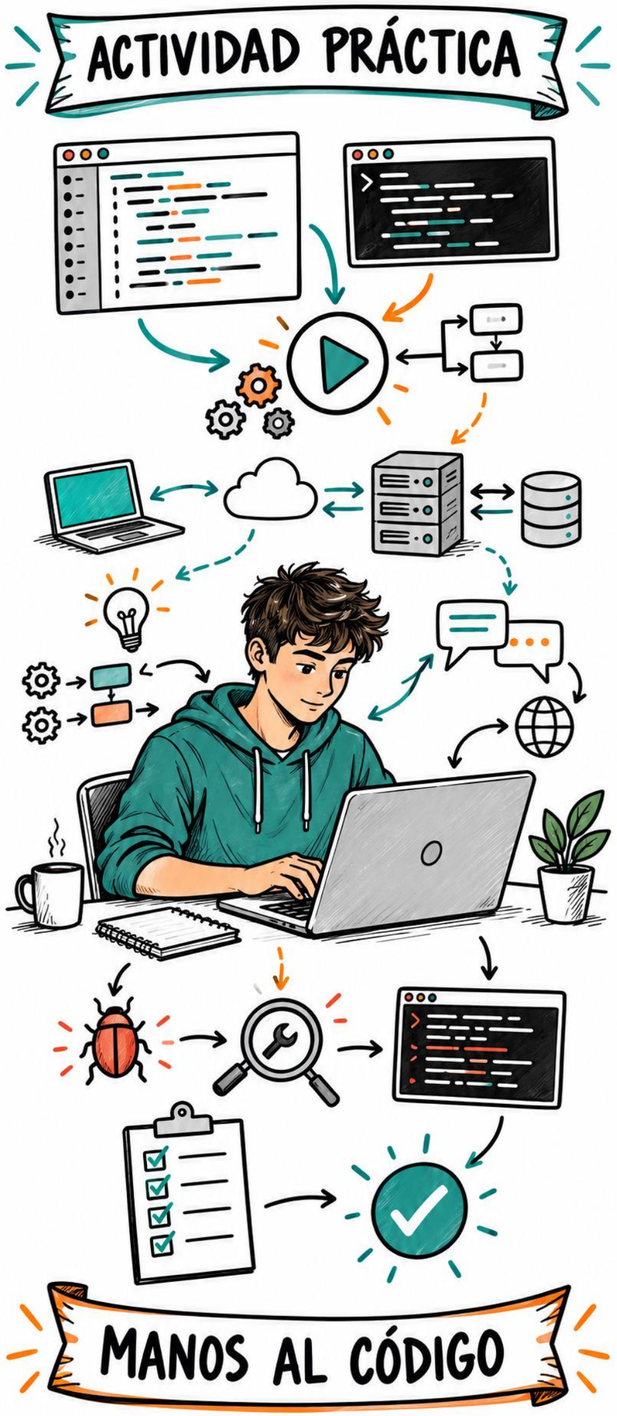
Conclusión prioridades

- La prioridad es una **sugerencia** al sistema operativo: solo se nota con la CPU saturada, y cada sistema la aplica a su manera.
- Un programa correcto **nunca depende de las prioridades**. Para controlar el orden se usan join() y la sincronización; para un reparto justo, los mecanismos justos.
- Úsalas con moderación, para dar preferencia a lo que el usuario nota, y deja el resto en NORM_PRIORITY.

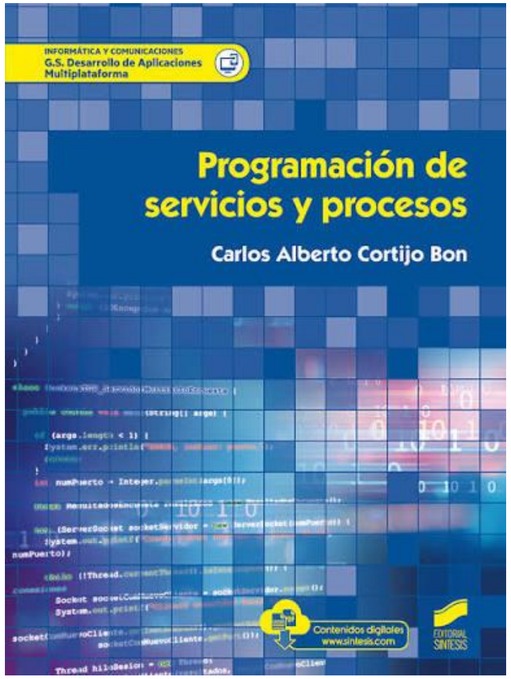


ACTIVIDAD 7

Prioridades de los hilos

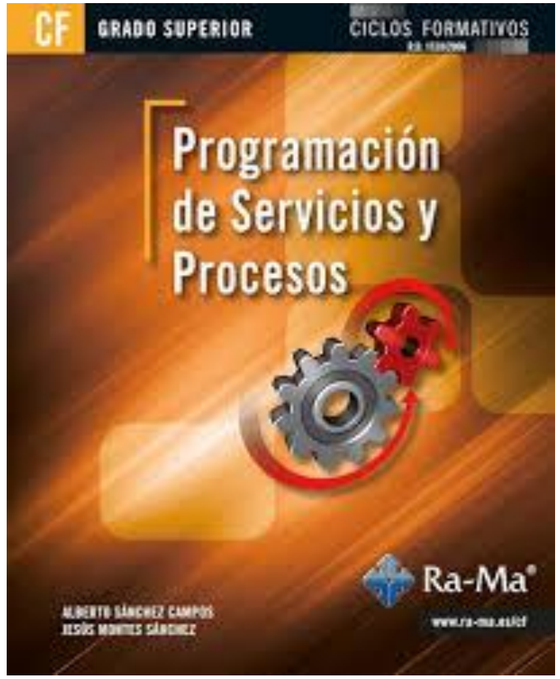


Bibliografía



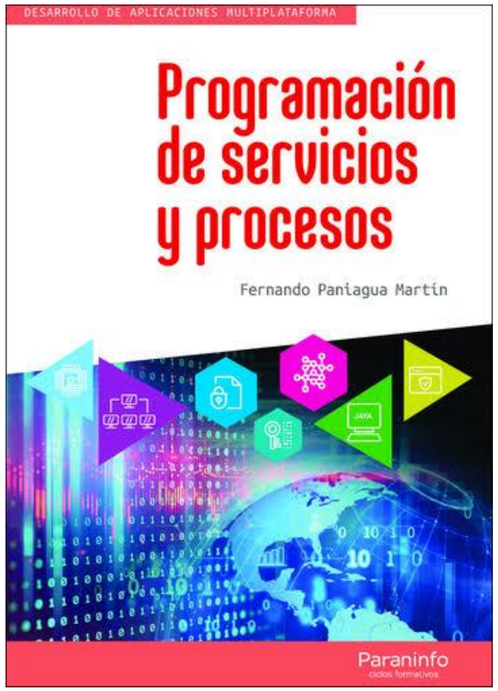
Cortijo Bon, Carlos A. (2021)

”Programación de servicios y procesos”. En editorial Síntesis (España). ISBN: 978-84-1357-104-1.



Córcoles Tendero, Jose Eduardo. (2014)

”Programación de servicios y procesos”. En editorial RA-MA (España). ISBN: 978-84-9964-391-5



Paniagua Martín, Fernando. (2022)

“Programación de servicios y procesos”. En editorial Paraninfo. ISBN 978-84-1366-526-9.